

发电负荷融合分析平台

数据分析报告

版本：V1.1

数据分析课题组

文档修订记录

版本	*变化状态	简要说明（变更内容和变更范围）	日期	作者	参与者
V1.0.0	A	新建	2026-5-28	诸葛瑞乐、孙志瑞、姜林	诸葛瑞乐、孙志瑞、姜林
V1.1.0	A	完善	2026-6-9	诸葛瑞乐、孙志瑞、姜林	诸葛瑞乐、孙志瑞、姜林

*变化状态：A——增加，M——修改，D——删除，N——正式发布

文档审阅信息

版本	审阅人	角色	审阅日期	签字	备注
V1.0.0	诸葛瑞乐	项目负责人	2026.5.29	诸葛瑞乐	无
V1.1.0	诸葛瑞乐	项目负责人	2026.6.9	诸葛瑞乐	无

文档批准信息

版本	批准人	角色	批准日期	签字	备注
V1.0.0	诸葛瑞乐	项目负责人	2026.5.29	诸葛瑞乐	无
V1.1.0	诸葛瑞乐	项目负责人	2026.6.9	诸葛瑞乐	无

目录

第 1 章 数据清洗与预处理.....	1
1.1 数据集重构与时序对齐.....	1
1.1.2 核心处理策略：.....	1
1.2 异常值与缺失值处理.....	1
1.2.2 基于 3σ 法则的异常值剔除.....	1
1.3 约束性线性插值与双向填充.....	2
1.4 规范化（无量纲）处理.....	2
1.4.2 具体实现方案：.....	2
1.5 数据集划分与防数据泄露.....	3
1.5.2 划分比例与逻辑：.....	3
第 2 章 模型构建.....	5
2.1 特征选取机制.....	5
2.2 防过拟合策略.....	5
2.3 最佳训练轮次的选取.....	5
2.4 回归模型（基准模型：随机森林）.....	5
2.4.1 原理解析.....	5
2.4.2 模型优势与应用.....	6
2.4.3 各预测时间档位模型训练评估图谱.....	6
2.4.4 源代码实现：.....	10
2.5 循环神经网络（RNN）.....	15
2.5.1 原理解析.....	15
2.5.2 模型优势与局限.....	15
2.5.3 各预测时间档位模型训练评估图谱.....	16
2.5.4 源代码：.....	20
2.6 长短期记忆网络（LSTM）.....	29
2.6.1 原理解析.....	29
2.6.2 模型优势与应用.....	29
2.6.3 各预测时间档位模型训练评估图谱.....	29
2.6.4 源代码：.....	33
2.7 卷积与长短期记忆混合网络（CNN-LSTM）.....	42
2.7.1 原理解析.....	42
2.7.2 模型优势与应用.....	42
2.7.3 各预测时间档位模型训练评估图谱.....	43
2.7.4 源代码：.....	47
2.8 极限梯度提升树（XGBoost）.....	56
2.8.1 原理解析.....	56
2.8.2 模型优势与应用.....	56
2.8.3 各预测时间档位模型训练评估图谱.....	57
2.8.4 各预测时间档位模型训练评估图谱（AGC）：.....	61
2.8.5 源代码：.....	65
2.9 基于自注意力机制的 Transformer.....	72
2.9.1 原理解析.....	72

2.9.2 模型优势与应用	72
2.9.3 各预测时间档位模型训练评估图谱	72
2.9.4 源代码:	76
2.10 多维度并发预测 (多输出 Transformer)	87
2.10.1 原理解析与架构痛点	87
2.10.2 多输出重构与优势	87
2.10.3 各预测时间档位模型训练评估图谱	87
2.10.4 源代码:	91
第 3 章 模型预测结果与性能分析	103
3.1 核心基准预测性能对比 (提前 15 分钟)	103
3.1.2 对比分析:	103
3.2 训练动态与过拟合控制分析	105
3.3 物理规律的可解释性挖掘 (XGBoost)	106
3.4 多挡位时间跨度预测与衰减分析	108
3.4.2 衰减效应分析:	108
第 4 章 结论与工程落地展望	111
4.1 核心结论: 算法选型与性能总结	111
4.2 最终模型确立: 面向数字孪生的“混合路由”双引擎架构	111
4.3 宏观趋势渲染引擎	111
4.4 微观精准路由引擎	112

第1章 数据清洗与预处理

在构建深度学习与机器学习预测模型之前，高质量的数据底座是保证模型预测精度的核心前提。本研究的原始数据来源于真实工业场景，存在多源异构、采样频率不一以及传感器噪声等问题。因此，本节首先对数据进行了严格的清洗与重构。

1.1 数据集重构与时序对齐

本研究的原始数据集由分布在不同时间段或不同传感器节点的大量独立文件（共计 369 个 CSV/Excel 文件）组成。为了满足后续多变量时间序列预测（Multivariate Time Series Forecasting）的任务需求，我们首先对数据进行了物理特征维度的横向拼接与时间维度的纵向对齐。

1.1.2 核心处理策略：

多源合并： 以核心预测目标（实际负荷 Actual_Load）为基准坐标，将 300 余个独立文件中的传感器特征进行自动化循环读取与横向合并，构建出一张高维特征大表。

重采样平滑（Resampling）： 由于真实工业系统中各个传感器的采样频率存在微小偏差，本研究采用 1 分钟（1-minute）为步长对全量数据进行了重采样（Resampling）对齐。在重采样过程中采用了取均值（Mean）的聚合策略。这一操作不仅完美对齐了时间戳，还起到了类似低通滤波器的作用，有效平滑了工业设备运转时产生的瞬时高频毛刺，为后续模型提取长期物理趋势提供了更稳定的数据源。

1.2 异常值与缺失值处理

在工业控制系统中，受限于传感器故障或网络通信波动，数据中不可避免地会出现缺失值（NaN）与不符合物理规律的极端异常值。我们采用了一套“先剔除、后填补”的严谨流水线进行处理。

1.2.2 基于 3σ 法则的异常值剔除

对于突发的极端噪声，本研究引入了统计学中经典的 3σ 法则（Pauta Criterion）。针对除时间戳以外的所有数值型物理特征列，程序自动计算其全局均值（Mean）与标准差（Standard Deviation）。我们将偏离均值超过 3 个标准差的数值（即在正态分布下发生概率小于 0.3% 的小概率极端事件）判定为传感器误报或物理设备瞬间异常，并利用向量化操作（`np.where`）精准地将其替换为空值（NaN），等待下一步的统一填补。

1.3 约束性线性插值与双向填充

在完成异常值剥离后，针对表格中存在的原始缺失值以及上一步新产生的 NaN 空洞，本研究设计了多级填补策略：

1. 限幅线性插值（Interpolate）：作为首选策略，采用线性插值法对时间序列上的空洞进行平滑填补。为了防止在设备长时间停机断联时产生严重失真的虚假直线，我们严格设置了 `limit=5` 的约束条件。即只有当连续缺失的数据点不超过 5 分钟时，才允许进行线性推导。
2. 前后向兜底填充（Forward/Backward Fill）：对于超过限幅阈值的长时间数据缺失，或者发生在时间序列最开头的空值，本研究采用就近的真实有效值进行前向（`ffill`）与后向（`bfill`）平移填充，确保最终输入给算法引擎的数据矩阵 100% 完整，消除了算法训练时因 NaN 导致的程序崩溃风险。

1.4 规范化（无量纲）处理

在工业物联网（IIoT）场景中，不同传感器采集到的物理量存在极大的量纲差异。例如，机组的总风量可能高达十几万（ m^3/h ），而汽包压力往往只有十几（MPa）。如果直接将这些带着巨大数值差异的原始数据喂给深度学习网络（如 LSTM、Transformer 等基于梯度下降优化的模型），会导致模型在权重更新时发生严重的梯度偏移，小数值的核心物理特征会被大数值特征彻底淹没。

为了消除这种量纲带来的负面影响，本研究在特征筛选完成后、时序截取之前，对所有输入特征（X）和预测目标（y）进行了严格的最小-最大归一化（Min-Max Scaling）处理。

1.4.2 具体实现方案：

利用 `scikit-learn` 库中的 `MinMaxScaler`，将所有变量的数据分布线性压缩至

[0, 1] 的标准区间内。

通过规范化处理，所有物理特征在进入算法引擎前被拉回到了同一条起跑线上，这极大地提升了神经网络的收敛速度，并保证了后续多维度特征交互时的权重公平性。(注：在最终输出评估结果时，预测值会通过 `inverse_transform` 反向还原为带有 MW 单位的真实负荷量。)

1.5 数据集划分与防数据泄露

对于传统截面数据（如图像分类、用户画像），通常采用随机打乱抽样（Random Split）的方式来划分数据集。但本研究面对的是具有极强时间依赖性的工业时序数据，如果采用随机划分，会严重破坏数据的时间连续性，甚至导致“利用未来数据预测过去”的数据泄露（Data Leakage / Look-ahead Bias）灾难。

因此，本研究严格采用了基于时间轴的顺序划分策略（Sequential Splitting）。

1.5.2 划分比例与逻辑：

- **训练集（Training Set）：80%**。截取时间轴前 80% 的连续数据，作为模型历史经验的“学习沙盘”，供算法引擎反复迭代寻找风量、压力与负荷之间的映射规律。
- **测试/验证集（Test/Validation Set）：20%**。严格保留时间轴最后 20% 的全新连续数据，将其视为模型从未见过的“未来”，专门用于检验（期末考试）模型的真实泛化能力。

这种严苛的划分方式，最大程度还原了真实工业界“基于已知过去，预测未知未来”的部署环境，确保了本研究最终得出的 RMSE 和 R^2 评估指标具备极高的工程可信度。

第2章 模型构建

为了确保不同架构的算法能够在公平的基准线上进行对比，本研究在模型训练前，统一制定了严格的特征工程与训练监控标准。

2.1 特征选取机制

本研究采用了“统计学+物理逻辑”的双重特征选取法。首先，通过计算皮尔逊相关系数（Pearson Correlation Coefficient），筛选出与目标变量相关性大于 0.6 的候选特征。随后，引入人工领域知识构建“作弊特征黑名单”，强制剔除所有包含“功率”、“指令”、“定值”等会引起数据泄露（Data Leakage）的控制系统衍生变量，以及高度冗余的多胞胎传感器点位。最终提取出真正反映物理工况的核心独立特征输入模型。

2.2 防过拟合策略

对于具备强大拟合能力的迭代类模型（如深度学习网络和极限梯度提升树），盲目增加训练轮次极易导致模型陷入死记硬背训练集噪音的“过拟合（Overfitting）”陷阱。为此，本研究在所有的迭代模型中引入了早停机制（Early Stopping），将模型在未见过数据的测试集上的表现作为唯一评估标准，实时监控泛化能力的衰减。

2.3 最佳训练轮次的选取

在具体实现中，我们将模型的理论最大训练轮次（Epochs/Trees）设定为一个极大值（500 轮），并设置容忍度（Patience）为 20。在每一轮训练结束后，计算验证集误差（Test Loss）。若连续 20 轮验证集误差未能下降，系统将自动触发中断，并回滚提取出历史误差最低那一轮的模型权重作为最终的“最佳模型”。该机制确保了模型始终处于欠拟合与过拟合之间的极值平衡点。

2.4 回归模型（基准模型：随机森林）

2.4.1 原理解析

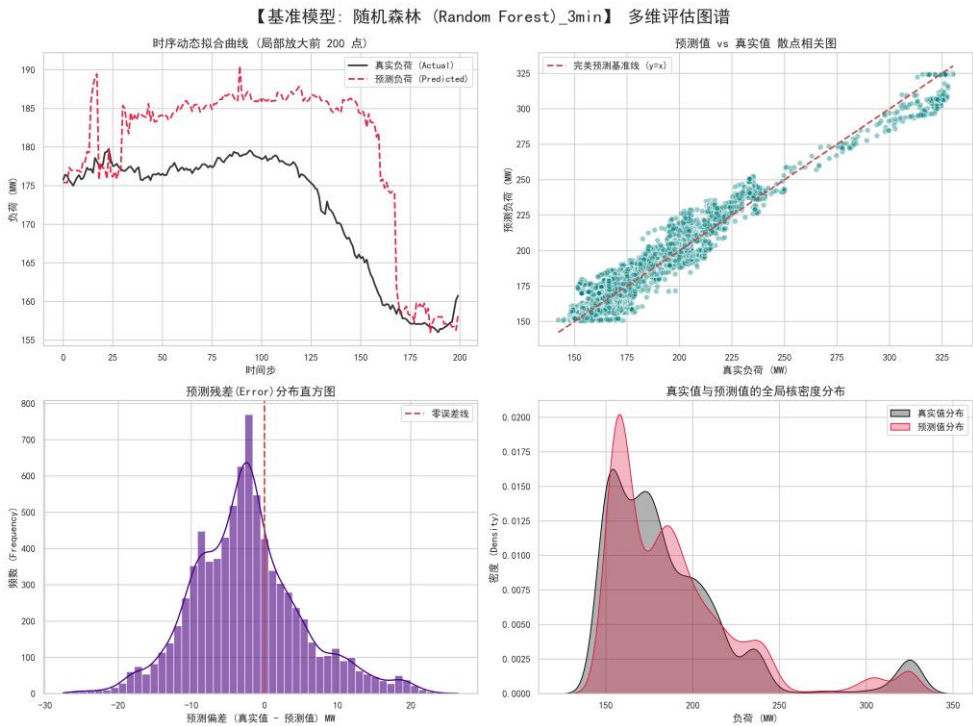
本研究选取随机森林回归（Random Forest Regressor）作为整个多步时间序列预测任务的基准模型（Baseline）。随机森林是一种基于 Bagging 思想的经典集成学习算法。它通过自助采样法（Bootstrap）从原始数据中构建出多个不同的数据子集，并独立训练出多棵决策树（Decision Tree）。在预测时，模型综合所有决策树的输出结果取平均值。

2.4.2 模型优势与应用

由于采用了群体决策机制，随机森林天生对工业数据中的异常值和局部噪声具有极强的鲁棒性（Robustness）。此外，树模型的节点分裂依赖于特征的相对大小而非绝对数值，因此它无需进行复杂的无量纲规范化处理。在本实验中，我们将树的数量设定为 50，最大深度限制为 10，以此控制单棵树的复杂度，使其作为一个极具参考价值的轻量级基准线。

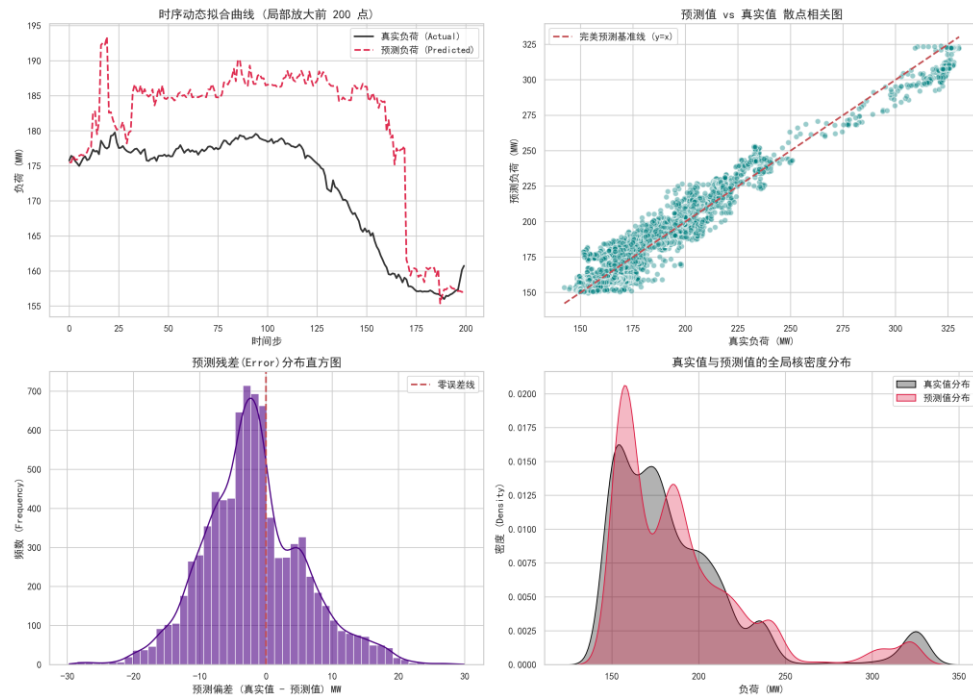
2.4.3 各预测时间档位模型训练评估图谱

2.4.3.1 预测 3min 之后的模型训练评估：



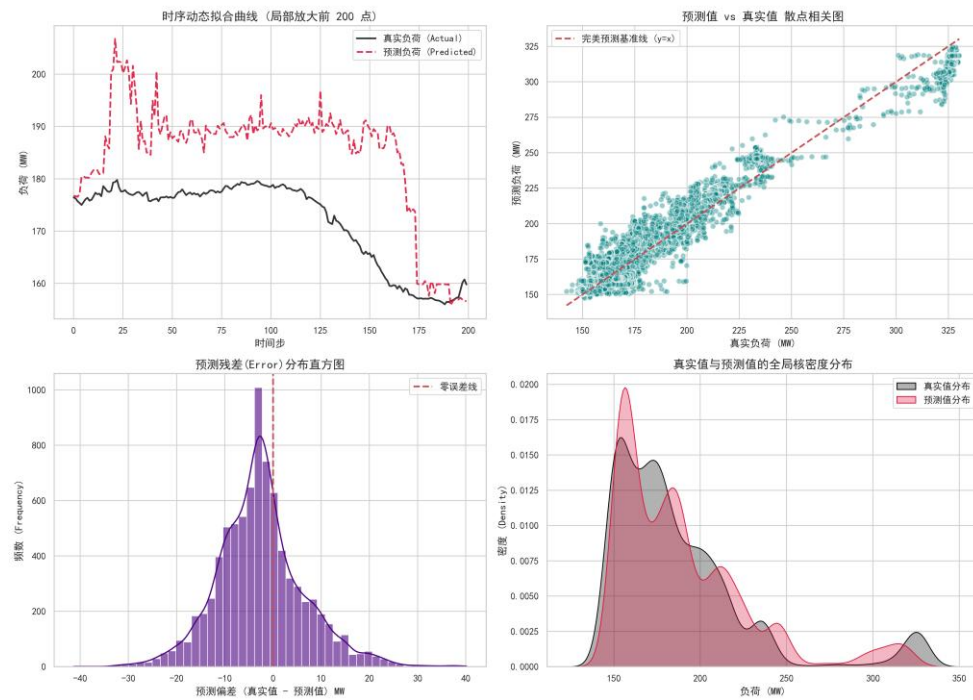
2.4.3.2 预测 5min 之后的模型训练评估：

【基准模型：随机森林 (Random Forest)_5min】 多维评估图谱



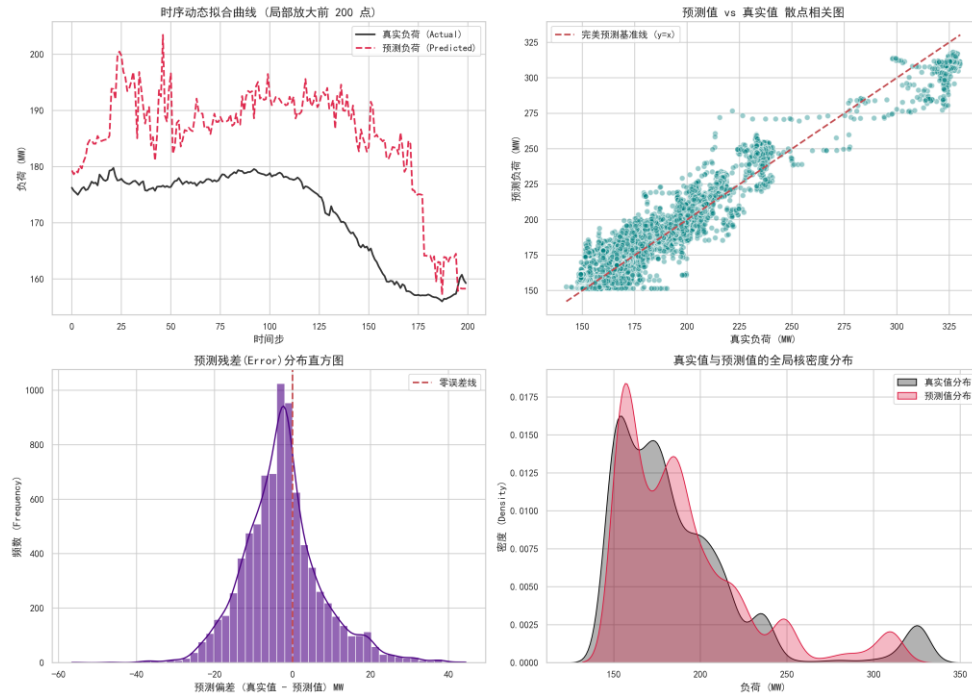
2.4.3.3 预测 10min 之后的模型训练评估:

【基准模型：随机森林 (Random Forest)_10min】 多维评估图谱



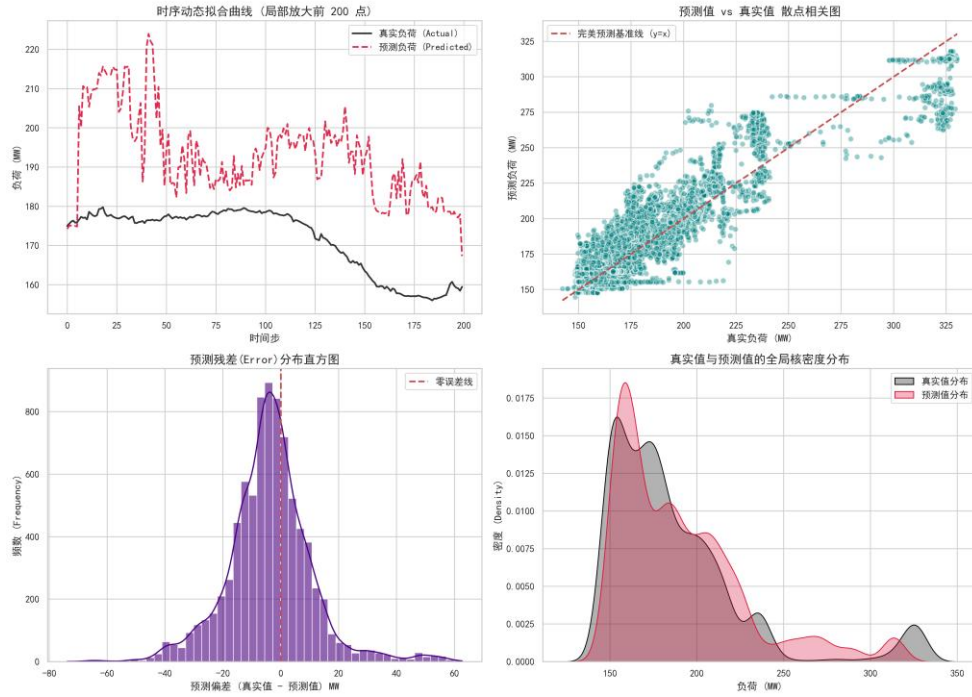
2.4.3.4 预测 15min 之后的模型训练评估:

【基准模型：随机森林 (Random Forest)_15min】 多维评估图谱



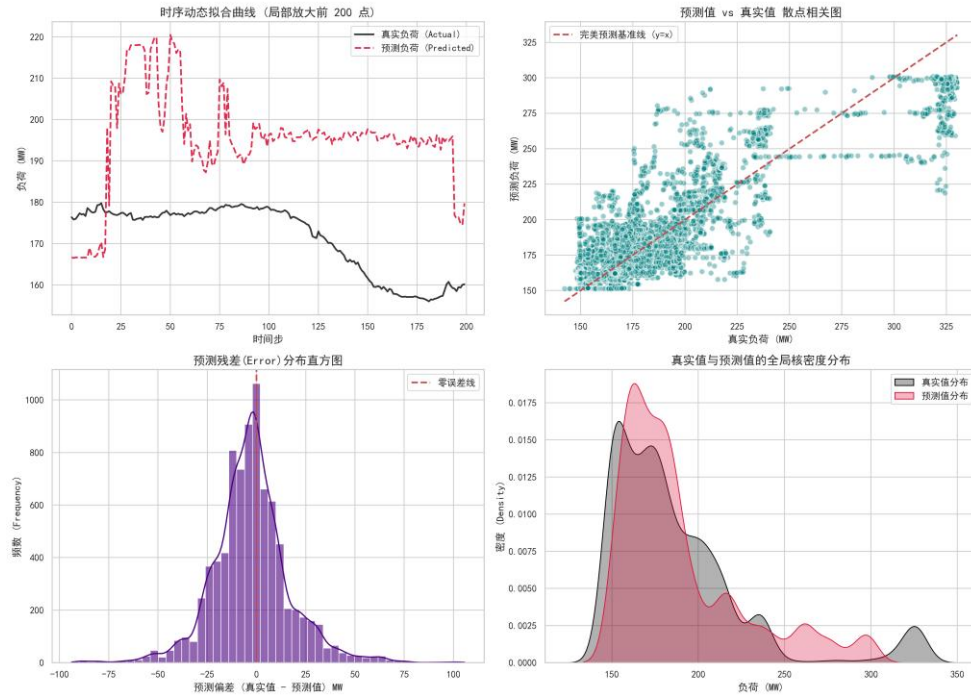
2.4.3.5 预测 30min 之后的模型训练评估:

【基准模型：随机森林 (Random Forest)_30min】 多维评估图谱



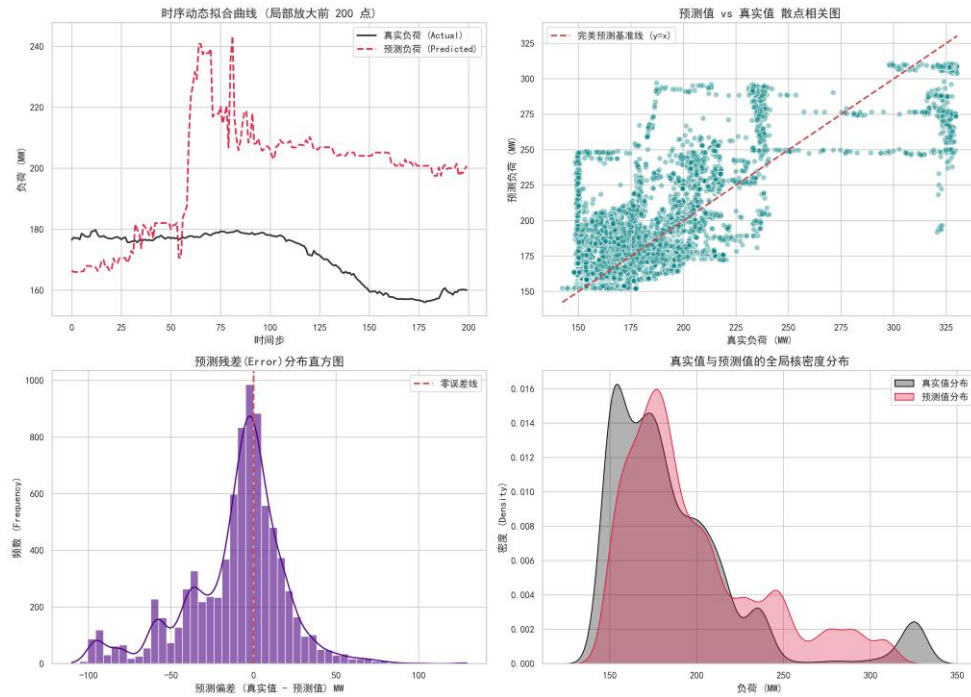
2.4.3.6 预测 45min 之后的模型训练评估:

【基准模型：随机森林 (Random Forest)_45min】 多维评估图谱

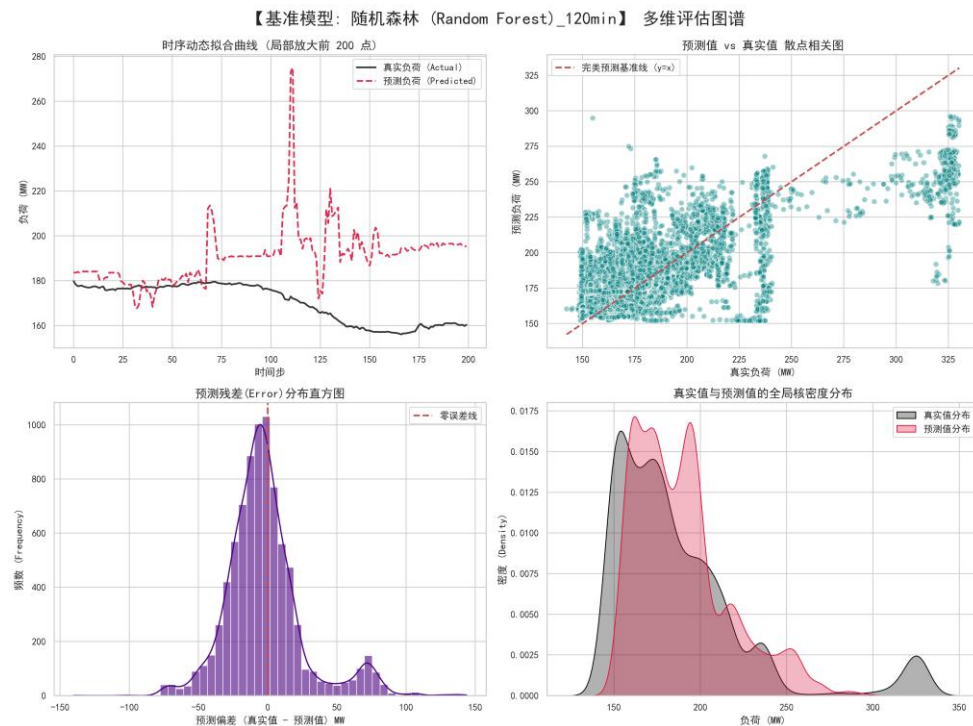


2.4.3.7 预测 60min 之后的模型训练评估:

【基准模型：随机森林 (Random Forest)_60min】 多维评估图谱



2.4.3.8 预测 120min 之后的模型训练评估:



2.4.4 源代码实现：

Python

```
import pandas as pd

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import numpy as np

import joblib # 【新增】专门用于高效保存大型机器学习模型的工业级库

input_file = 'power_plant_cleaned_data.csv'

print("正在加载数据并准备算法引擎...")

df_original = pd.read_csv(input_file, index_col=0, parse_dates=True)

sign = True

# =====

# 1: 时间平移, 构建“预测未来”的真实场景
```

```

# =====

horizons = [ 3, 5, 10, 15, 30, 45, 60, 120]

for predict_minutes in horizons:

    # 初始化 df

    df = df_original.copy()

    print(f"\n 正在构建时间序列... 模型将尝试提前 {predict_minutes} 分钟
预测负荷。")

    target_col = 'Actual_Load'

    future_target_col = f'Future_Load_{predict_minutes}m'

    df[future_target_col] = df[target_col].shift(-predict_minutes)

    df.dropna(subset=[future_target_col], inplace=True)

# =====

#2: 更严的特征过滤

# =====

correlations = df.corr()[future_target_col].abs()

selected_features = correlations[correlations > 0.6].index.tolist()

cheat_keywords = [

    # 1. 答案泄露与物理代理 (防止模型作弊抄答案)

    '功率', '负荷', 'AGC', '指令', '主蒸汽流量', 'Actual_Load',

    # 2. 控制系统内部衍生变量 (防止引入计算公式导致的过拟合)

    '校正后', '定值', '偏置', '前馈量', '风量比值',

    # 3. 空间与物理分支 (逼迫模型去看"总和"特征, 解决共线性)

```

```

# 注意：这里保留了煤粉和给煤机，因为大概率没有"总给煤量"
'侧',

# 4. 极端冗余的多胞胎传感器 (只保留均值或主管道)
# 针对汽包压力、主蒸汽压力、再热器等重复点位进行精准击杀
'压力 1', '压力 2', '压力 3', '压力 237',
'#1', '#2',
'A 压力', 'B 压力',
'248', '249', '250',
'253', '254'
]

clean_features = []

for col in selected_features:

    is_cheat = False

    for keyword in cheat_keywords:

        if keyword in col:

            is_cheat = True

            break

    if not is_cheat and col != future_target_col:

        clean_features.append(col)

print(f'剔除高度耦合的物理代理后，剩余真实工况特征
{len(clean_features)} 个。")

#
=====

# 输出当前选择的特征及其相关系数

```

#

=====

```
if(sign):  
    print("\n--- 最终保留的特征及其相关系数 (Threshold > 0.6) ---")  
    for feature in clean_features:  
        coef = correlations[feature]  
        print(f"[{coef:.4f}] - {feature}")  
    print("-----\n")  
    sign = False
```

#

=====

后续流程：划分数据集与训练

#

=====

```
train_ratio = 0.8  
split_index = int(len(df) * train_ratio)  
  
train_data = df.iloc[:split_index]  
test_data = df.iloc[split_index:]  
  
X_train = train_data[clean_features]  
y_train = train_data[future_target_col]  
X_test = test_data[clean_features]  
y_test = test_data[future_target_col]  
  
print(f"\n 正在训练 {predict_minutes} 分钟未来预测模型 (Random
```

```

Forest)...")

        model = RandomForestRegressor(n_estimators=50, max_depth=10,
random_state=42, n_jobs=-1)

        model.fit(X_train, y_train)


print("\n 训练完成！ 正在评估模型真正的预测能力...")

y_pred = model.predict(X_test)


mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)


print("-" * 30)

print(f'【提前 {predict_minutes} 分钟预测成绩单】')

print(f'平均绝对误差 (MAE): {mae:.2f} MW')

print(f'均方根误差 (RMSE): {rmse:.2f} MW')

print(f'真实预测准确度 (R²): {r2:.4f}')

print("-" * 30)


result_df = pd.DataFrame({

    'Real_Future_Load': y_test,

    'Predicted_Load': y_pred

}, index=y_test.index)

result_df.to_csv(f'rf_prediction_results_{predict_minutes}.csv')

```

#

=====

【新增】工程化封装：导出模型大脑与特征名单

#

```
print("\n 正在序列化模型并生成部署产物...")
```

```
# 1. 保存模型权重文件
```

```
model_filename = f'random_forest_model_{predict_minutes}.joblib'
```

```
joblib.dump(model, model_filename)
```

```
# 2. 保存模型所需的特征输入契约（极其重要）
```

```
features_filename = f'rf_features_{predict_minutes}.joblib'
```

```
joblib.dump(clean_features, features_filename)
```

```
print(f'✅ 模型结构与权重已成功导出至: {model_filename}')
```

```
print(f'✅ 输入特征契约名单已成功导出至: {features_filename}')
```

```
print("后续微服务可以直接加载这些文件进行毫秒级实时推理！")
```

2.5 循环神经网络 (RNN)

2.5.1 原理解析

循环神经网络（Recurrent Neural Network, RNN）是处理时间序列数据的传统深度学习基石。与常规前馈神经网络不同，RNN 在隐藏层中引入了“环状”的定向连接，使其具备了“记忆”功能。在处理当前时刻的传感器数据时，RNN 会融合上一时刻保留的隐藏状态（Hidden State），从而能够捕捉到时间序列前后的动态变化规律。

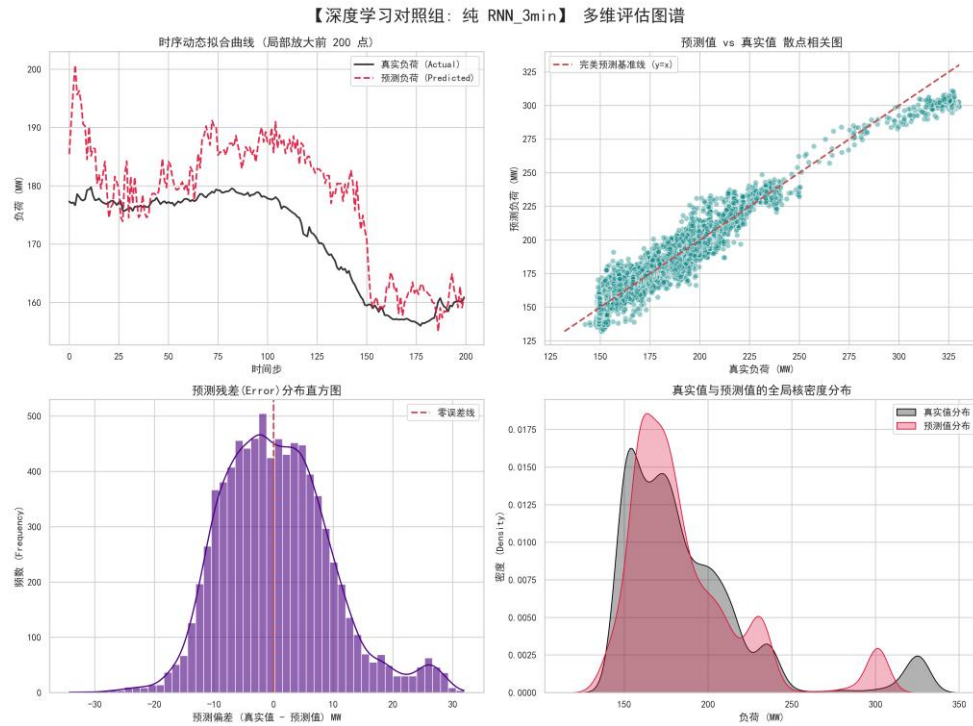
2.5.2 模型优势与局限

在本研究中，RNN 被引入作为深度学习序列模型的底层对照组。虽然它能有效处理短时依赖问题，但由于传统的 RNN 在反向传播时容易出现“梯度消失

（Vanishing Gradient）”问题，导致它在面对较长的时间窗口（如本实验中回顾过去的 30 分钟数据）时，往往会遗忘早期的关键物理状态。

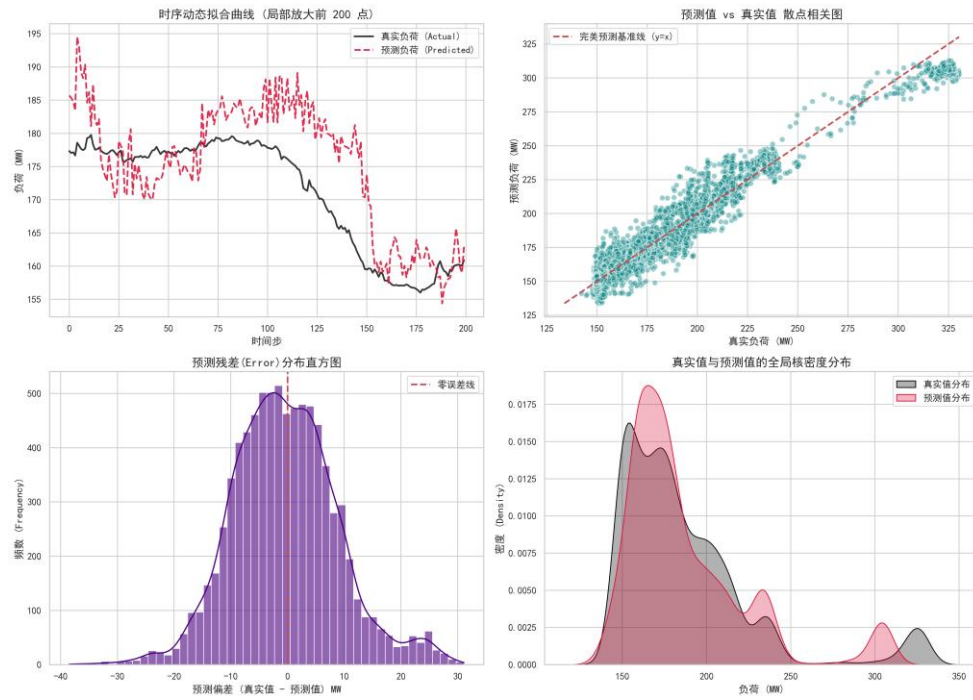
2.5.3 各预测时间档位模型训练评估图谱

2.5.3.1 预测 3min 之后的模型训练评估：



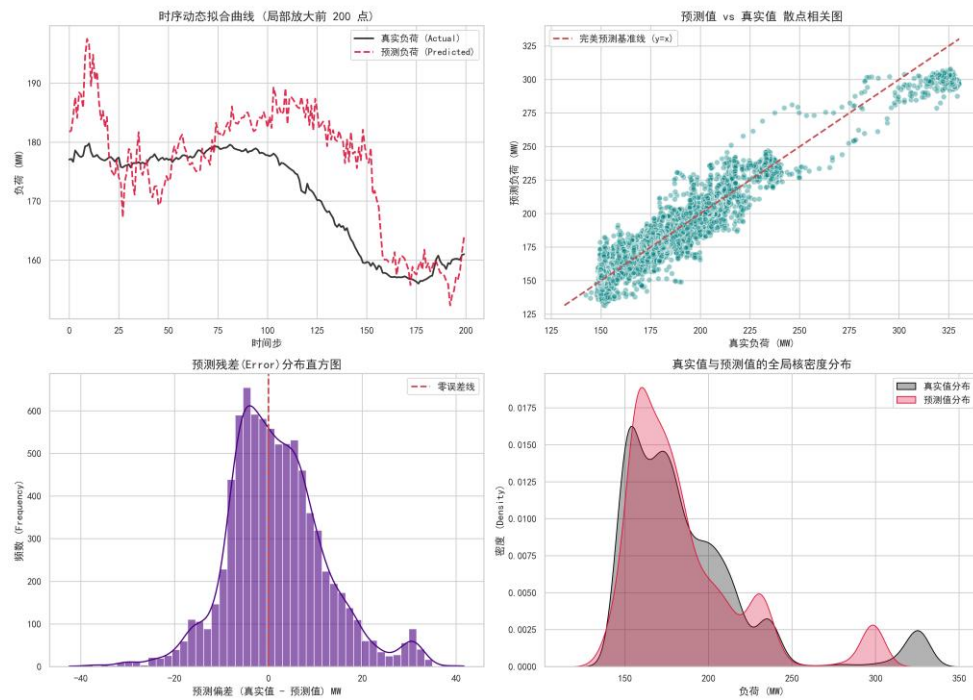
2.5.3.2 预测 5min 之后的模型训练评估：

【深度学习对照组：纯 RNN_5min】 多维评估图谱



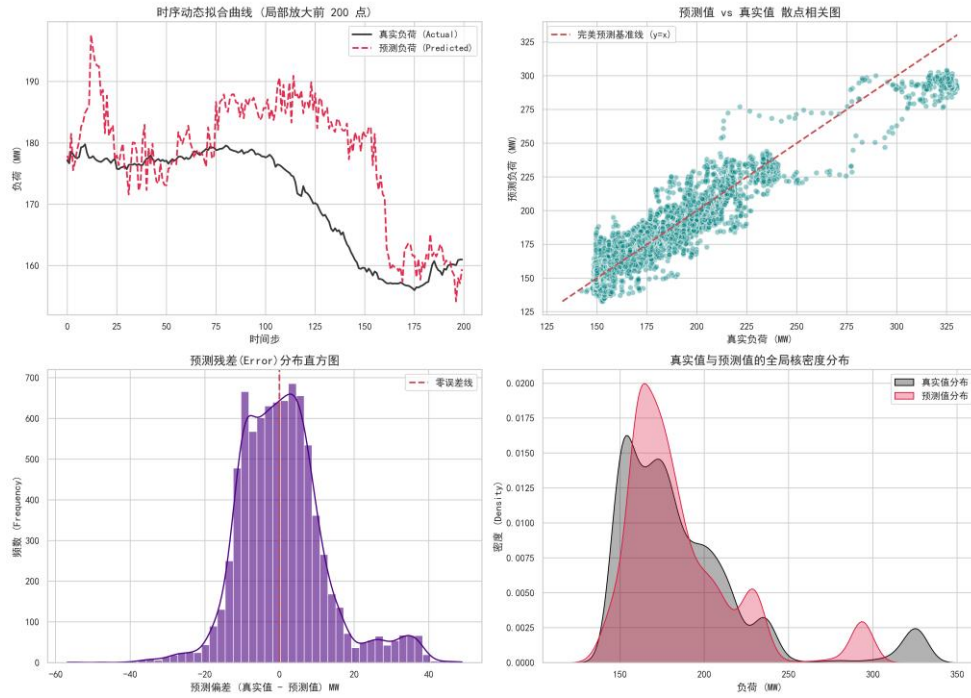
2.5.3.3 预测 10min 之后的模型训练评估:

【深度学习对照组：纯 RNN_10min】 多维评估图谱



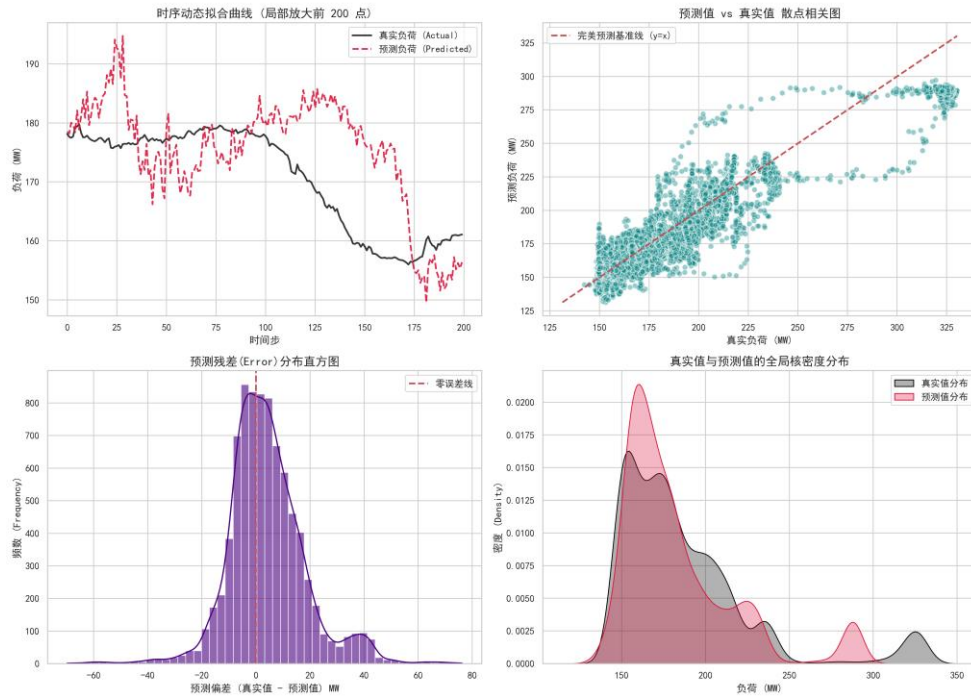
2.5.3.4 预测 15min 之后的模型训练评估:

【深度学习对照组：纯 RNN_15min】 多维评估图谱



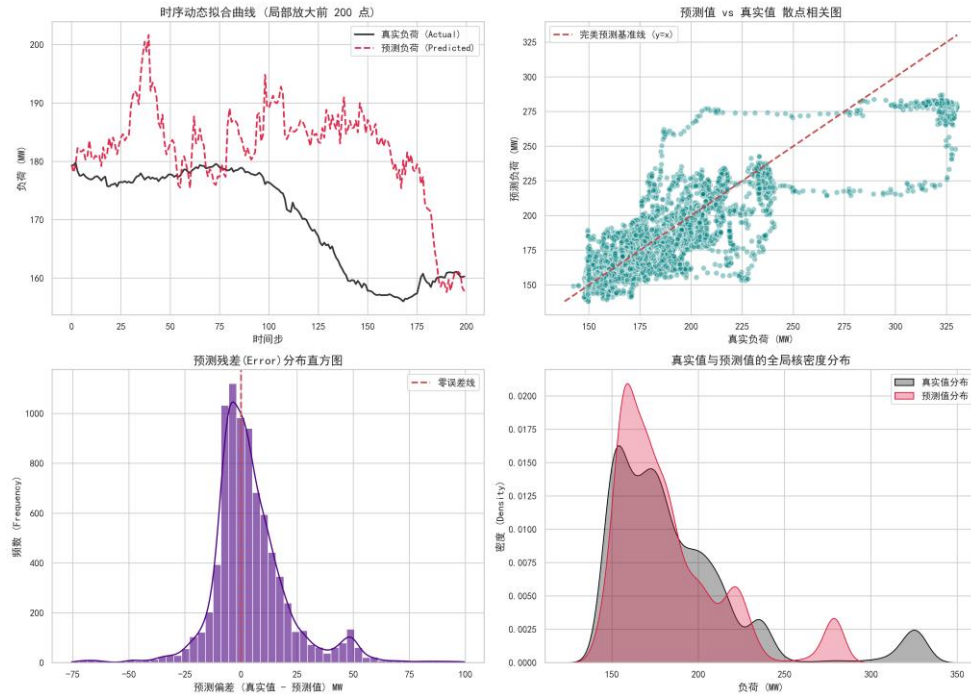
2.5.3.5 预测 30min 之后的模型训练评估:

【深度学习对照组：纯 RNN_30min】 多维评估图谱



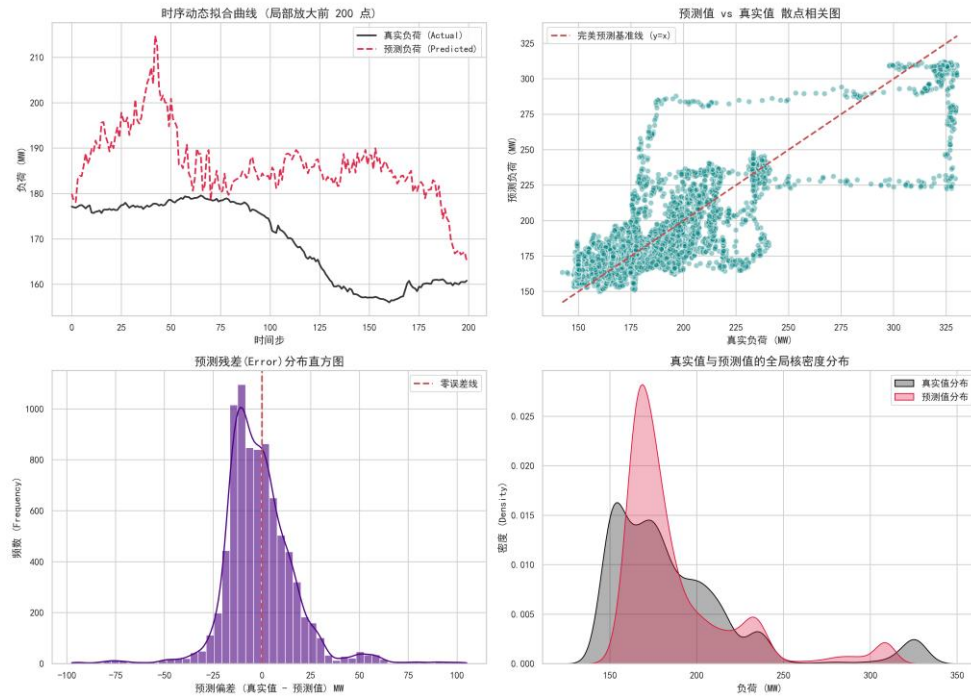
2.5.3.6 预测 45min 之后的模型训练评估:

【深度学习对照组：纯 RNN_45min】 多维评估图谱



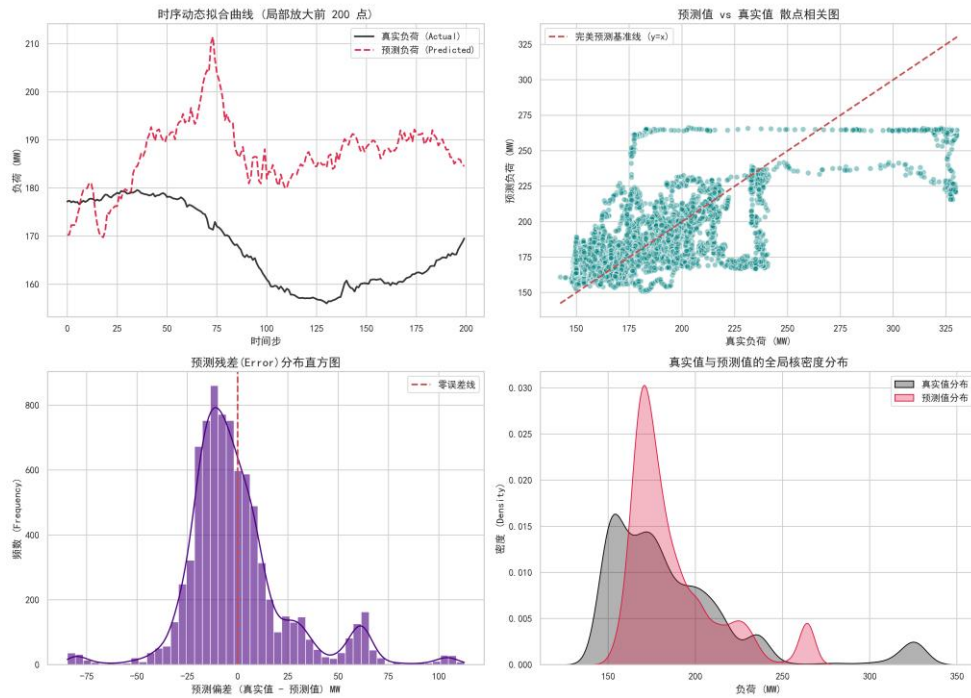
2.5.3.7 预测 60min 之后的模型训练评估:

【深度学习对照组：纯 RNN_60min】 多维评估图谱



2.5.3.8 预测 120min 之后的模型训练评估:

【深度学习对照组：纯 RNN_120min】 多维评估图谱



2.5.4 源代码:

```
import pandas as pd

import numpy as np

import torch

import matplotlib.pyplot as plt

import torch.nn as nn

from torch.utils.data import DataLoader, TensorDataset

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


# =====

# 0. 环境设置 (GPU 狂暴模式 & 画图补丁)

# =====

plt.rcParams['font.sans-serif'] = ['SimHei']

plt.rcParams['axes.unicode_minus'] = False
```

```
horizons = [ 3, 5, 10, 15, 30, 45, 60, 120]
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print(f"🔥 当前计算设备: {device} (RTX 4060 准备就绪)")
```

```
# =====
```

```
# 1. 数据准备与特征筛选 (完全对齐基准)
```

```
# =====
```

```
print("1. 正在加载数据并进行特征筛选...")
```

```
input_file = 'power_plant_cleaned_data.csv'
```

```
df = pd.read_csv(input_file, index_col=0, parse_dates=True)
```

```
target_col = 'Actual_Load'
```

```
correlations = df.corr()[target_col].abs()
```

```
selected_features = correlations[correlations > 0.6].index.tolist()
```

```
cheat_keywords = [
```

```
    # 1. 答案泄露与物理代理 (防止模型作弊抄答案)
```

```
    '功率', '负荷', 'AGC', '指令', '主蒸汽流量', 'Actual_Load',
```

```
    # 2. 控制系统内部衍生变量 (防止引入计算公式导致的过拟合)
```

```
    '校正后', '定值', '偏置', '前馈量', '风量比值',
```

```
    # 3. 空间与物理分支 (逼迫模型去看"总和"特征, 解决共线性)
```

```
    # 注意: 这里保留了煤粉和给煤机, 因为大概率没有"总给煤量"
```

```

    '侧',

    # 4. 极端冗余的多胞胎传感器 (只保留均值或主管道)
    # 针对汽包压力、主蒸汽压力、再热器等重复点位进行精准击杀
    '压力 1', '压力 2', '压力 3', '压力 237',
    '#1', '#2',
    'A 压力', 'B 压力',
    '248', '249', '250',
    '253', '254'
]

features = []

for col in selected_features:
    is_cheat = False

    for keyword in cheat_keywords:
        if keyword in col:
            is_cheat = True
            break

    if not is_cheat and col != target_col:
        features.append(col)

print(f"纯 RNN 模型使用的核心特征数量: {len(features)}")

df = df[features + [target_col]]

scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
scaled_X = scaler_X.fit_transform(df[features])
scaled_y = scaler_y.fit_transform(df[[target_col]])

```

```

def create_sequences(data_X, data_y, seq_length, predict_ahead):

    xs, ys = [], []

    for i in range(len(data_X) - seq_length - predict_ahead):

        x_window = data_X[i : (i + seq_length)]

        y_target = data_y[i + seq_length + predict_ahead]

        xs.append(x_window)

        ys.append(y_target)

    return np.array(xs), np.array(ys)

# =====

# 训练多时间版本模型

# =====

for predict_ahead in horizons:

    seq_length = max(60, int(predict_ahead * 1.5))

    model_name = f'pure_rnn_best_{predict_ahead}.pth'

    print(f'2. 正在切割时序窗口 (回顾 {seq_length}m -> 预测未来 {predict_ahead}m)...')

    X_seq, y_seq = create_sequences(scaled_X, scaled_y, seq_length,
predict_ahead)

    split_idx = int(len(X_seq) * 0.8)

    X_train, y_train = X_seq[:split_idx], y_seq[:split_idx]

    X_test, y_test = X_seq[split_idx:], y_seq[split_idx:]

```

```

X_train_tensor = torch.tensor(X_train, dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).to(device)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).to(device)

train_loader = DataLoader(TensorDataset(X_train_tensor, y_train_tensor),
batch_size=128, shuffle=False)

# =====
# 2. 定义纯 RNN 模型 (对照组核心)
# =====

class Pure_RNN_Model(nn.Module):

    def __init__(self, input_features, hidden_dim, num_layers):

        super(Pure_RNN_Model, self).__init__()

        # 【核心区别】: 这里使用的是最基础的 nn.RNN, 而不是
nn.LSTM

        self.rnn = nn.RNN(input_size=input_features,
hidden_size=hidden_dim, num_layers=num_layers, batch_first=True)

        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):

        # RNN 的输出结构与 LSTM 略有不同, 它没有细胞状态(Cell
State), 只有隐藏状态

        rnn_out, _ = self.rnn(x)

        final_prediction = self.fc(rnn_out[:, -1, :])

        return final_prediction

```



```

print("3. 实例化纯 RNN 模型...")

model = Pure_RNN_Model(input_features=len(features), hidden_dim=64,
num_layers=1).to(device)

# =====

# 3. 工业级训练引擎 (早停与截胡机制)

# =====

criterion = nn.MSELoss()

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

epochs = 500

print(f"\n4. 开始训练纯 RNN (最高 {epochs} 轮，开启过拟合监控)...")

train_losses = []
test_losses = []
best_test_loss = float('inf')
patience = 20
no_improve_count = 0
best_epoch = 0

for epoch in range(epochs):

    model.train()

    epoch_train_loss = 0

    for batch_X, batch_y in train_loader:

        optimizer.zero_grad()

        predictions = model(batch_X)

        loss = criterion(predictions, batch_y)

```

```

        loss.backward()

        optimizer.step()

        epoch_train_loss += loss.item()

    avg_train_loss = epoch_train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    model.eval()
    with torch.no_grad():
        test_preds = model(X_test_tensor)
        avg_test_loss = criterion(test_preds, y_test_tensor).item()
    test_losses.append(avg_test_loss)

    if avg_test_loss < best_test_loss:
        best_test_loss = avg_test_loss
        # 独立保存 RNN 的模型文件
        torch.save(model.state_dict(), model_name)
        no_improve_count = 0
        best_epoch = epoch + 1
    else:
        no_improve_count += 1

    if (epoch + 1) % 5 == 0:
        print(f"    Epoch [{epoch+1}/{epochs}] | Train Loss:
{avg_train_loss:.6f} | Test Loss: {avg_test_loss:.6f}")

    if no_improve_count >= patience:

```

```

        print(f"\n 🚨 触发早停！测试集误差连续 {patience} 轮未下
降。")

        print(f" 🌟 最佳 RNN 模型已在第 {best_epoch} 轮成功截
胡！")

        break

# =====

# 4. 评估与可视化 (巅峰状态)

# =====

print("\n5. 正在加载巅峰状态的大脑，计算纯 RNN 最终得分...")

model.load_state_dict(torch.load(model_name, weights_only=True))

model.eval()

with torch.no_grad():

    test_predictions = model(X_test_tensor).cpu()

                                real_y_test =
scaler_y.inverse_transform(y_test_tensor.cpu().numpy().reshape(-1, 1))

    real_y_pred = scaler_y.inverse_transform(test_predictions.numpy())

mae = mean_absolute_error(real_y_test, real_y_pred)

rmse = np.sqrt(mean_squared_error(real_y_test, real_y_pred))

r2 = r2_score(real_y_test, real_y_pred)

print("-" * 30)

print(f" 【纯 RNN 成绩单 (早停巅峰版)】 预测 {predict_ahead} min")

print(f" 平均绝对误差 (MAE): {mae:.2f} MW")

```

```

print(f'均方根误差 (RMSE): {rmse:.2f} MW')

print(f'真实预测准确度 (R²): {r2:.4f} ')

print("-" * 30)

pd.DataFrame({
    'Real_Load': real_y_test.flatten(),
    'Pure_RNN_Pred': real_y_pred.flatten()
}).to_csv(f'pure_rnn_results_{predict_ahead}.csv', index=False)

print(f"✅ 巅峰权重已保存为 {model_name}！预测数据已保存至
pure_rnn_results_{predict_ahead}.csv！")

# 画出 Train 和 Test 双曲线对比图

actual_epochs = len(train_losses)

plt.figure(figsize=(10, 6))

plt.plot(range(1, actual_epochs + 1), train_losses, label='Train Loss (训练误差)',
color='blue', alpha=0.7)

plt.plot(range(1, actual_epochs + 1), test_losses, label='Test Loss (测试误差)',
color='red', alpha=0.7)

best_epoch_index = test_losses.index(min(test_losses))

plt.axvline(x=best_epoch_index + 1, color='green', linestyle='--', label=f'Best
Model (Epoch {best_epoch_index + 1})')

plt.title('Pure RNN Loss Curve (Early Stopping)')

plt.xlabel('Epochs')

plt.ylabel('Loss (MSE)')

plt.grid(True, linestyle='--', alpha=0.6)

```

```
plt.legend()
```

```
plt.show()
```

2.6 长短期记忆网络 (LSTM)

2.6.1 原理解析

长短期记忆网络 (Long Short-Term Memory, LSTM) 是针对传统 RNN 梯度消失问题而提出的一种极其优雅的改进架构。LSTM 的核心创新在于引入了复杂的“门控机制 (Gating Mechanism)”——包括遗忘门、输入门和输出门，以及一条贯穿始终的细胞状态 (Cell State) 主线。

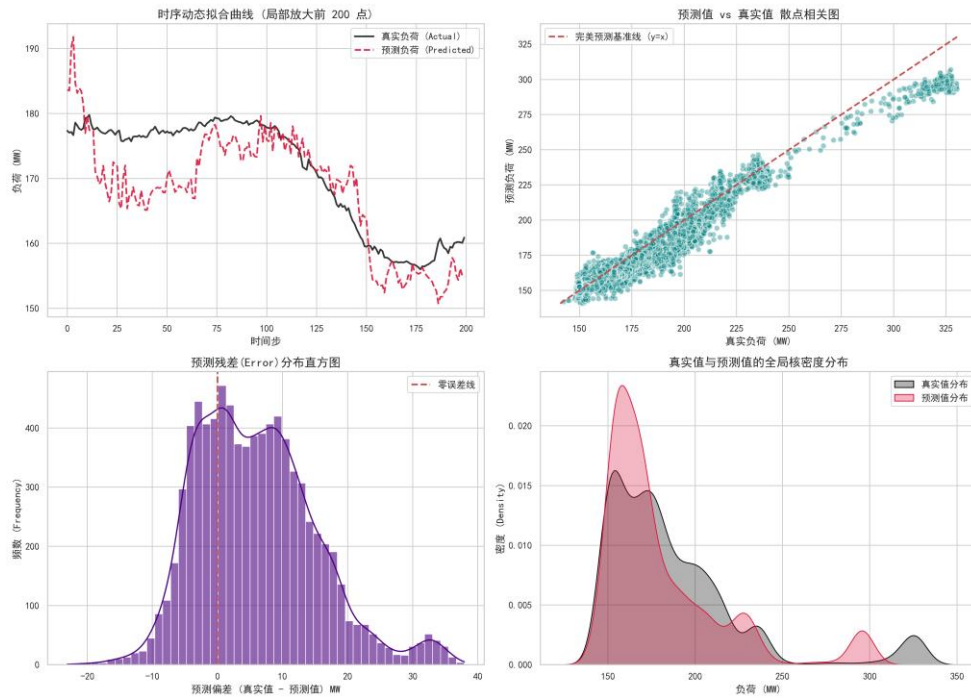
2.6.2 模型优势与应用

这些门控结构充当了信息的“过滤器”，使得 LSTM 能够自主学习哪些过去的传感器数据（如 20 分钟前的一次剧烈给煤量波动）需要被长期保留，哪些无用的高频噪音应当被遗忘。在本次火电厂负荷预测任务中，纯 LSTM 模型被用作消融实验的核心参照物，用来验证纯粹的时序记忆能力对预测精度的贡献下限。

2.6.3 各预测时间档位模型训练评估图谱

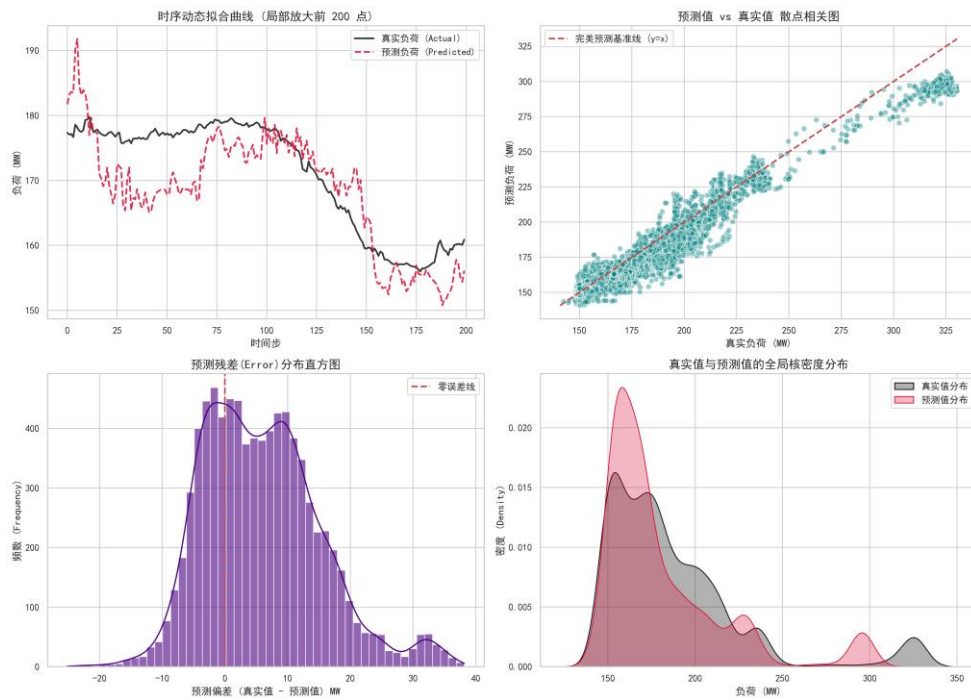
2.6.3.1 预测 3min 之后的模型训练评估：

【时序记忆网络：纯 LSTM_3min】 多维评估图谱



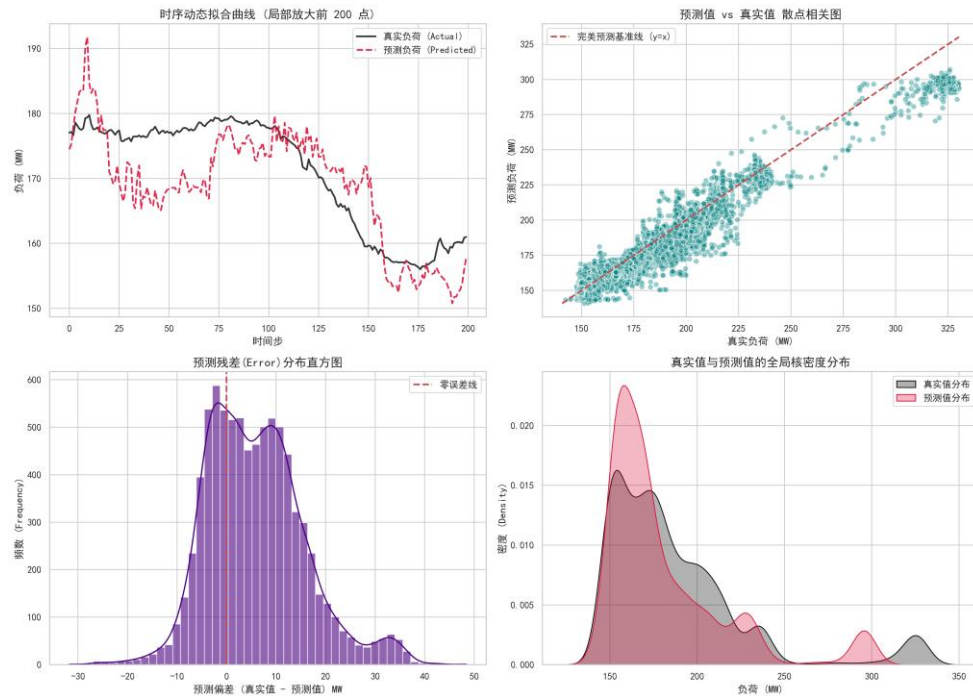
2.6.3.2 预测 5min 之后的模型训练评估:

【时序记忆网络：纯 LSTM_5min】 多维评估图谱



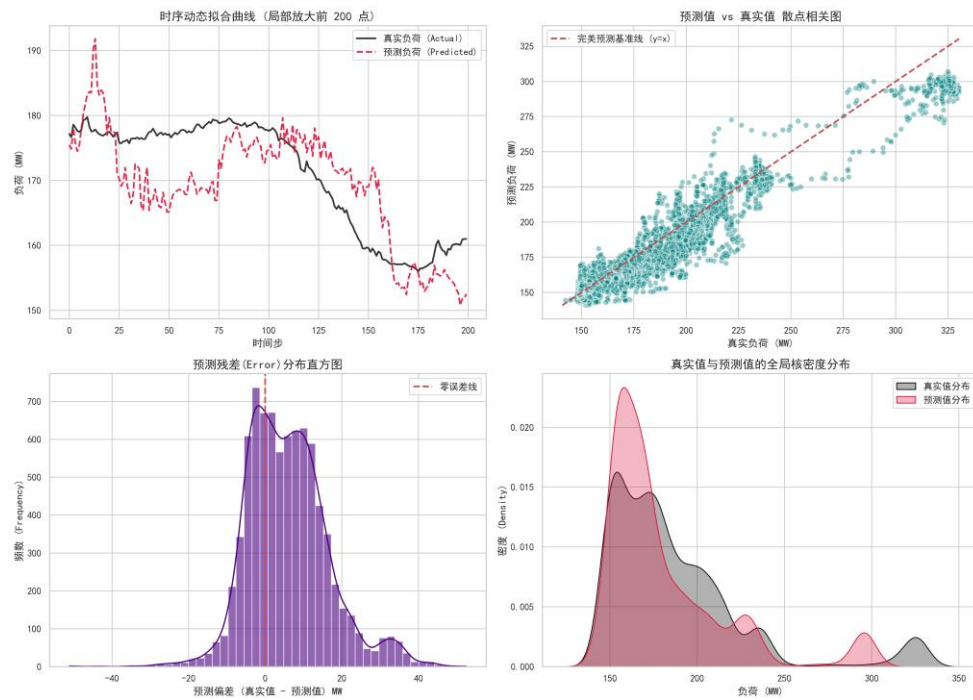
2.6.3.3 预测 10min 之后的模型训练评估:

【时序记忆网络：纯 LSTM_10min】 多维评估图谱



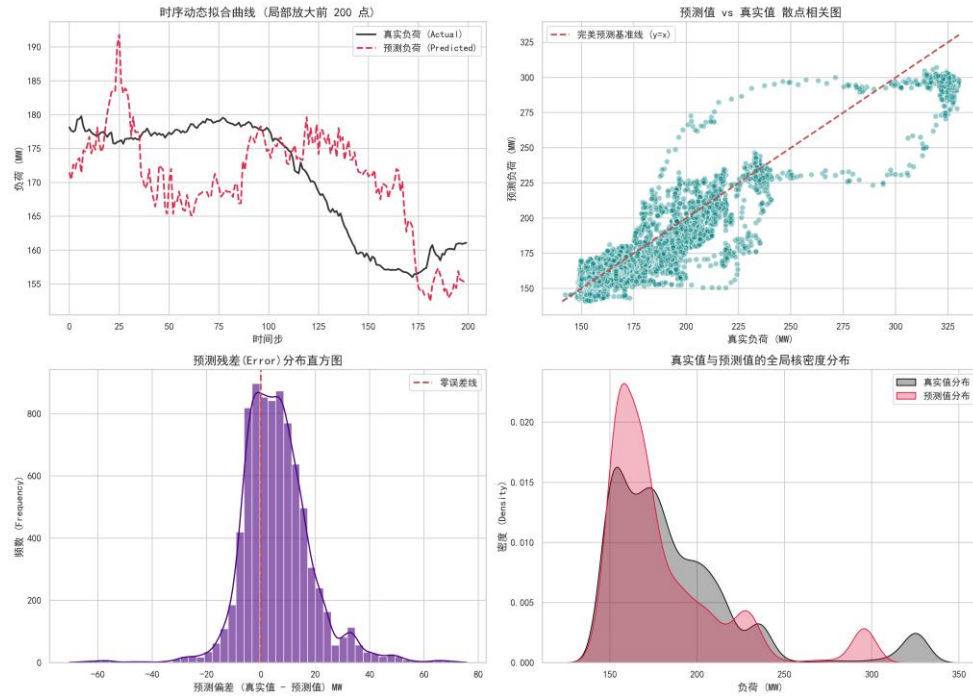
2.6.3.4 预测 15min 之后的模型训练评估:

【时序记忆网络：纯 LSTM_15min】 多维评估图谱



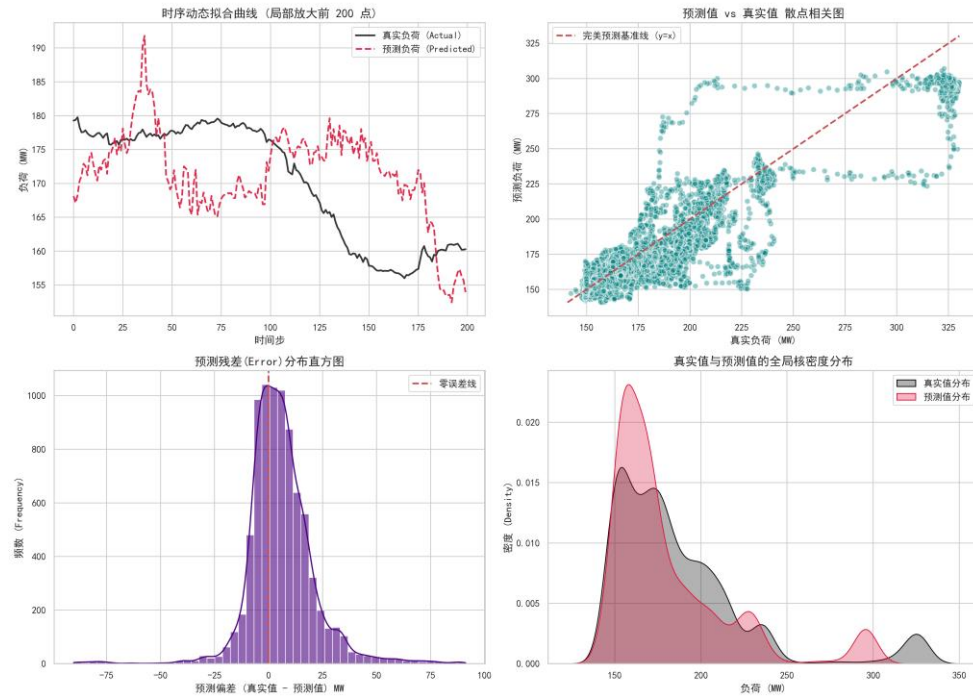
2.6.3.5 预测 30min 之后的模型训练评估:

【时序记忆网络：纯 LSTM_30min】多维评估图谱



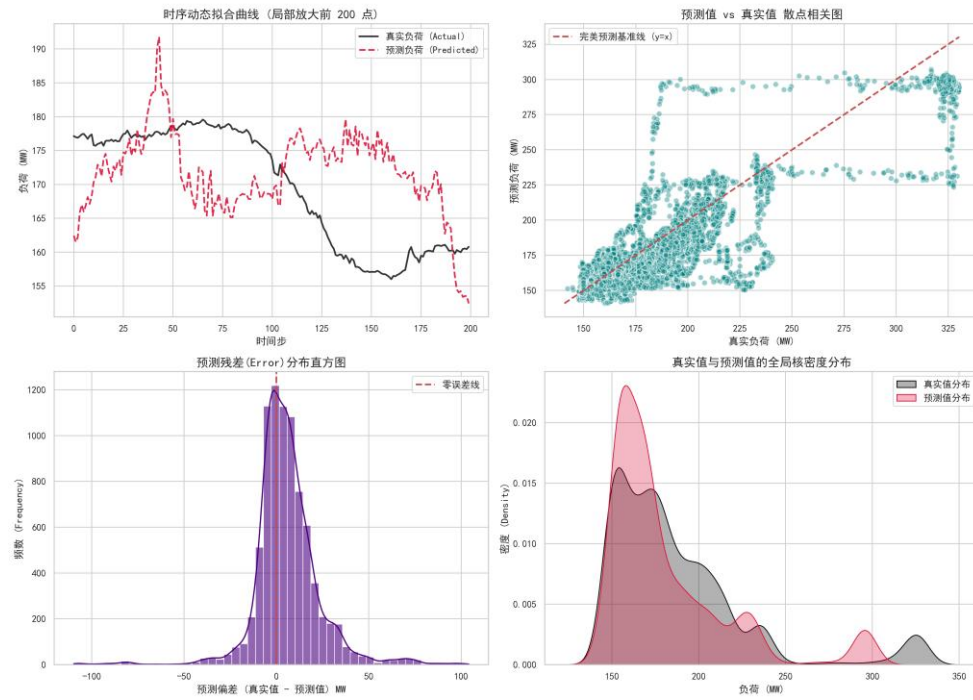
2.6.3.6 预测 45min 之后的模型训练评估:

【时序记忆网络：纯 LSTM_45min】多维评估图谱



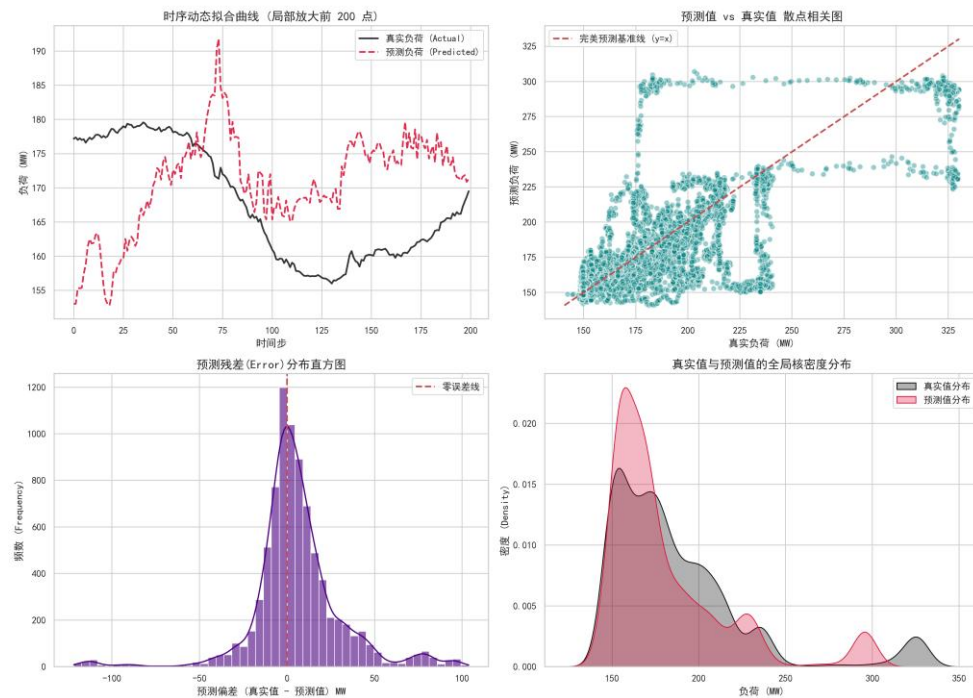
2.6.3.7 预测 60min 之后的模型训练评估:

【时序记忆网络：纯 LSTM_60min】 多维评估图谱



2.6.3.8 预测 120min 之后的模型训练评估:

【时序记忆网络：纯 LSTM_120min】 多维评估图谱



2.6.4 源代码:

```
import pandas as pd
```

```

import numpy as np

import torch

import matplotlib.pyplot as plt

import torch.nn as nn

from torch.utils.data import DataLoader, TensorDataset

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# =====

# 0. 环境设置 (GPU 狂暴模式 & 画图补丁)

# =====

plt.rcParams['font.sans-serif'] = ['SimHei'] # 强制使用系统自带的黑体解决
中文乱码

plt.rcParams['axes.unicode_minus'] = False # 防止坐标轴负号变成方块

horizons = [ 3, 5, 10, 15, 30, 45, 60, 120]

# 自动探测 NVIDIA 显卡

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"🔥 当前计算设备: {device} (RTX 4060 准备就绪)")

# =====

# 1. 数据准备与特征筛选

# =====

print("1. 正在加载数据并进行特征筛选...")

input_file = 'power_plant_cleaned_data.csv'

df = pd.read_csv(input_file, index_col=0, parse_dates=True)

```

```
target_col = 'Actual_Load'
```

```
correlations = df.corr()[target_col].abs()
```

```
selected_features = correlations[correlations > 0.6].index.tolist()
```

```
cheat_keywords = [
```

```
    # 1. 答案泄露与物理代理 (防止模型作弊抄答案)
```

```
    '功率', '负荷', 'AGC', '指令', '主蒸汽流量', 'Actual_Load',
```

```
    # 2. 控制系统内部衍生变量 (防止引入计算公式导致的过拟合)
```

```
    '校正后', '定值', '偏置', '前馈量', '风量比值',
```

```
    # 3. 空间与物理分支 (逼迫模型去看"总和"特征，解决共线性)
```

```
    # 注意：这里保留了煤粉和给煤机，因为大概率没有"总给煤量"
```

```
    '侧',
```

```
    # 4. 极端冗余的多胞胎传感器 (只保留均值或主管道)
```

```
    # 针对汽包压力、主蒸汽压力、再热器等重复点位进行精准击杀
```

```
    '压力 1', '压力 2', '压力 3', '压力 237',
```

```
    '#1', '#2',
```

```
    'A 压力', 'B 压力',
```

```
    '248', '249', '250',
```

```
    '253', '254'
```

```
]
```

```
features = []
```

```
for col in selected_features:
```

```

is_cheat = False

for keyword in cheat_keywords:

    if keyword in col:

        is_cheat = True

        break

if not is_cheat and col != target_col:

    features.append(col)

print(f'纯 LSTM 模型使用的核心特征数量: {len(features)}')

df = df[features + [target_col]]

scaler_X = MinMaxScaler()

scaler_y = MinMaxScaler()

scaled_X = scaler_X.fit_transform(df[features])

scaled_y = scaler_y.fit_transform(df[[target_col]])

def create_sequences(data_X, data_y, seq_length, predict_ahead):

    xs, ys = [], []

    for i in range(len(data_X) - seq_length - predict_ahead):

        x_window = data_X[i : (i + seq_length)]

        y_target = data_y[i + seq_length + predict_ahead]

        xs.append(x_window)

        ys.append(y_target)

    return np.array(xs), np.array(ys)

# =====

# 训练多时间版本模型

```

```

# =====

for predict_ahead in horizons:

    seq_length = max(60, int(predict_ahead * 1.5))

    model_name = f"pure_lstm_best_{predict_ahead}.pth"

    print(f'2. 正在切割时序窗口 (回顾 {seq_length}m -> 预测未来 {predict_ahead}m)...')

    X_seq, y_seq = create_sequences(scaled_X, scaled_y, seq_length,
predict_ahead)

    split_idx = int(len(X_seq) * 0.8)

    X_train, y_train = X_seq[:split_idx], y_seq[:split_idx]

    X_test, y_test = X_seq[split_idx:], y_seq[split_idx:]

    X_train_tensor = torch.tensor(X_train, dtype=torch.float32).to(device)

    y_train_tensor = torch.tensor(y_train, dtype=torch.float32).to(device)

    X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)

    # 【修复点】: 测试集答案推到显卡，防止验证时跨设备报错

    y_test_tensor = torch.tensor(y_test, dtype=torch.float32).to(device)

    # 调大 batch_size 充分喂饱显卡

    train_loader = DataLoader(TensorDataset(X_train_tensor, y_train_tensor),
batch_size=128, shuffle=False)

# =====

# 2. 定义纯 LSTM 模型 (消融实验核心)

```

```

# =====

class Pure_LSTM_Model(nn.Module):

    def __init__(self, input_features, hidden_dim, num_layers):

        super(Pure_LSTM_Model, self).__init__()

        self.lstm = nn.LSTM(input_size=input_features,
hidden_size=hidden_dim, num_layers=num_layers, batch_first=True)

        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):

        lstm_out, _ = self.lstm(x)

        final_prediction = self.fc(lstm_out[:, -1, :])

        return final_prediction

print("3. 实例化纯 LSTM 模型...")

model = Pure_LSTM_Model(input_features=len(features), hidden_dim=64,
num_layers=1).to(device)

# =====

# 3. 工业级训练引擎 (早停与截胡机制)

# =====

criterion = nn.MSELoss()

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

epochs = 500 # 放开手脚，让早停决定

print(f"\n4. 开始训练纯 LSTM (最高 {epochs} 轮, 开启过拟合监控)...")

train_losses = []

```

```

test_losses = []

best_test_loss = float('inf')

patience = 20

no_improve_count = 0

best_epoch = 0


for epoch in range(epochs):

    # --- 训练阶段 ---

    model.train()

    epoch_train_loss = 0

    for batch_X, batch_y in train_loader:

        optimizer.zero_grad()

        predictions = model(batch_X)

        loss = criterion(predictions, batch_y)

        loss.backward()

        optimizer.step()

        epoch_train_loss += loss.item()

    avg_train_loss = epoch_train_loss / len(train_loader)

    train_losses.append(avg_train_loss)


    # --- 验证阶段 (摸底考试) ---

    model.eval()

    with torch.no_grad():

        test_preds = model(X_test_tensor)

        avg_test_loss = criterion(test_preds, y_test_tensor).item()

    test_losses.append(avg_test_loss)

```

```

# --- 早停监控 ---

if avg_test_loss < best_test_loss:

    best_test_loss = avg_test_loss

    torch.save(model.state_dict(), model_name)

    no_improve_count = 0

    best_epoch = epoch + 1

else:

    no_improve_count += 1


if (epoch + 1) % 5 == 0:

    print(f"    Epoch [{epoch+1}/{epochs}] | Train Loss:
{avg_train_loss:.6f} | Test Loss: {avg_test_loss:.6f}")


if no_improve_count >= patience:

    print(f"\n 🚨 触发早停！测试集误差连续 {patience} 轮未下
降。")

    print(f" 🌟 最佳 LSTM 模型已在第 {best_epoch} 轮成功截
胡！")

    break


# =====

# 4. 评估与可视化 (巅峰状态)

# =====

print("\n5. 正在加载巅峰状态的大脑，计算纯 LSTM 最终得分...")

# 【修复点】：增加 weights_only=True 消除安全警告

model.load_state_dict(torch.load('pure_lstm_best.pth', weights_only=True))

```



```

model.eval()

with torch.no_grad():
    test_predictions = model(X_test_tensor).cpu()

# 【修复点】: 把 y_test_tensor 从显卡拉回 CPU (.cpu()) 进行打分
real_y_test = scaler_y.inverse_transform(y_test_tensor.cpu().numpy().reshape(-1, 1))
real_y_pred = scaler_y.inverse_transform(test_predictions.numpy())

mae = mean_absolute_error(real_y_test, real_y_pred)
rmse = np.sqrt(mean_squared_error(real_y_test, real_y_pred))
r2 = r2_score(real_y_test, real_y_pred)

print("-" * 30)
print(f"【纯 LSTM 成绩单 (早停巅峰版)】 预测 {predict_ahead} min")
print(f"平均绝对误差 (MAE): {mae:.2f} MW")
print(f"均方根误差 (RMSE): {rmse:.2f} MW")
print(f"真实预测准确度 (R²): {r2:.4f}")
print("-" * 30)

pd.DataFrame({
    'Real_Load': real_y_test.flatten(),
    'Pure_LSTM_Pred': real_y_pred.flatten()
}).to_csv(f'pure_lstm_results_{predict_ahead}.csv', index=False)

print(f"✅ 巅峰权重已保存为 {model_name}！预测数据已保存至
pure_lstm_results_{predict_ahead}.csv！")

```

```

# 画出 Train 和 Test 双曲线对比图

actual_epochs = len(train_losses)

plt.figure(figsize=(10, 6))

plt.plot(range(1, actual_epochs + 1), train_losses, label='Train Loss (训练误差)',
color='blue', alpha=0.7)

plt.plot(range(1, actual_epochs + 1), test_losses, label='Test Loss (测试误差)',
color='red', alpha=0.7)

# 标记截胡位置

best_epoch_index = test_losses.index(min(test_losses))

plt.axvline(x=best_epoch_index + 1, color='green', linestyle='--', label=f'Best
Model (Epoch {best_epoch_index + 1})')

plt.title('Pure LSTM Loss Curve (Early Stopping)')

plt.xlabel('Epochs')

plt.ylabel('Loss (MSE)')

plt.grid(True, linestyle='--', alpha=0.6)

plt.legend()

plt.show()

```

2.7 卷积与长短期记忆混合网络 (CNN-LSTM)

2.7.1 原理解析

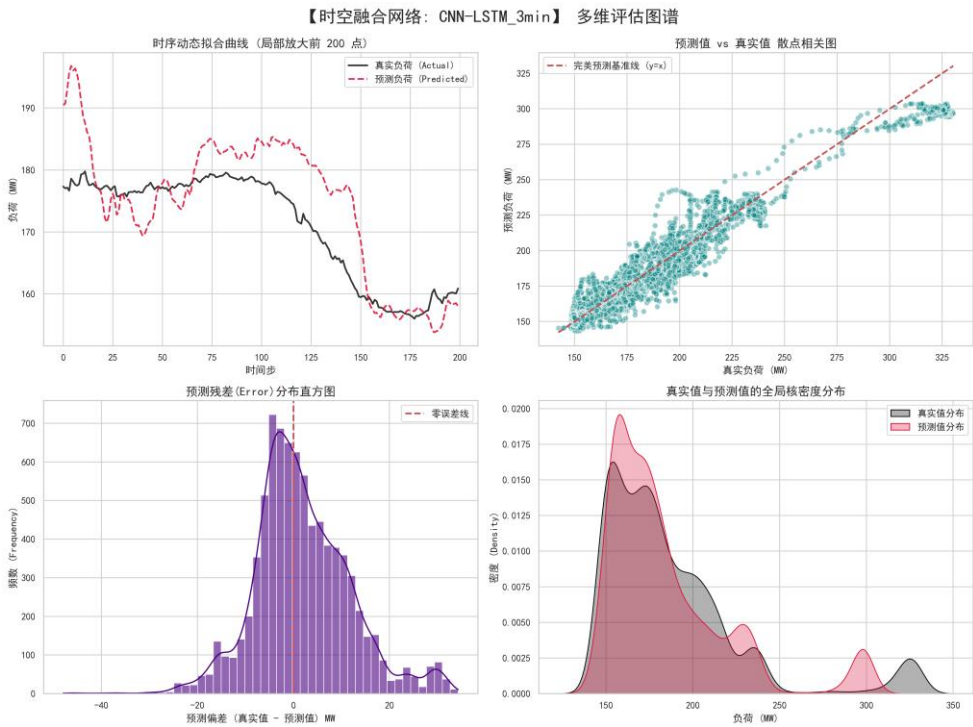
CNN-LSTM 是一种典型的时空融合架构 (Spatio-Temporal Fusion)。该架构在前端利用一维卷积层 (Conv1D) 作为特征提取器，在后端连接 LSTM 处理时序记忆。

2.7.2 模型优势与应用

在面对复杂的工业多变量时间序列时，单纯依靠 LSTM 会使得计算负荷过重且容易被局部突变干扰。本研究通过引入 CNN 层，让卷积核在时间窗口上滑动扫描。CNN 就像一个“质检员”，能够敏锐地捕捉到多个物理量（如总风量与汽包压力）在极短时间内的局部联合突变特征。经过 CNN 提纯和降维后的高级特征再输入给 LSTM 进行长线趋势研判，极大地增强了模型对复杂工况波动的特征捕捉能力。

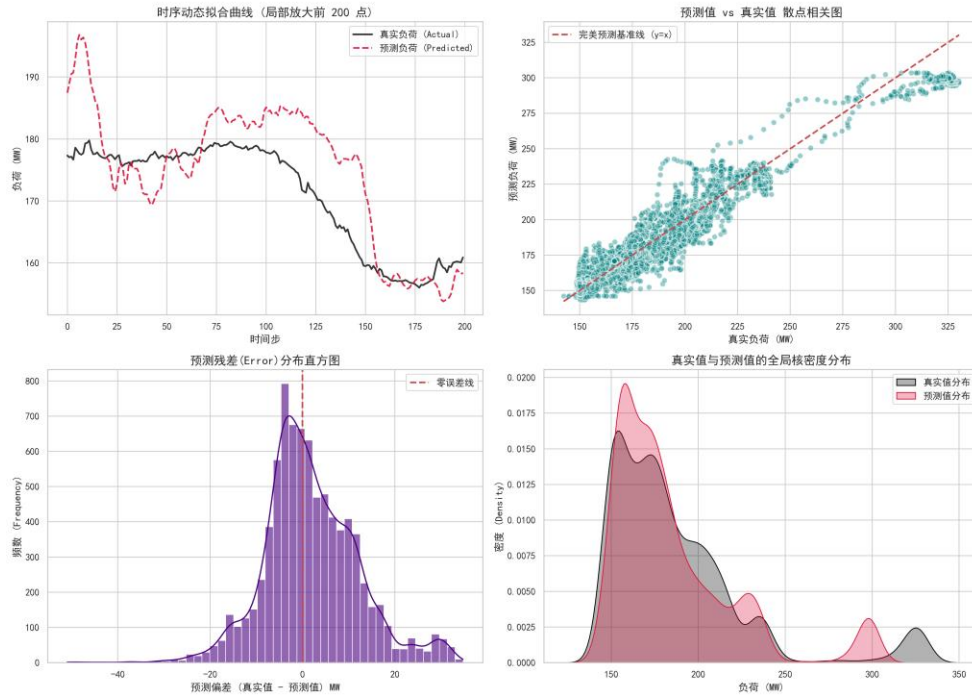
2.7.3 各预测时间档位模型训练评估图谱

2.7.3.1 预测 3min 之后的模型训练评估：



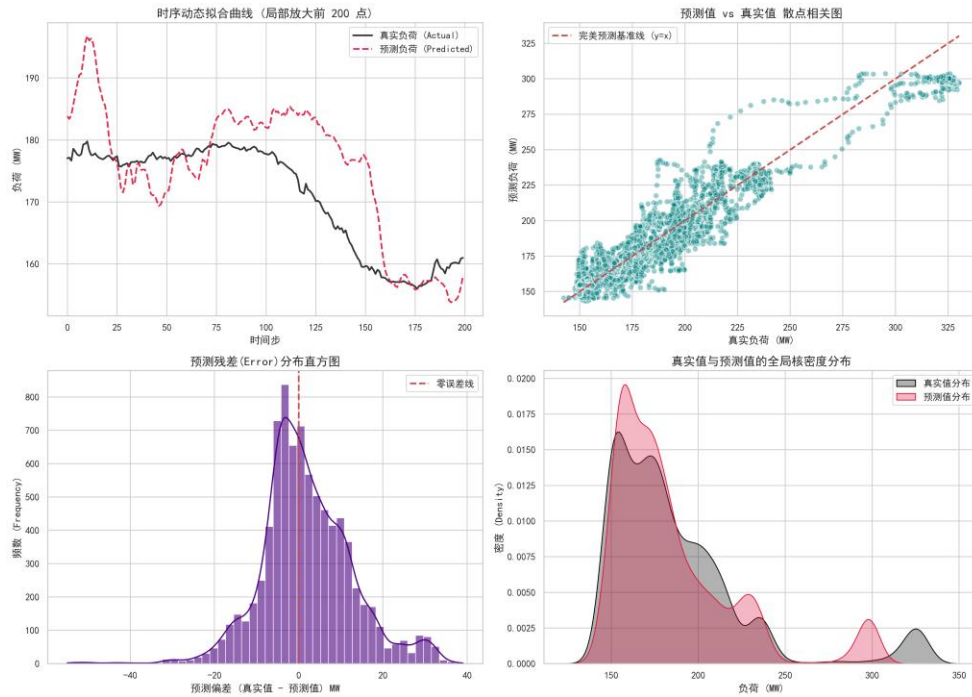
2.7.3.2 预测 5min 之后的模型训练评估：

【时空融合网络: CNN-LSTM_5min】 多维评估图谱



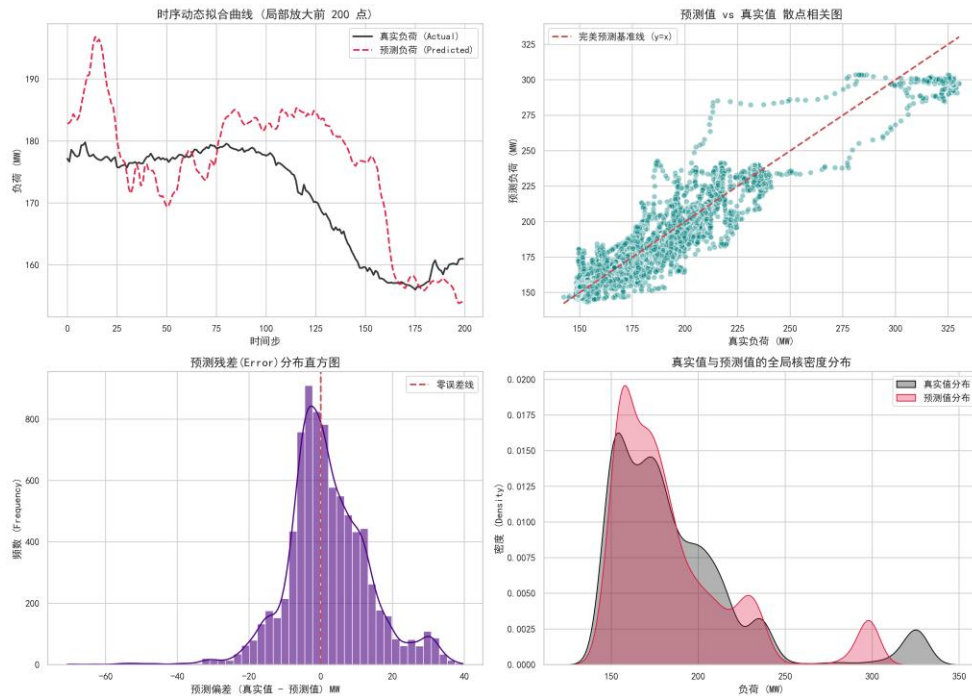
2.7.3.3 预测 10min 之后的模型训练评估:

【时空融合网络: CNN-LSTM_10min】 多维评估图谱



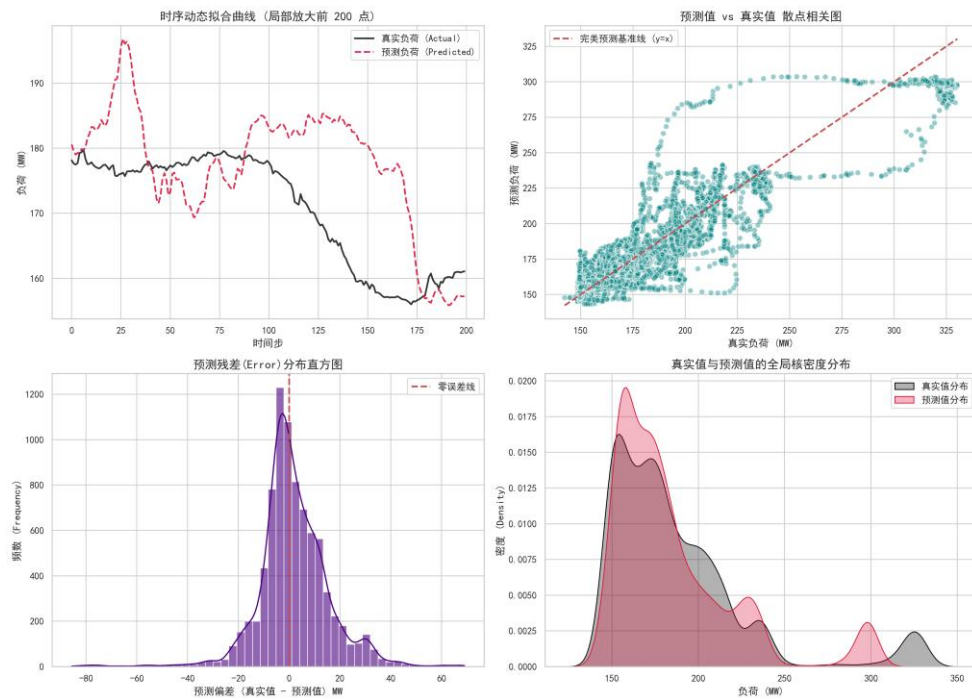
2.7.3.4 预测 15min 之后的模型训练评估:

【时空融合网络: CNN-LSTM_15min】 多维评估图谱



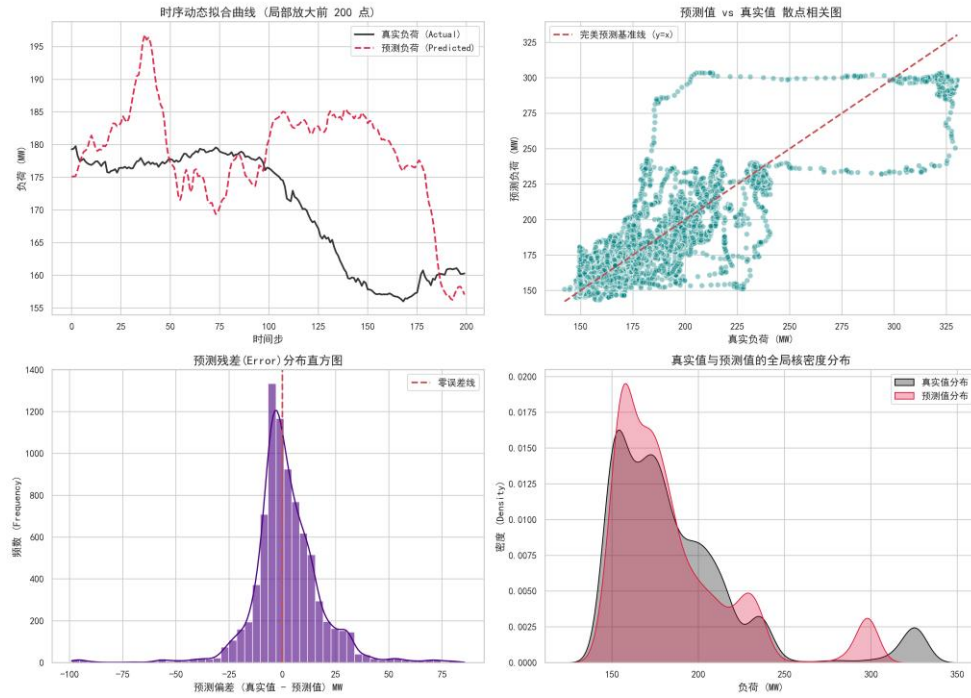
2.7.3.5 预测 30min 之后的模型训练评估:

【时空融合网络: CNN-LSTM_30min】 多维评估图谱



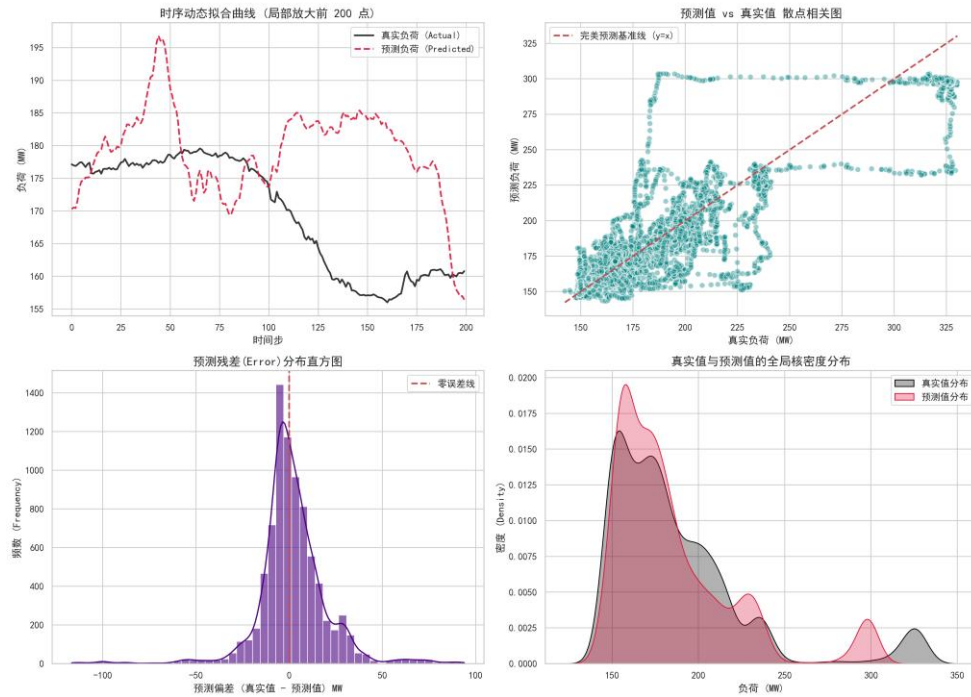
2.7.3.6 预测 45min 之后的模型训练评估:

【时空融合网络: CNN-LSTM_45min】 多维评估图谱



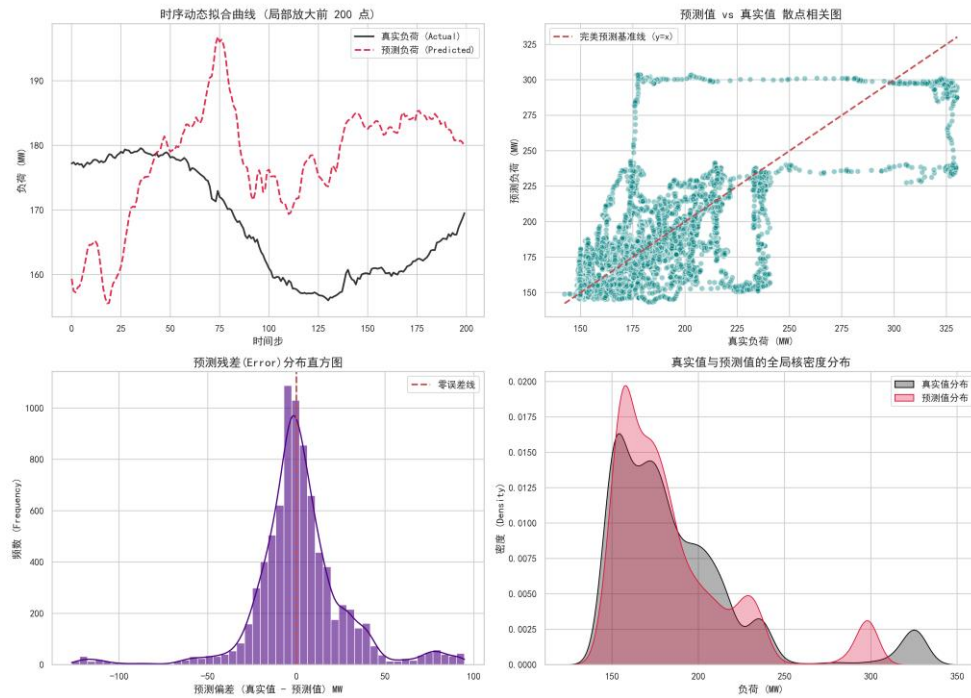
2.7.3.7 预测 60min 之后的模型训练评估:

【时空融合网络: CNN-LSTM_60min】 多维评估图谱



2.7.3.8 预测 120min 之后的模型训练评估:

【时空融合网络: CNN-LSTM_120min】多维评估图谱



2.7.4 源代码:

```
import pandas as pd

import numpy as np

import torch

import matplotlib.pyplot as plt

import torch.nn as nn

from torch.utils.data import DataLoader, TensorDataset

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


# =====

# 0. 环境设置 (GPU 狂暴模式 & 画图补丁)

# =====

plt.rcParams['font.sans-serif'] = ['SimHei'] # 强制使用系统自带的黑体解决
中文乱码
```

```

plt.rcParams['axes.unicode_minus'] = False      # 防止坐标轴负号变成方块

# 自动探测 NVIDIA 显卡

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"🔥 当前计算设备: {device} (RTX 4060 准备接管 CNN 卷积运算)")

horizons = [ 3, 5, 10, 15, 30, 45, 60, 120]

# =====

# 1. 数据准备与特征筛选

# =====

print("1. 正在加载数据并进行特征筛选...")

input_file = 'power_plant_cleaned_data.csv'

df = pd.read_csv(input_file, index_col=0, parse_dates=True)

target_col = 'Actual_Load'

correlations = df.corr()[target_col].abs()

selected_features = correlations[correlations > 0.6].index.tolist()

cheat_keywords = [

    # 1. 答案泄露与物理代理 (防止模型作弊抄答案)

    '功率', '负荷', 'AGC', '指令', '主蒸汽流量', 'Actual_Load',

    # 2. 控制系统内部衍生变量 (防止引入计算公式导致的过拟合)

    '校正后', '定值', '偏置', '前馈量', '风量比值',

    # 3. 空间与物理分支 (逼迫模型去看"总和"特征, 解决共线性)

    # 注意: 这里保留了煤粉和给煤机, 因为大概率没有"总给煤量"

```



```

    '侧',

    # 4. 极端冗余的多胞胎传感器 (只保留均值或主管道)
    # 针对汽包压力、主蒸汽压力、再热器等重复点位进行精准击杀
    '压力 1', '压力 2', '压力 3', '压力 237',
    '#1', '#2',
    'A 压力', 'B 压力',
    '248', '249', '250',
    '253', '254'
]

features = []

for col in selected_features:
    is_cheat = False

    for keyword in cheat_keywords:
        if keyword in col:
            is_cheat = True
            break

    if not is_cheat and col != target_col:
        features.append(col)

print(f"CNN-LSTM 模型使用的核心特征数量: {len(features)}")

df = df[features + [target_col]]

scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
scaled_X = scaler_X.fit_transform(df[features])
scaled_y = scaler_y.fit_transform(df[[target_col]])

```

```

def create_sequences(data_X, data_y, seq_length, predict_ahead):

    xs, ys = [], []

    for i in range(len(data_X) - seq_length - predict_ahead):

        x_window = data_X[i : (i + seq_length)]

        y_target = data_y[i + seq_length + predict_ahead]

        xs.append(x_window)

        ys.append(y_target)

    return np.array(xs), np.array(ys)

# =====

# 训练多时间版本模型

# =====

for predict_ahead in horizons:

    seq_length = max(60, int(predict_ahead * 1.5))

    print(f'2. 正在切割时序窗口 (回顾 {seq_length}m -> 预测未来 {predict_ahead}m)...')

    X_seq, y_seq = create_sequences(scaled_X, scaled_y, seq_length,
predict_ahead)

    split_idx = int(len(X_seq) * 0.8)

    X_train, y_train = X_seq[:split_idx], y_seq[:split_idx]

    X_test, y_test = X_seq[split_idx:], y_seq[split_idx:]

```

```

X_train_tensor = torch.tensor(X_train, dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).to(device)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
# 【修复点】: 测试集答案推到显卡，防止早停验证时跨设备报错
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).to(device)

# 调大 batch_size 充分喂饱显卡

train_loader = DataLoader(TensorDataset(X_train_tensor, y_train_tensor),
batch_size=128, shuffle=False)

# =====
# 2. 定义 CNN-LSTM 缝合怪模型
# =====

class CNN_LSTM_Model(nn.Module):

    def __init__(self, input_features, hidden_dim, num_layers):

        super(CNN_LSTM_Model, self).__init__()

        # CNN 特征提取

        self.cnn = nn.Conv1d(in_channels=input_features,
out_channels=32, kernel_size=3, padding=1)

        self.relu = nn.ReLU()

        # LSTM 时序记忆

        self.lstm = nn.LSTM(input_size=32, hidden_size=hidden_dim,
num_layers=num_layers, batch_first=True)

        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):

        x = x.permute(0, 2, 1)

```

```

        x = self.cnn(x)

        x = self.relu(x)

        x = x.permute(0, 2, 1)

        lstm_out, _ = self.lstm(x)

        final_prediction = self.fc(lstm_out[:, -1, :])

        return final_prediction

print("3. 实例化 CNN-LSTM 模型...")

model = CNN_LSTM_Model(input_features=len(features), hidden_dim=64,
num_layers=1).to(device)

# =====
# 3. 工业级训练引擎 (早停与截胡机制)
# =====

criterion = nn.MSELoss()

optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

epochs = 500 # 放开手脚，让早停决定

print(f"\n4. 开始训练 CNN-LSTM (最高 {epochs} 轮，开启过拟合监
控)...")

train_losses = []

test_losses = []

best_test_loss = float('inf')

patience = 20

```

```

no_improve_count = 0

best_epoch = 0

for epoch in range(epochs):
    # --- 训练阶段 ---
    model.train()

    epoch_train_loss = 0

    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()

        predictions = model(batch_X)

        loss = criterion(predictions, batch_y)

        loss.backward()

        optimizer.step()

        epoch_train_loss += loss.item()

    avg_train_loss = epoch_train_loss / len(train_loader)
    train_losses.append(avg_train_loss)

    # --- 验证阶段 (摸底考试) ---
    model.eval()

    with torch.no_grad():
        test_preds = model(X_test_tensor)

        avg_test_loss = criterion(test_preds, y_test_tensor).item()

    test_losses.append(avg_test_loss)

    # --- 早停监控 ---

    if avg_test_loss < best_test_loss:

```

```

        best_test_loss = avg_test_loss

        # 独立保存 CNN-LSTM 的模型文件

        model_name = f'XGBoost_best_model_{predict_ahead}.json'

        torch.save(model.state_dict(), model_name)

        no_improve_count = 0

        best_epoch = epoch + 1

    else:

        no_improve_count += 1

    if (epoch + 1) % 5 == 0:

        print(f'    Epoch [{epoch+1}/{epochs}] | Train Loss:
{avg_train_loss:.6f} | Test Loss: {avg_test_loss:.6f}')

    if no_improve_count >= patience:

        print(f'\n 🚨 触发早停！测试集误差连续 {patience} 轮未下
降。")

        print(f' 🌟 最佳 CNN-LSTM 模型已在第 {best_epoch} 轮成
功截胡！")

        break

# =====

# 4. 评估与可视化 (巅峰状态)

# =====

print("\n5. 正在加载巅峰状态的大脑，计算最终得分...")

model.load_state_dict(torch.load('cnn_lstm_best.pth', weights_only=True))

model.eval()

```

```

with torch.no_grad():
    test_predictions = model(X_test_tensor).cpu()

# 把 y_test_tensor 从显卡拉回 CPU 进行打分
real_y_test =
scaler_y.inverse_transform(y_test_tensor.cpu().numpy().reshape(-1, 1))

real_y_pred = scaler_y.inverse_transform(test_predictions.numpy())

mae = mean_absolute_error(real_y_test, real_y_pred)
rmse = np.sqrt(mean_squared_error(real_y_test, real_y_pred))
r2 = r2_score(real_y_test, real_y_pred)

print("-" * 30)
print(f"【CNN-LSTM 成绩单 (早停巅峰版)】")
print(f"平均绝对误差 (MAE): {mae:.2f} MW")
print(f"均方根误差 (RMSE): {rmse:.2f} MW")
print(f"真实预测准确度 (R²): {r2:.4f}")
print("-" * 30)

pd.DataFrame({
    'Real_Load': real_y_test.flatten(),
    'CNN_LSTM_Pred': real_y_pred.flatten()
}).to_csv(f'cnn_lstm_results_{predict_ahead}.csv', index=False)

print(f"✅ 巅峰权重已保存为 {model_name}！预测数据已保存至
cnn_lstm_results_{predict_ahead}.csv！")

# 画出 Train 和 Test 双曲线对比图

```

```

actual_epochs = len(train_losses)

plt.figure(figsize=(10, 6))

plt.plot(range(1, actual_epochs + 1), train_losses, label='Train Loss (训练误差)',
color='blue', alpha=0.7)

plt.plot(range(1, actual_epochs + 1), test_losses, label='Test Loss (测试误差)',
color='red', alpha=0.7)

# 标记截胡位置
best_epoch_index = test_losses.index(min(test_losses))

plt.axvline(x=best_epoch_index + 1, color='green', linestyle='--', label=f'Best
Model (Epoch {best_epoch_index + 1})')

plt.title('CNN-LSTM Loss Curve (Early Stopping)')

plt.xlabel('Epochs')

plt.ylabel('Loss (MSE)')

plt.grid(True, linestyle='--', alpha=0.6)

plt.legend()

plt.show()

```

2.8 极限梯度提升树 (XGBoost)

2.8.1 原理解析

XGBoost (eXtreme Gradient Boosting) 是基于梯度提升决策树 (GBDT) 算法的极致工程优化版本。它通过不断迭代地生成新的决策树，且每一棵新树都在拟合（纠正）前一轮模型预测残差的过程，最终将无数个弱学习器组合成一个极其强悍的预测引擎。

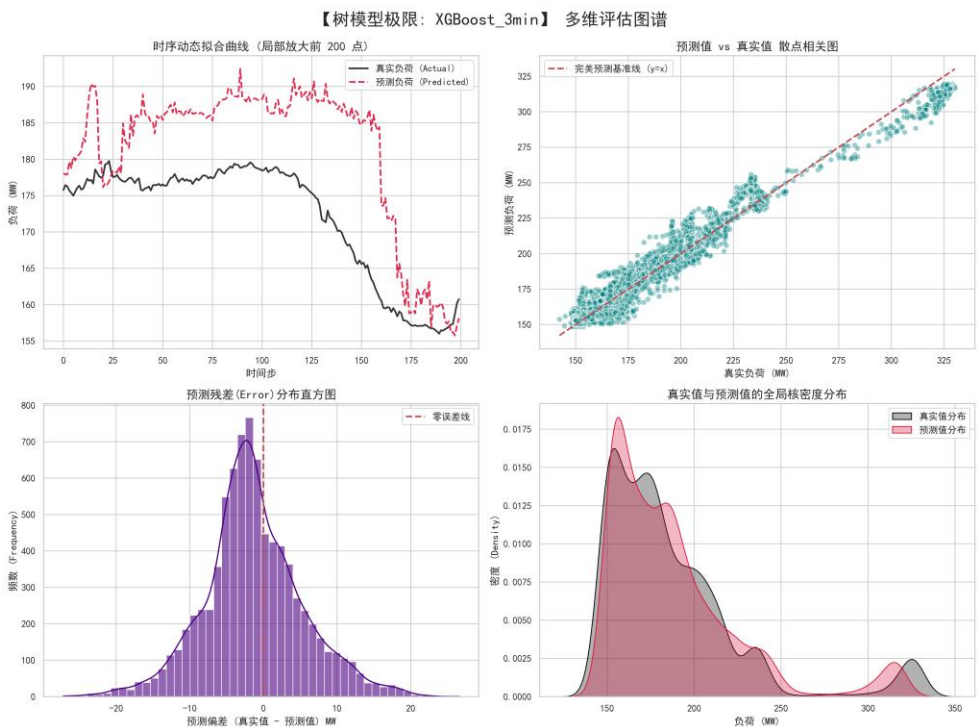
2.8.2 模型优势与应用

在学术界与工业界的各类表格数据 (Tabular Data) 竞赛中，XGBoost 被公

认为无可争议的王者。相较于黑盒化的深度学习模型，本研究引入 XGBoost 的核心目的在于获取其极具工程价值的“特征重要性（Feature Importance）”指标。通过提取 XGBoost 内部的 F-score，我们能够清晰地向业务端解释：在预测未来负荷时，究竟是哪几个核心传感器物理量起到了决定性作用。同时，其支持的直方图算法与 GPU 硬件加速，使其在训练效率上表现极其优异。

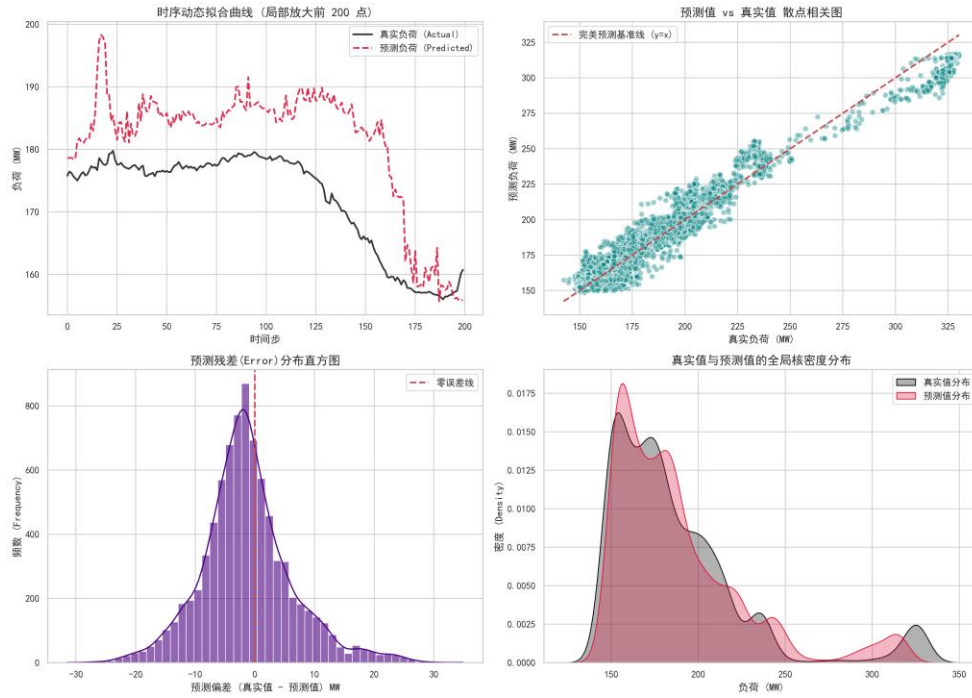
2.8.3 各预测时间档位模型训练评估图谱

2.8.3.1 预测 3min 之后的模型训练评估：



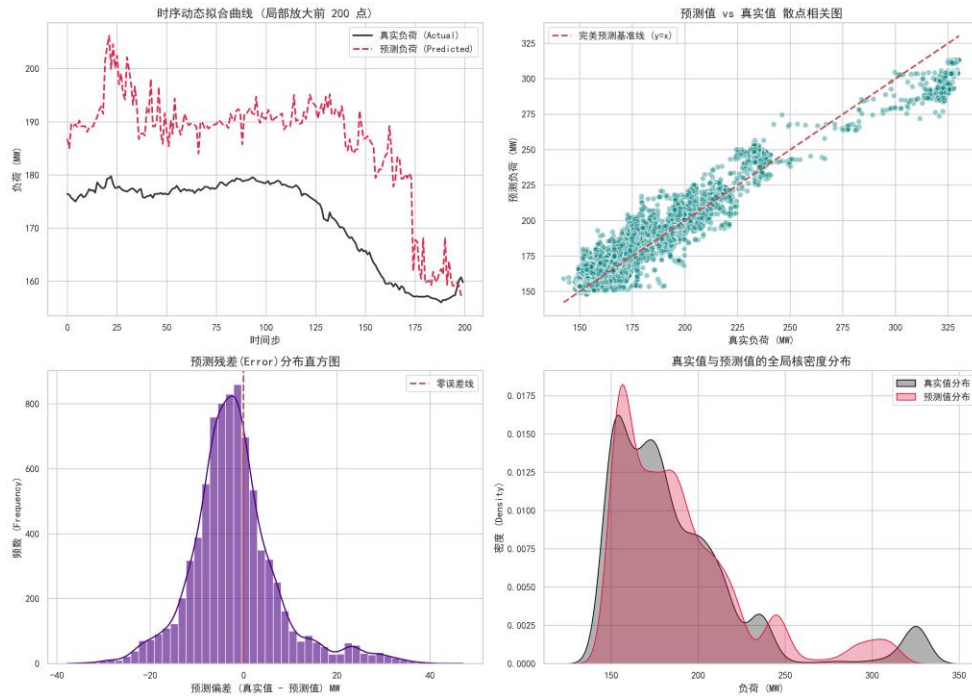
2.8.3.2 预测 5min 之后的模型训练评估：

【树模型极限: XGBoost_5min】 多维评估图谱



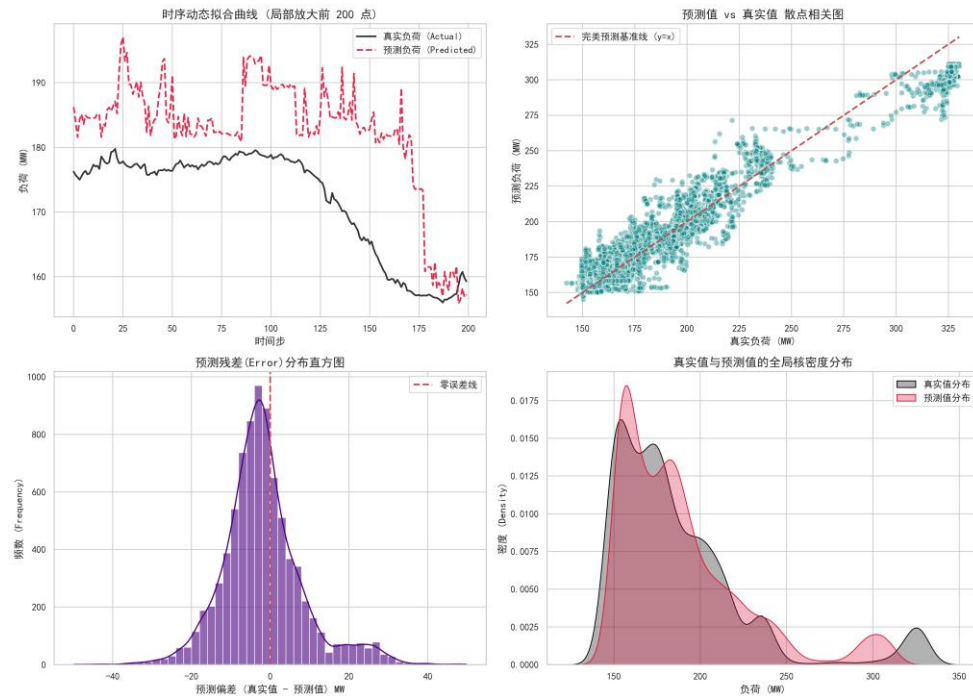
2.8.3.3 预测 10min 之后的模型训练评估:

【树模型极限: XGBoost_10min】 多维评估图谱



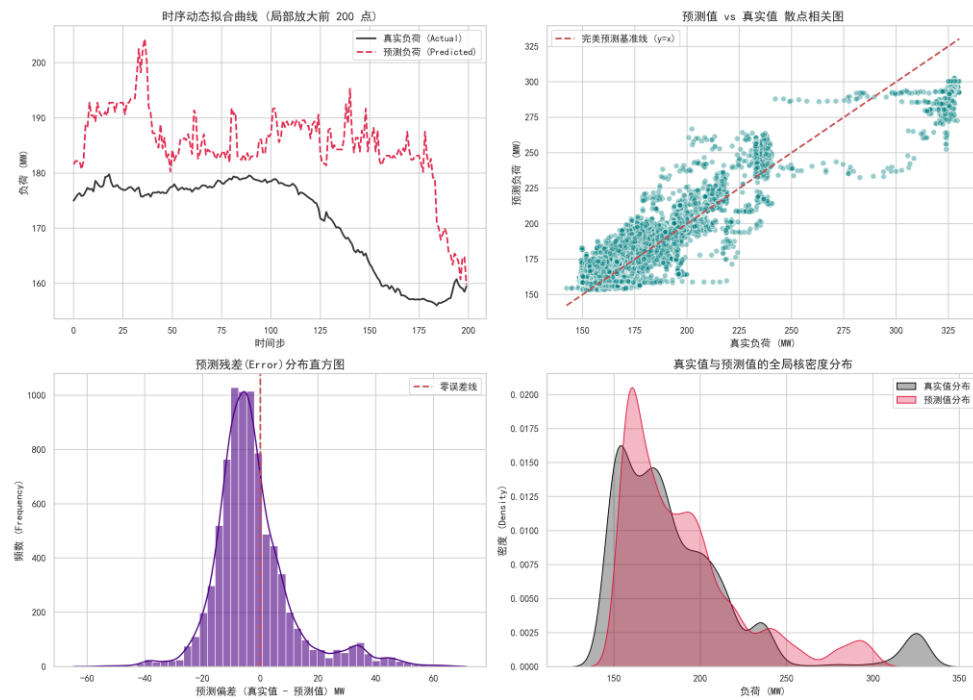
2.8.3.4 预测 15min 之后的模型训练评估:

【树模型极限: XGBoost_15min】 多维评估图谱



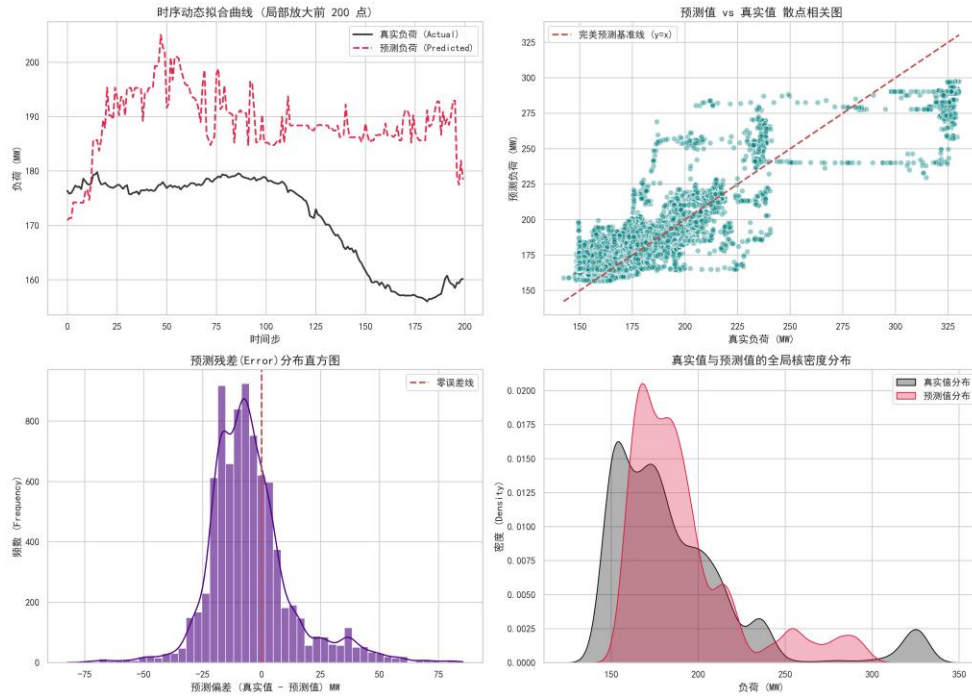
2.8.3.5 预测 30min 之后的模型训练评估:

【树模型极限: XGBoost_30min】 多维评估图谱



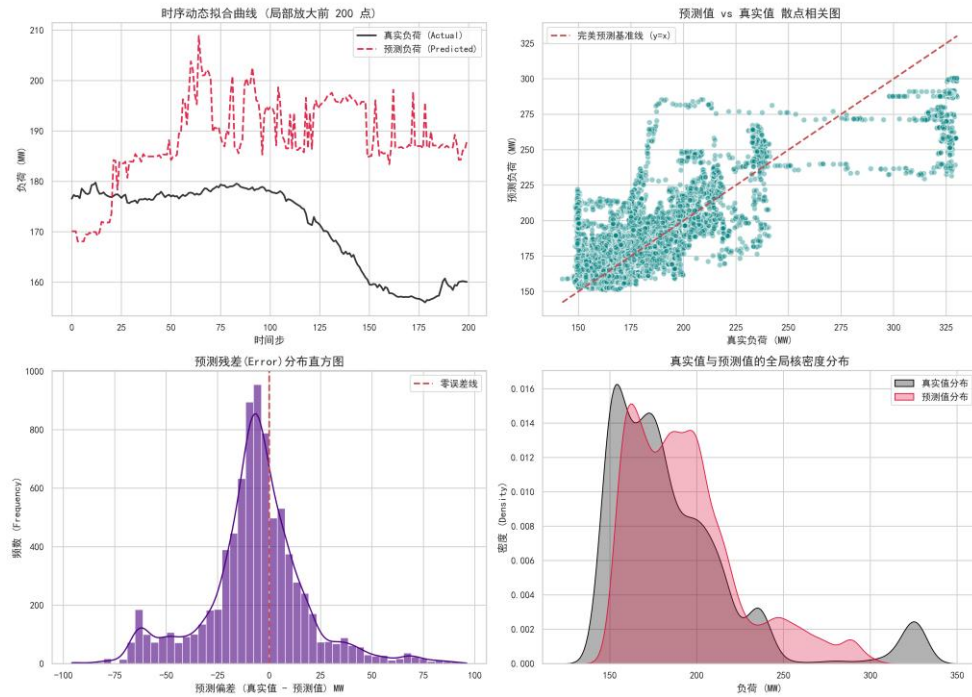
2.8.3.6 预测 45min 之后的模型训练评估:

【树模型极限: XGBoost_45min】 多维评估图谱



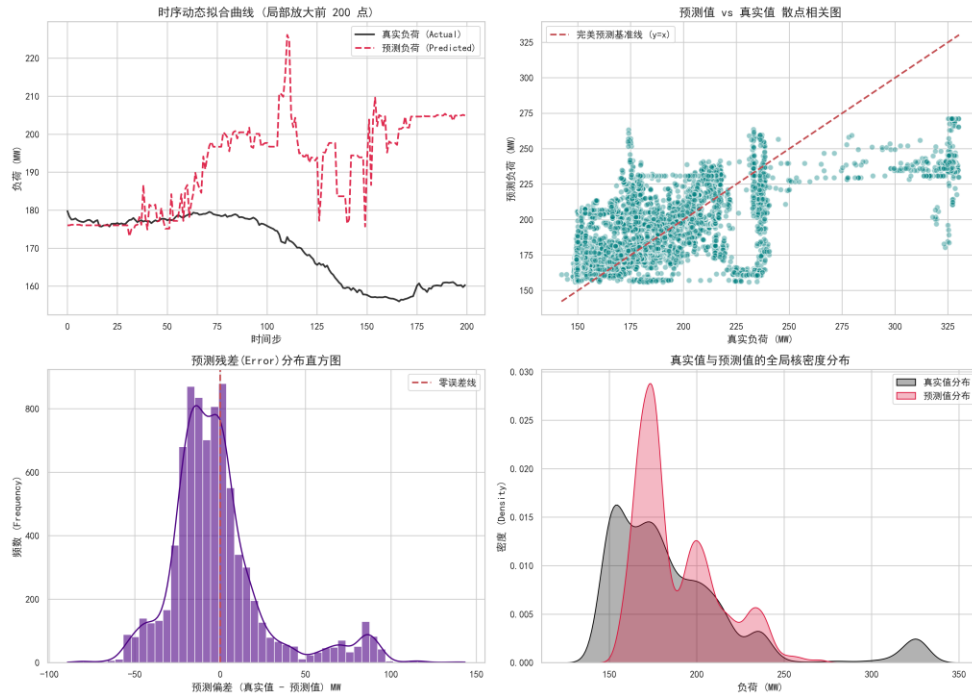
2.8.3.7 预测 60min 之后的模型训练评估:

【树模型极限: XGBoost_60min】 多维评估图谱



2.8.3.8 预测 120min 之后的模型训练评估:

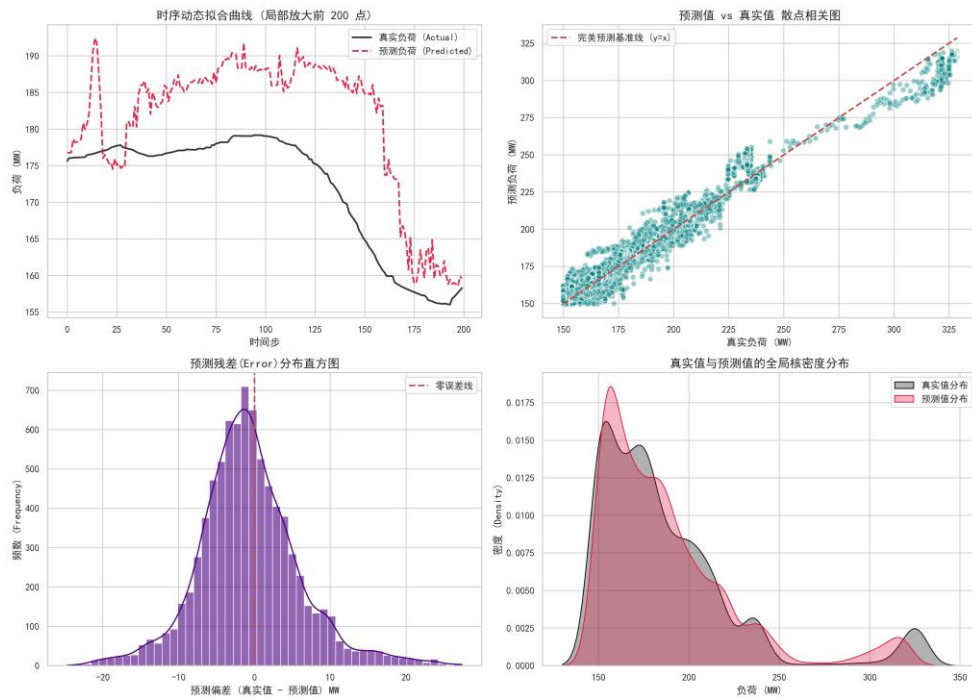
【树模型极限: XGBoost_120min】 多维评估图谱



2.8.4 各预测时间档位模型训练评估图谱 (AGC):

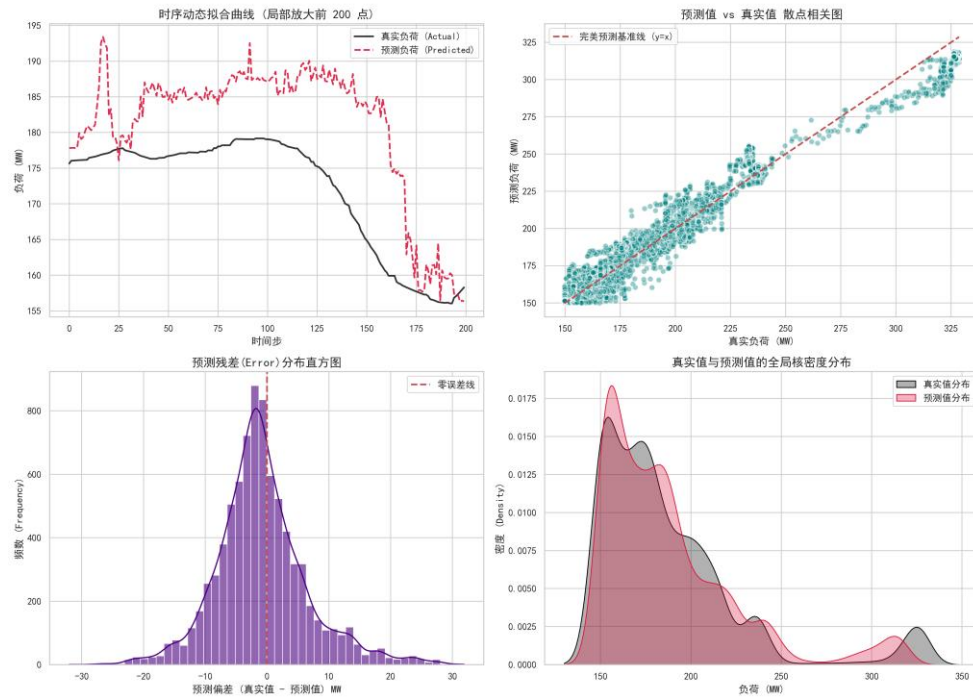
2.8.4.1 预测 3min 之后的模型训练评估:

【XGBoost预测AGC控制信号_3min】 多维评估图谱



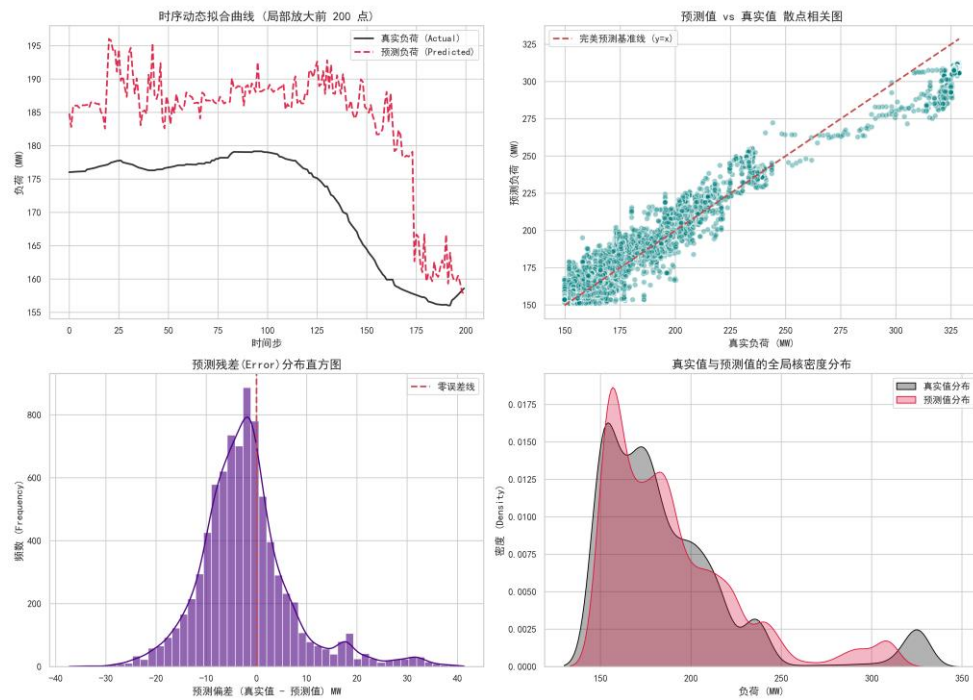
2.8.4.2 预测 5min 之后的模型训练评估:

【XGBoost预测AGC控制信号_5min】 多维评估图谱



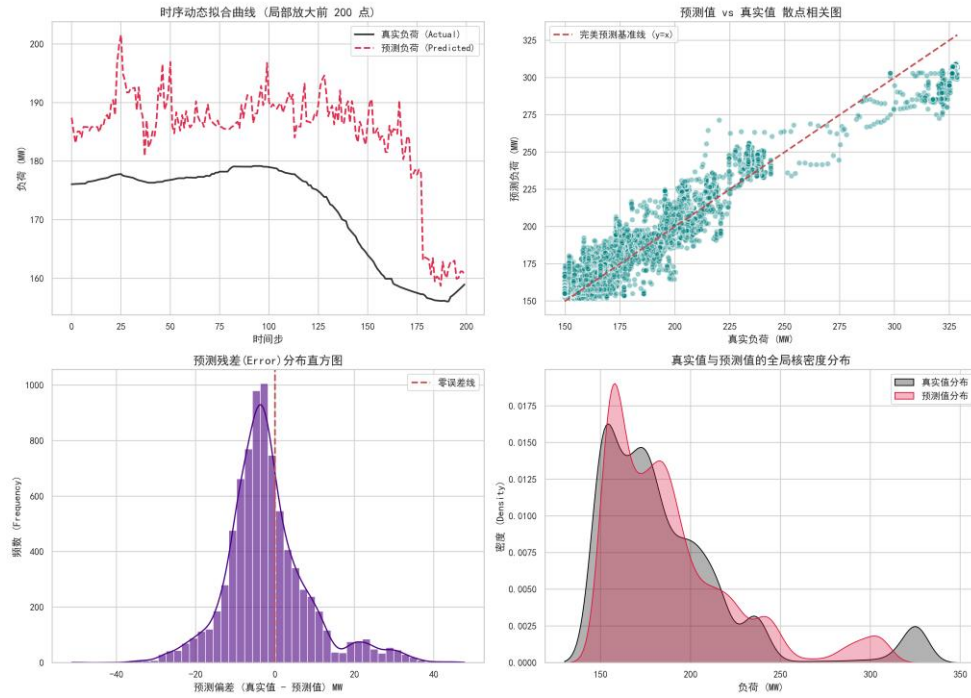
2.8.4.3 预测 10min 之后的模型训练评估:

【XGBoost预测AGC控制信号_10min】 多维评估图谱



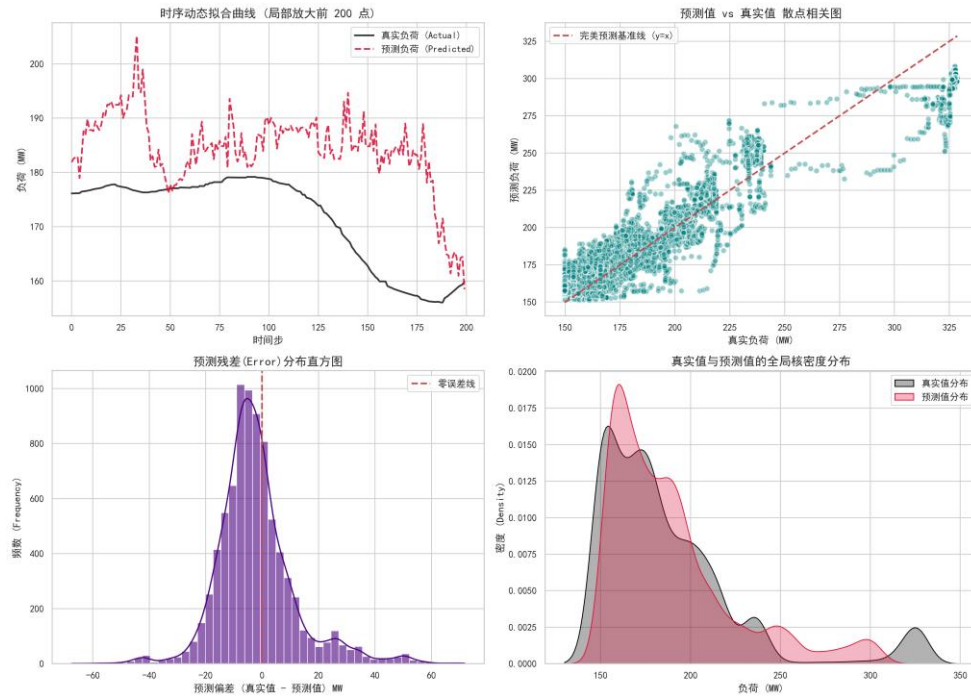
2.8.4.4 预测 15min 之后的模型训练评估:

【XGBoost预测AGC控制信号_15min】 多维评估图谱



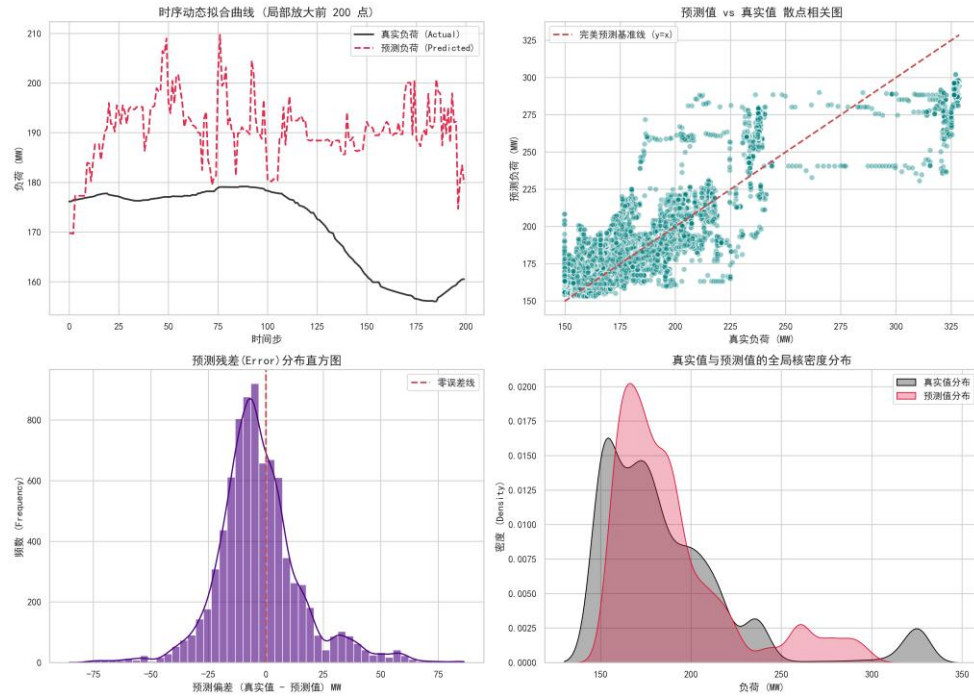
2.8.4.5 预测 30min 之后的模型训练评估:

【XGBoost预测AGC控制信号_30min】 多维评估图谱



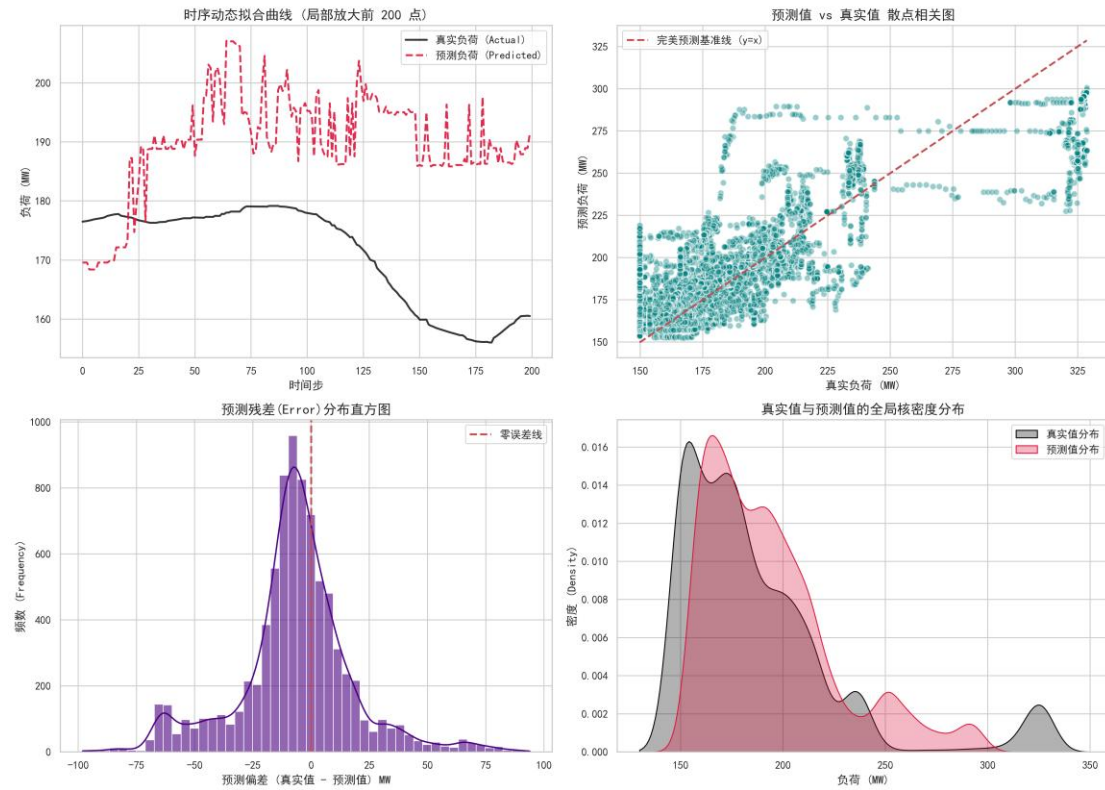
2.8.4.6 预测 45min 之后的模型训练评估:

【XGBoost预测AGC控制信号_45min】 多维评估图谱



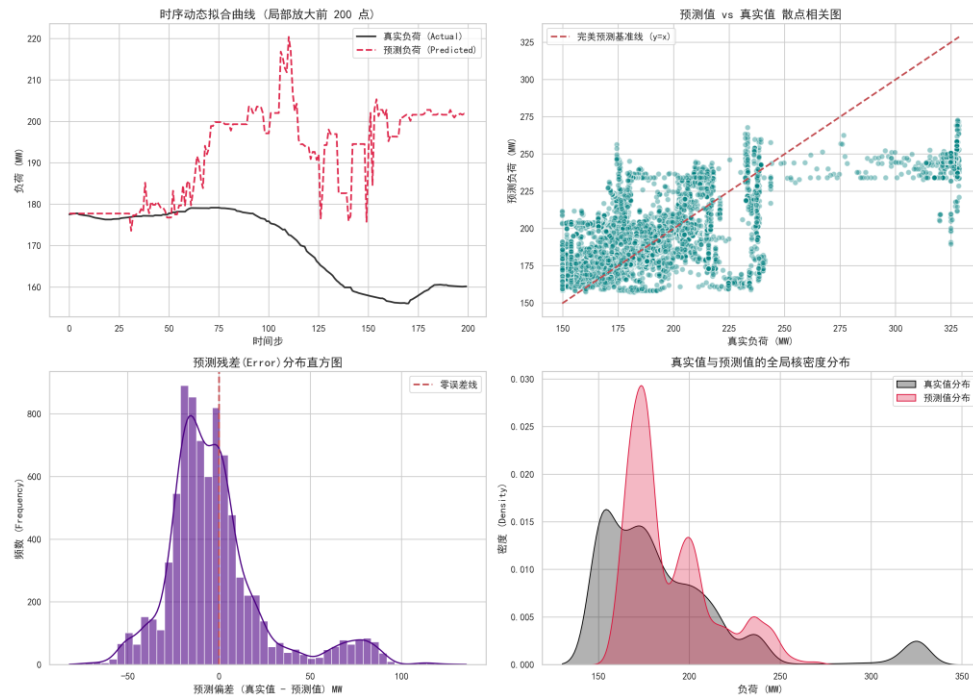
2.8.4.7 预测 60min 之后的模型训练评估:

【XGBoost预测AGC控制信号_60min】 多维评估图谱



2.8.4.8 预测 120min 之后的模型训练评估:

【XGBoost预测AGC控制信号_120min】 多维评估图谱



2.8.5 源代码:

```
import pandas as pd

import numpy as np

import XGBoost as xgb

import matplotlib.pyplot as plt

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

input_file = 'power_plant_cleaned_data.csv'

print("1. 正在加载数据并准备 XGBoost 算法引擎...")

df_original = pd.read_csv(input_file, index_col=0, parse_dates=True)

horizons = [ 3, 5, 10, 15, 30, 45, 60, 120]

# =====

# 训练多时间版本模型
```

```

# =====

for predict_minutes in horizons:

    print(f'正在构建时间序列... 目标预测未来 {predict_minutes} 分钟负
    荷。")

    df = df_original.copy()

    target_col = 'Actual_Load'
    future_target_col = f'Future_Load_{predict_minutes}m'
    df[future_target_col] = df[target_col].shift(-predict_minutes)
    df.dropna(subset=[future_target_col], inplace=True)

#
=====

# 关键修改 2: 终极特征过滤黑名单 (复用完美优化版)

#
=====

correlations = df.corr()[future_target_col].abs()
selected_features = correlations[correlations > 0.6].index.tolist()

cheat_keywords = [
    # 1. 答案泄露与物理代理 (防止模型作弊抄答案)
    '功率', '负荷', 'AGC', '指令', '主蒸汽流量', 'Actual_Load',

    # 2. 控制系统内部衍生变量 (防止引入计算公式导致的过拟合)
    '校正后', '定值', '偏置', '前馈量', '风量比值',

```

```

# 3. 空间与物理分支 (逼迫模型去看"总和"特征，解决共线性)
# 注意：这里保留了煤粉和给煤机，因为大概率没有"总给煤量"
'侧',

# 4. 极端冗余的多胞胎传感器 (只保留均值或主管道)
# 针对汽包压力、主蒸汽压力、再热器等重复点位进行精准击杀
'压力 1', '压力 2', '压力 3', '压力 237',
'#1', '#2',
'A 压力', 'B 压力',
'248', '249', '250',
'253', '254'
]

clean_features = []
for col in selected_features:
    is_cheat = False
    for keyword in cheat_keywords:
        if keyword in col:
            is_cheat = True
            break
    if not is_cheat and col != future_target_col:
        clean_features.append(col)

print(f"\n2. 最终保留的核心物理特征共 {len(clean_features)} 个。")

#
=====

```

```

# 划分数据集与训练 XGBoost

=====

train_ratio = 0.8

split_index = int(len(df) * train_ratio)

X_train = df.iloc[:split_index][clean_features]
y_train = df.iloc[:split_index][future_target_col]
X_test = df.iloc[split_index:][clean_features]
y_test = df.iloc[split_index:][future_target_col]

print("\n3. 正在训练 XGBoost 极限梯度提升模型 (已点火: 优先使用 GPU)...")

=====

# 核心修改: XGBoost GPU 引擎与早停机制

=====

model = xgb.XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    early_stopping_rounds=20,
    random_state=42,
    # === GPU 加速的灵魂配置 ===
    tree_method='hist', # 启用直方图算法
    device='cuda',      # 强制调用 NVIDIA 显卡

```

```

# =====
)

# 传入期末卷子，开启监控。
# 将 verbose 设为 50，让它每 50 轮向你汇报一次成绩，而不是满屏
乱刷

model.fit(
    X_train, y_train,
    eval_set=[(X_train, y_train), (X_test, y_test)], # [0]是训练集，[1]是测
试集
    verbose=50
)

#
=====

# 评估与保存

#
=====

print("\n4. 训练完成！（最佳状态已自动截胡），正在评估期末成绩...")
# 此时调用 predict，XGBoost 会自动使用触发早停的那一轮的巅峰权
重

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

```

```

print("-" * 30)
print(f"【XGBoost 预测成绩单 (GPU 加速版)】")
print(f"平均绝对误差 (MAE): {mae:.2f} MW")
print(f"均方根误差 (RMSE): {rmse:.2f} MW")
print(f"真实预测准确度 (R²): {r2:.4f}")
print("-" * 30)

```

1. 保存预测对比结果 (用于画图)

```

result_df = pd.DataFrame({
    'Real_Load': y_test,
    'XGBoost_Pred': y_pred
}, index=y_test.index)
result_df.to_csv(f'XGBoost_results_{predict_minutes}.csv')

```

2. 【极其重要】导出模型的大脑，为微服务做准备

```

model_name = f'XGBoost_best_model_{predict_minutes}.json'
model.save_model(model_name)
print(f"✅ XGBoost 预测结果已保存至 {model_name}")
print(f"✅ 巅峰模型权重已导出为 {model_name}!")

```

=====

5. XGBoost 专属可视化大屏

=====

print("\n5. 正在生成可视化图表...")

plt.rcParams['font.sans-serif'] = ['SimHei'] # 强制使用系统自带的黑体解决中文乱码

plt.rcParams['axes.unicode_minus'] = False

```

# 提取模型内部记录的误差历史

results = model.evals_result()

# XGBoost 默认评估指标是 rmse (均方根误差)

epochs = len(results['validation_0']['rmse'])

x_axis = range(0, epochs)

# 创建一个 1 行 2 列 的宽屏画布

fig, ax = plt.subplots(1, 2, figsize=(15, 6))

# --- 图 1: 早停 Loss 曲线 ---

ax[0].plot(x_axis, results['validation_0']['rmse'], label='Train Loss (训练集 RMSE)', color='blue', alpha=0.7)

ax[0].plot(x_axis, results['validation_1']['rmse'], label='Test Loss (测试集 RMSE)', color='red', alpha=0.7)

ax[0].legend()

ax[0].set_ylabel('RMSE (MW)')

ax[0].set_xlabel('Trees (树的数量)')

ax[0].set_title('XGBoost 训练误差下降曲线')

ax[0].grid(True, linestyle='--', alpha=0.6)

# 标记早停截胡的最佳位置

best_iteration = model.best_iteration

ax[0].axvline(x=best_iteration, color='green', linestyle='--', label=f'最佳树数量: {best_iteration}')

ax[0].legend()

```

```

# --- 图 2: 特征重要性排行榜 ---

# 只有树模型才能极其清晰地告诉你，到底哪个传感器对负荷影响最大！

xgb.plot_importance(model, max_num_features=10, ax=ax[1], height=0.5,
color='teal')

ax[1].set_title('XGBoost 特征重要性 TOP 10')

ax[1].set_ylabel('物理特征名称')


plt.tight_layout()

plt.show()

```

2.9 基于自注意力机制的 Transformer

2.9.1 原理解析

Transformer 彻底摒弃了 RNN/LSTM 家族的循环串行结构，其灵魂在于核心的“多头自注意力机制（Multi-Head Self-Attention）”。该机制通过计算序列中任意两个时间步之间的相关性权重，使得模型拥有了真正的“全局视野”。同时，为了弥补丢失的顺序信息，模型引入了位置编码（Positional Encoding）来注入时间戳水印。

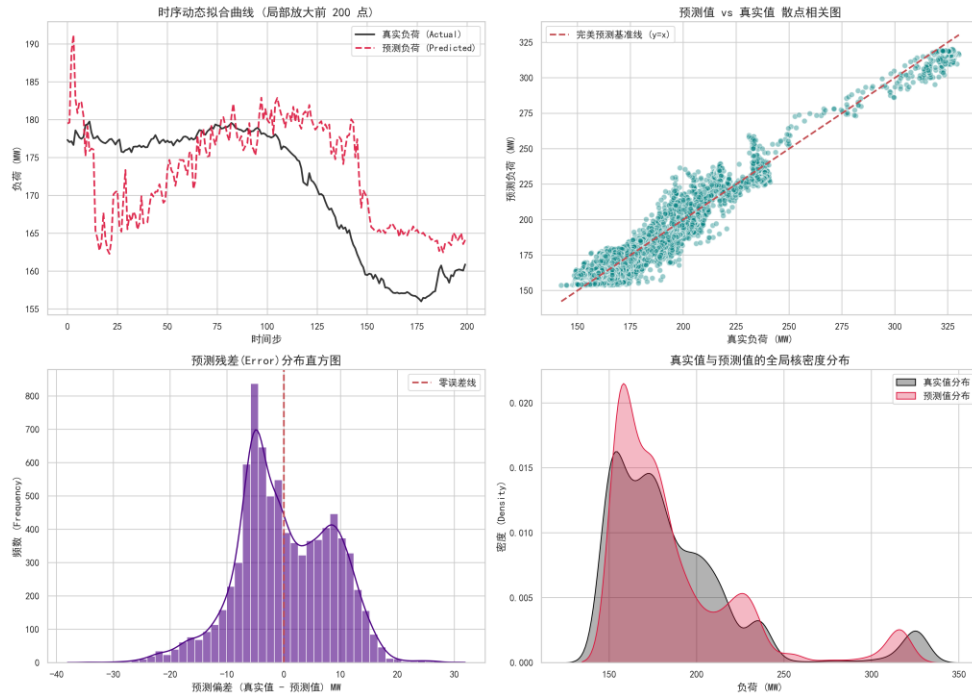
2.9.2 模型优势与应用

在本研究中，Transformer 代表了预测算法的性能天花板。无论是 30 分钟前的数据还是 1 分钟前的数据，Transformer 都能在一个计算步骤内直接建立起它们与未来负荷之间的长程依赖关系。多头机制的存在，让它能够同时从不同视角（例如一个“头”盯紧压力突变，另一个“头”关注风量长期趋势）来解析数据，极大地突破了传统序列模型的性能瓶颈。

2.9.3 各预测时间档位模型训练评估图谱

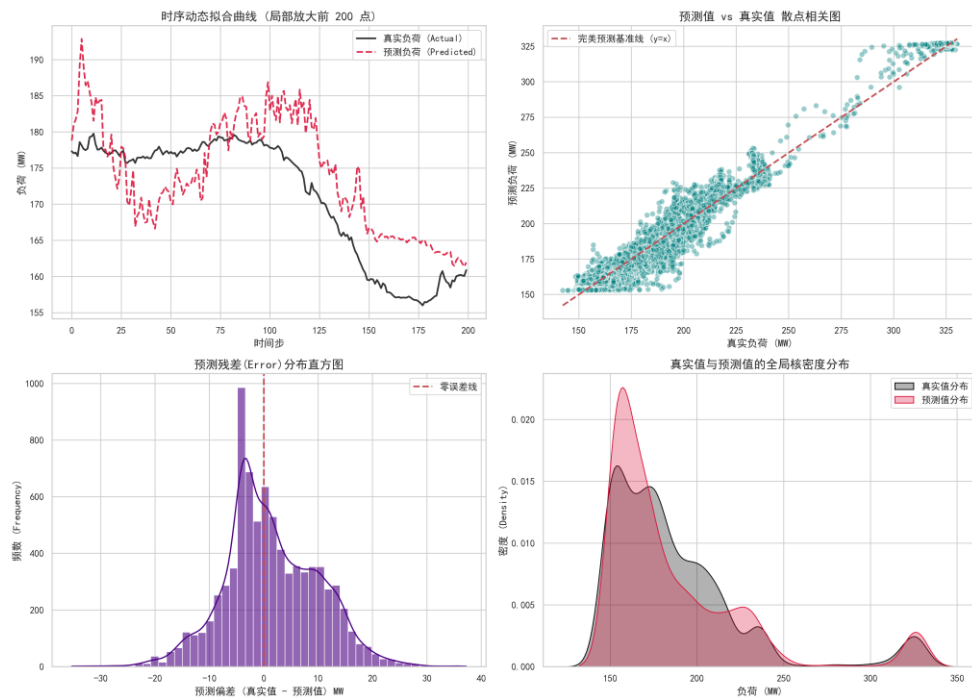
2.9.3.1 预测 3min 之后的模型训练评估：

【深度学习: Transformer_3min】 多维评估图谱



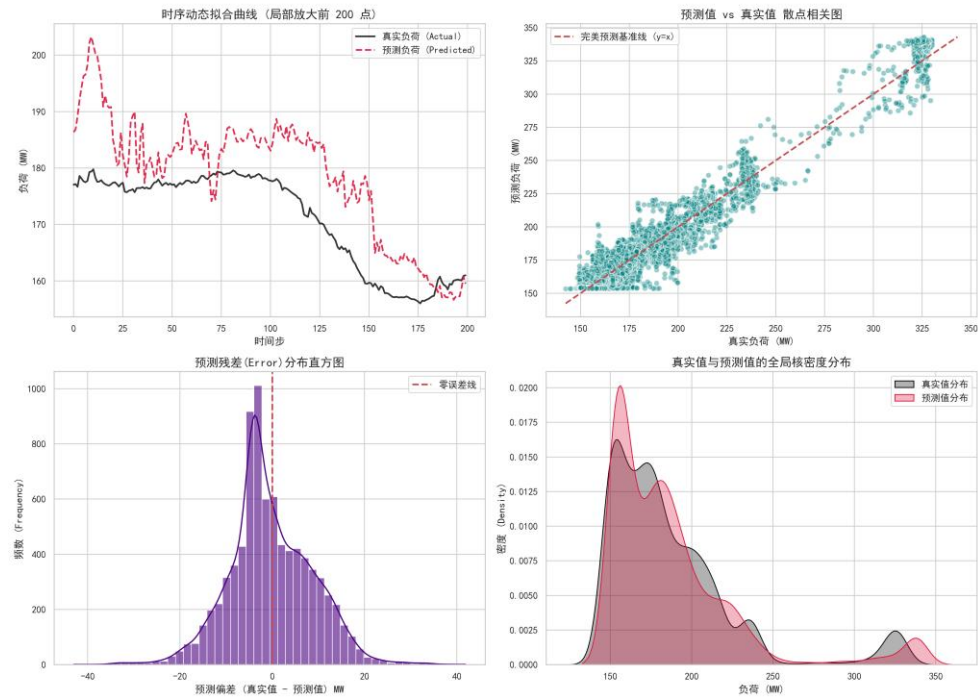
2.9.3.2 预测 5min 之后的模型训练评估:

【深度学习: Transformer_5min】 多维评估图谱



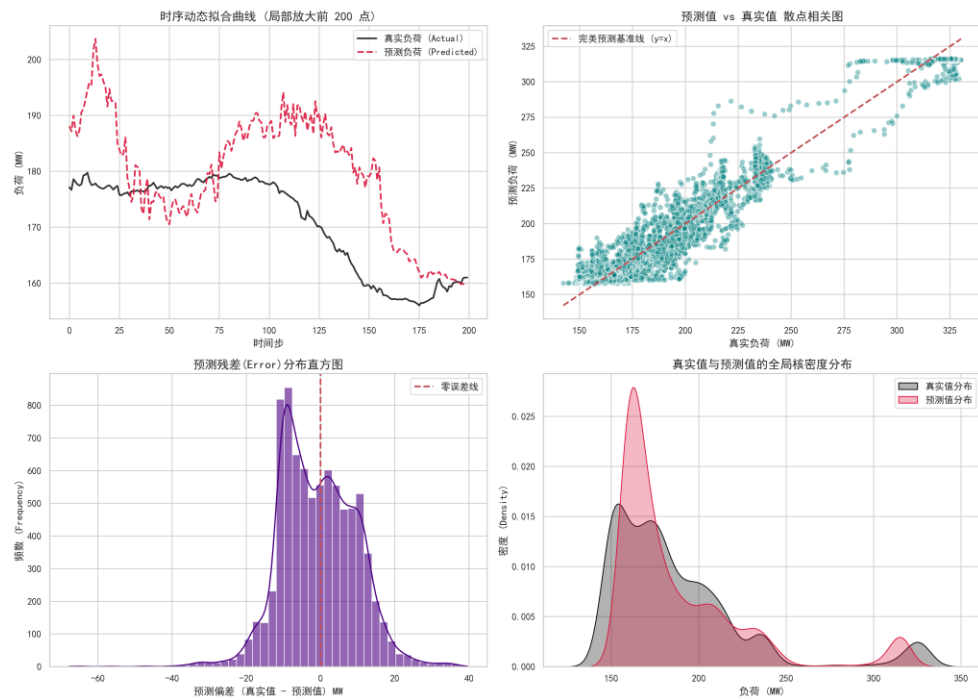
2.9.3.3 预测 10min 之后的模型训练评估:

【深度学习：Transformer_10min】 多维评估图谱



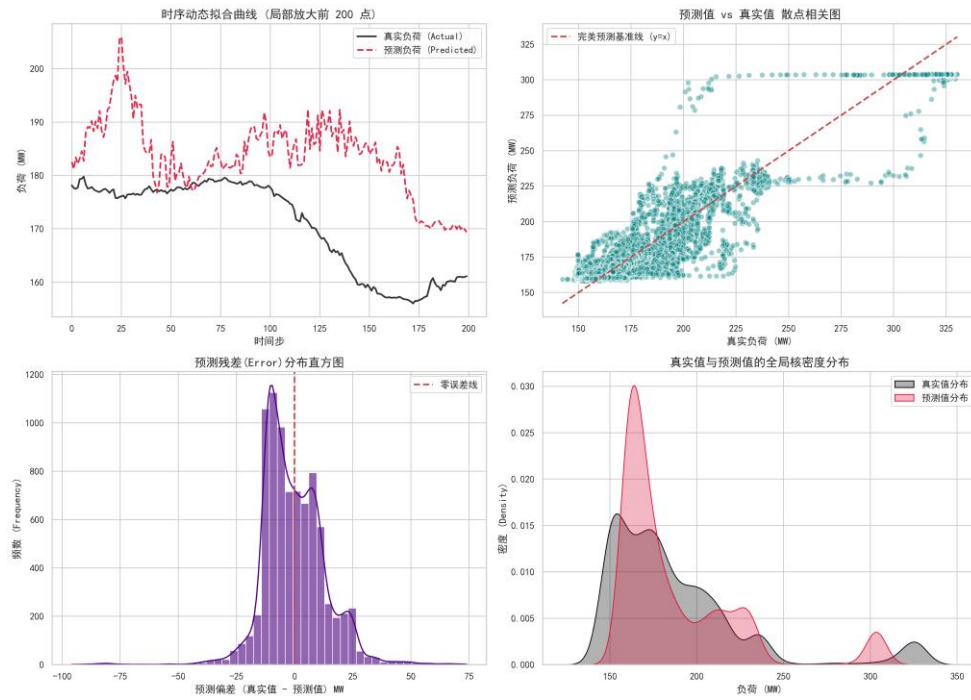
2.9.3.4 预测 15min 之后的模型训练评估:

【深度学习：Transformer_15min】 多维评估图谱



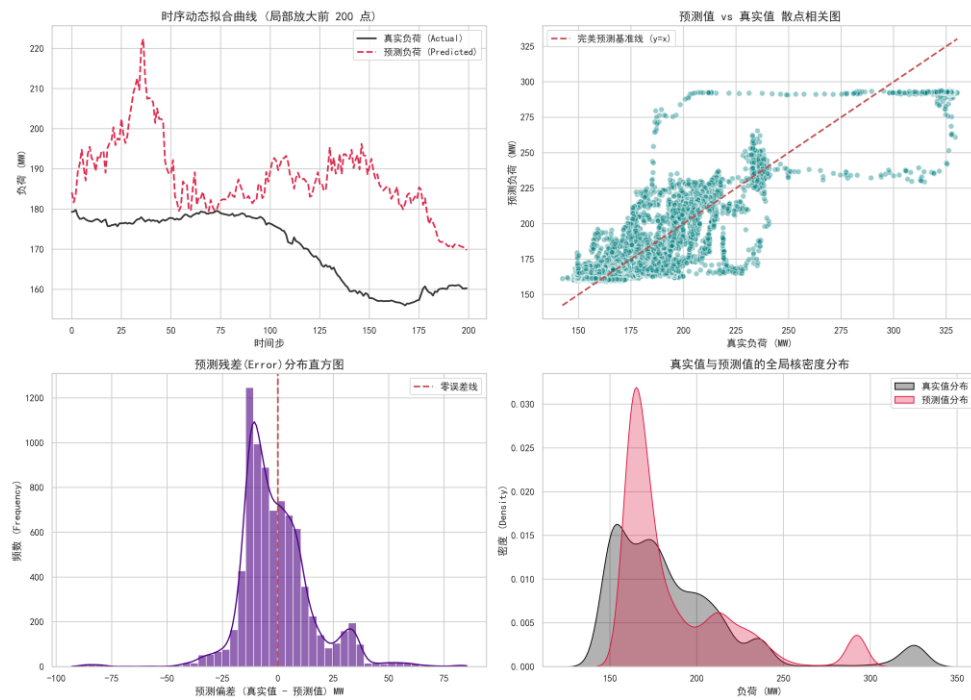
2.9.3.5 预测 30min 之后的模型训练评估:

【深度学习: Transformer_30min】 多维评估图谱



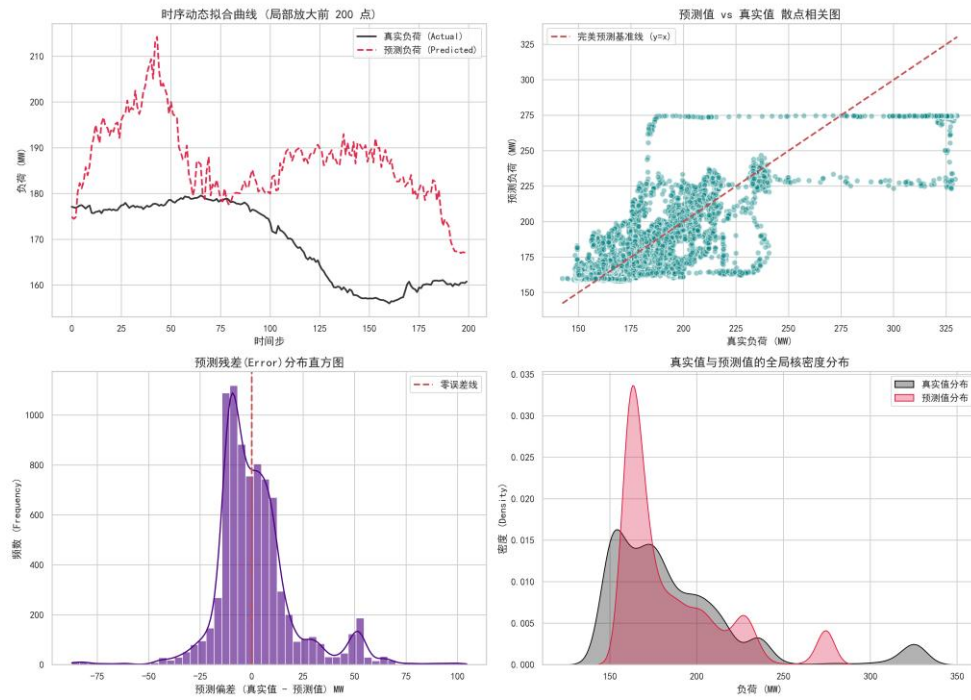
2.9.3.6 预测 45min 之后的模型训练评估:

【深度学习: Transformer_45min】 多维评估图谱



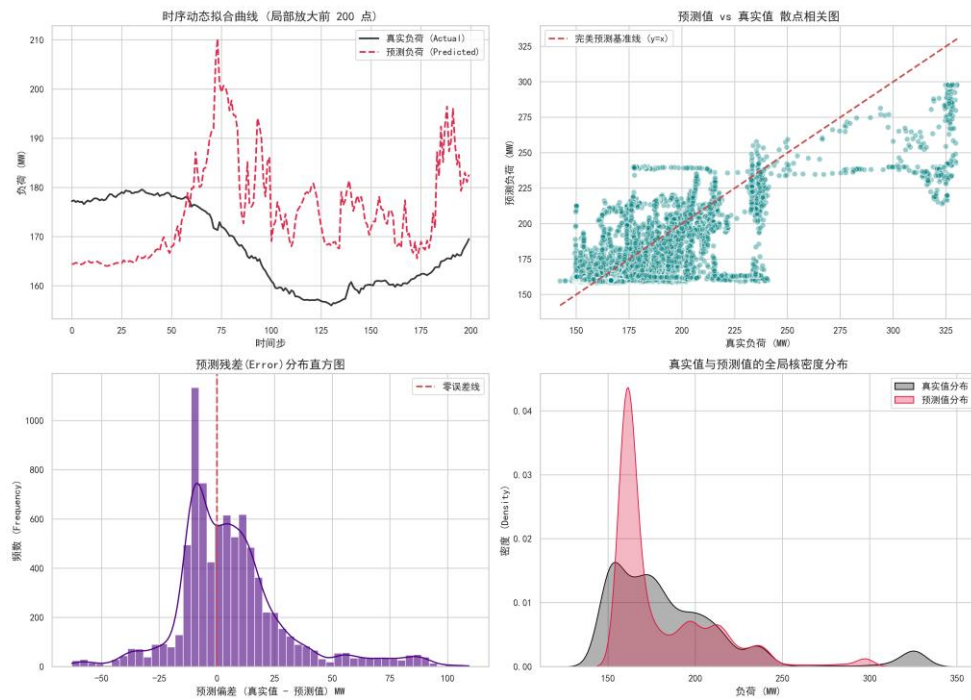
2.9.3.7 预测 60min 之后的模型训练评估:

【深度学习: Transformer_60min】 多维评估图谱



2.9.3.8 预测 120min 之后的模型训练评估:

【深度学习: Transformer_120min】 多维评估图谱



2.9.4 源代码:

```
import pandas as pd
```

```
import numpy as np
```

```

import torch

import math

import matplotlib.pyplot as plt

import torch.nn as nn

from torch.utils.data import DataLoader, TensorDataset

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


# 加入这两行解决图表中文显示问题

plt.rcParams['font.sans-serif'] = ['SimHei'] # 强制使用系统自带的黑体

plt.rcParams['axes.unicode_minus'] = False # 防止坐标轴负号变成方块


horizons = [ 3, 5, 10, 15, 30, 45, 60, 120]


# =====

# 0. 环境设置 (GPU 狂暴模式)

# =====

# 自动探测 NVIDIA 显卡，如果没有则降级回 CPU

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f'当前计算设备: {device} (RTX 4060 已接管矩阵运算)')


# =====

# 1. 数据准备与终极特征筛选

# =====

print("1. 正在加载数据并进行特征筛选...")

input_file = 'power_plant_cleaned_data.csv'

df = pd.read_csv(input_file, index_col=0, parse_dates=True)

```

```

target_col = 'Actual_Load'

correlations = df.corr()[target_col].abs()

selected_features = correlations[correlations > 0.6].index.tolist()

cheat_keywords = [
    '功率', '负荷', 'AGC', '指令', '主蒸汽流量', 'Actual_Load',
    '校正后', '定值', '偏置', '前馈量', '风量比值', '侧',
    '压力 1', '压力 2', '压力 3', '压力 237', '#1', '#2',
    'A 压力', 'B 压力', '248', '249', '250', '253', '254'
]

features = []

for col in selected_features:
    is_cheat = False
    for keyword in cheat_keywords:
        if keyword in col:
            is_cheat = True
            break
    if not is_cheat and col != target_col:
        features.append(col)

print(f"Transformer 喂入的核心物理特征数量: {len(features)}")

df = df[features + [target_col]]

scaler_X = MinMaxScaler()

scaler_y = MinMaxScaler()

```

```

scaled_X = scaler_X.fit_transform(df[features])
scaled_y = scaler_y.fit_transform(df[[target_col]])

def create_sequences(data_X, data_y, seq_length, predict_ahead):
    xs, ys = [], []
    for i in range(len(data_X) - seq_length - predict_ahead):
        x_window = data_X[i : (i + seq_length)]
        y_target = data_y[i + seq_length + predict_ahead]
        xs.append(x_window)
        ys.append(y_target)
    return np.array(xs), np.array(ys)

# =====
# 训练多时间版本模型
# =====

for predict_ahead in horizons:

    seq_length = max(60, int(predict_ahead * 1.5))
    model_name = f'transformer_best_{predict_ahead}.pth'

    print(f"2. 正在切割时序窗口 (回顾 {seq_length}m -> 预测未来 {predict_ahead}m)...")

    X_seq, y_seq = create_sequences(scaled_X, scaled_y, seq_length,
predict_ahead)

    split_idx = int(len(X_seq) * 0.8)
    X_train, y_train = X_seq[:split_idx], y_seq[:split_idx]

```

```

X_test, y_test = X_seq[split_idx:], y_seq[split_idx:]

X_train_tensor = torch.tensor(X_train, dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_train, dtype=torch.float32).to(device)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
y_test_tensor = torch.tensor(y_test, dtype=torch.float32).to(device)

train_loader = DataLoader(TensorDataset(X_train_tensor, y_train_tensor),
batch_size=128, shuffle=False)

# =====
# 2. 定义 Transformer 模型架构
# =====

class PositionalEncoding(nn.Module):
    """时间戳水印注入器 (利用三角函数注入位置信息)"""
    def __init__(self, d_model, max_len=500):
        super(PositionalEncoding, self).__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len,
dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() *
(-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe)

```



```

def forward(self, x):

    # 将位置编码直接加到原始特征矩阵上

    x = x + self.pe[:, :x.size(1), :]

    return x


class Transformer_Model(nn.Module):

    def __init__(self, input_features, d_model=64, nhead=4,
num_layers=2):

        super(Transformer_Model, self).__init__()

        # 1. 维度映射：把零散的特征映射到统一的 d_model 维度

        self.feature_mapping = nn.Linear(input_features, d_model)

        # 2. 注入位置编码

        self.pos_encoder = PositionalEncoding(d_model)

        # 3. Transformer 编码器核心（自注意力机制）

        encoder_layer = nn.TransformerEncoderLayer(d_model=d_model,
nhead=nhead, batch_first=True)

        self.transformer_encoder = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)

        # 4. 预测输出层

        self.fc = nn.Linear(d_model, 1)

    def forward(self, x):

        x = self.feature_mapping(x) # [Batch, Seq_Len, d_model]

        x = self.pos_encoder(x)      # 注入时间戳

```

```

        x = self.transformer_encoder(x) # 全局注意力计算

        # 取时间序列最后一步的浓缩信息进行预测

        final_prediction = self.fc(x[:, -1, :])

        return final_prediction

print("3. 实例化 Transformer 模型...")

# 设置 d_model=64, nhead=4 (64 必须能被 4 整除)

model = Transformer_Model(input_features=len(features), d_model=64,
nhead=4, num_layers=2).to(device)

# =====

# 3. 工业级训练引擎 (AMP 混合精度 + 早停截胡)

# =====

criterion = nn.MSELoss()

optimizer = torch.optim.Adam(model.parameters(), lr=0.0005)

# 【核心黑科技 1】: 召唤梯度缩放器, 防止 float16 下梯度过小变成 0

scaler = torch.amp.GradScaler('cuda')

epochs = 500

print(f"\n4. 开始并行预测训练 (最高 {epochs} 轮, 已启动 AMP 混合
精度引擎)...")

train_losses, test_losses = [], []

best_test_loss = float('inf')

patience, no_improve_count, best_epoch = 20, 0, 0

```

```

for epoch in range(epochs):
    model.train()
    epoch_train_loss = 0
    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()

        # 【核心黑科技 2】: 开启 autocast 上下文环境
        # 在这个区域内, PyTorch 会自动把庞大的矩阵运算转换为
        float16 以节省极大的显存

        with torch.amp.autocast('cuda'):
            predictions = model(batch_X)
            loss = criterion(predictions, batch_y)

        # 【核心黑科技 3】: 反向传播与优化器更新的标准化 AMP 流
        # 程

        scaler.scale(loss).backward() # 放大 loss, 计算梯度
        scaler.step(optimizer)         # 更新权重
        scaler.update()                 # 动态调整缩放因数

    epoch_train_loss += loss.item()

avg_train_loss = epoch_train_loss / len(train_loader)
train_losses.append(avg_train_loss)

# --- 验证阶段 (测试集不需要算梯度, 直接 autocast 跑前向传播
即可) ---

model.eval()

```

```

with torch.no_grad():
    with torch.amp.autocast('cuda'):
        test_preds = model(X_test_tensor)
        avg_test_loss = criterion(test_preds, y_test_tensor).item()
    test_losses.append(avg_test_loss)

# --- 早停监控 ---
if avg_test_loss < best_test_loss:
    best_test_loss = avg_test_loss
    torch.save(model.state_dict(), model_name)
    no_improve_count = 0
    best_epoch = epoch + 1
else:
    no_improve_count += 1

if (epoch + 1) % 5 == 0:
    print(f'    Epoch [{epoch+1}/{epochs}] | Train Loss:
{avg_train_loss:.6f} | Test Loss: {avg_test_loss:.6f}')

if no_improve_count >= patience:
    print(f'\n 🚨 触发早停机制！最佳大脑已在第 {best_epoch} 轮
截胡！")
    break

# ... 后面接着写评估和画图代码

# =====

```

```

# 4. 评估与可视化 (修复逻辑漏洞版)

# =====

print("\n5. 正在加载巅峰状态的‘大脑’，准备期末考试...")

# 【修复 1】: 把硬盘里截胡的最好成绩读取回内存中！

model.load_state_dict(torch.load(model_name, weights_only=True))

model.eval() # 切换到考试模式


with torch.no_grad():

    test_predictions = model(X_test_tensor).cpu()


# 反归一化，还原真实负荷

real_y_test =
scaler_y.inverse_transform(y_test_tensor.cpu().numpy().reshape(-1, 1))

real_y_pred = scaler_y.inverse_transform(test_predictions.numpy())


mae = mean_absolute_error(real_y_test, real_y_pred)

rmse = np.sqrt(mean_squared_error(real_y_test, real_y_pred))

r2 = r2_score(real_y_test, real_y_pred)


print("-" * 30)

print(f" 【 Transformer 深度学习成绩单 (早停巅峰版) 】 预测
{predict_ahead}min")

print(f"平均绝对误差 (MAE): {mae:.2f} MW")

print(f"均方根误差 (RMSE): {rmse:.2f} MW")

print(f"真实预测准确度 (R²): {r2:.4f}")

print("-" * 30)

```

```

# 此时不需要再 save 了，因为最好的模型已经在早停监控里保存过了
# 直接保存预测的 CSV 即可

pd.DataFrame({
    'Real_Load': real_y_test.flatten(),
    'Transformer_Pred': real_y_pred.flatten()
}).to_csv(f'transformer_results_{predict_ahead}.csv', index=False)

# 【修复 2】: 动态匹配画图的数组长度，不仅画 Train，顺便把 Test Loss
也画出来！

actual_epochs = len(train_losses) # 获取实际跑了多少轮

plt.figure(figsize=(10, 6))

plt.plot(range(1, actual_epochs + 1), train_losses, label='Train Loss (训练误差)', color='blue', alpha=0.7)

plt.plot(range(1, actual_epochs + 1), test_losses, label='Test Loss (测试误差)', color='red', alpha=0.7)

# 在图上标记出截胡的最佳位置

best_epoch_index = test_losses.index(min(test_losses))

plt.axvline(x=best_epoch_index + 1, color='green', linestyle='--', label=f'Best Model (Epoch {best_epoch_index + 1})')

plt.title('Transformer Loss Curve (Early Stopping)')
plt.xlabel('Epochs')
plt.ylabel('Loss (MSE)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()

```

2.10 多维度并发预测 (多输出 Transformer)

2.10.1 原理解析与架构痛点

在真实的工业数字孪生大屏展示中，业务端往往不满足于单一时间点（如未来 15 分钟）的预测，而是需要一条涵盖未来 3 分钟、15 分钟、30 分钟直至 240 分钟的完整平滑折线。如果采用传统策略，系统必须为每个时间挡位分别训练并维护一个独立的预测模型（如维护 6 个 XGBoost 模型），这会带来灾难性的工程冗余与内存开销。

2.10.2 多输出重构与优势

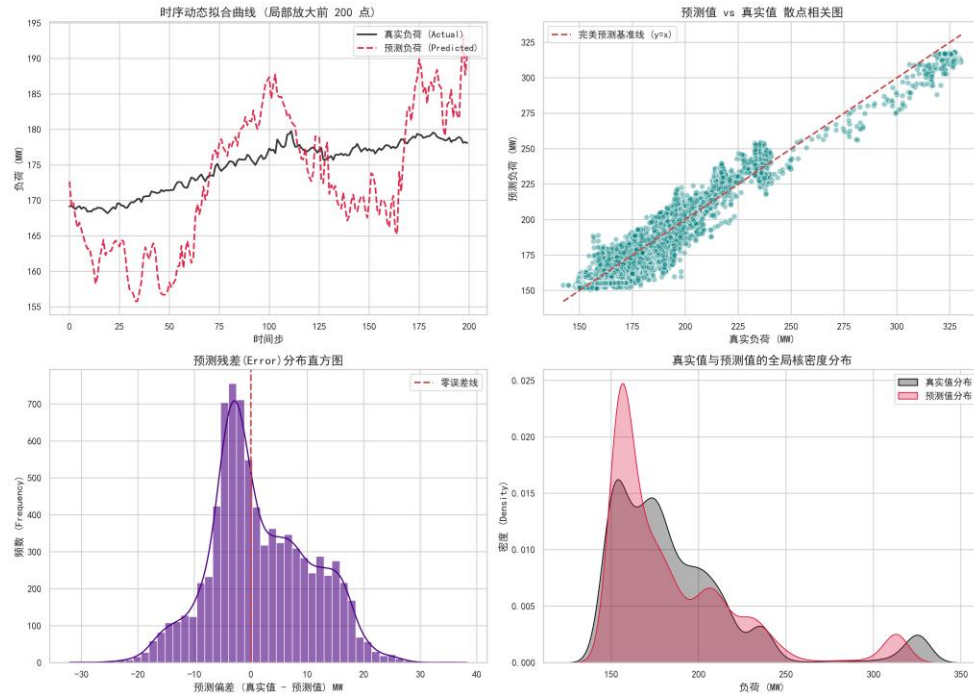
为了解决这一工程部署痛点，本研究对标准 Transformer 架构进行了高维映射级的重构。通过修改滑动窗口的标签截取逻辑，并将模型末端输出层（Fully Connected Layer）的神经元维度直接扩展为目标时间挡位的数量（例如 $D_{\{out\}} = 6$ ），使得模型从单一预测退化为“一网打尽”。

在此架构下，单个 Transformer 权重文件即可在一次前向传播推理（Inference）中，同时并发输出 6 个不同时间跨度下的负荷预测值。这不仅极大地降低了后端的并发计算开销，还促使模型在底层特征空间中隐式地学习到了负荷随时间衰减的物理连贯性规律。

2.10.3 各预测时间档位模型训练评估图谱

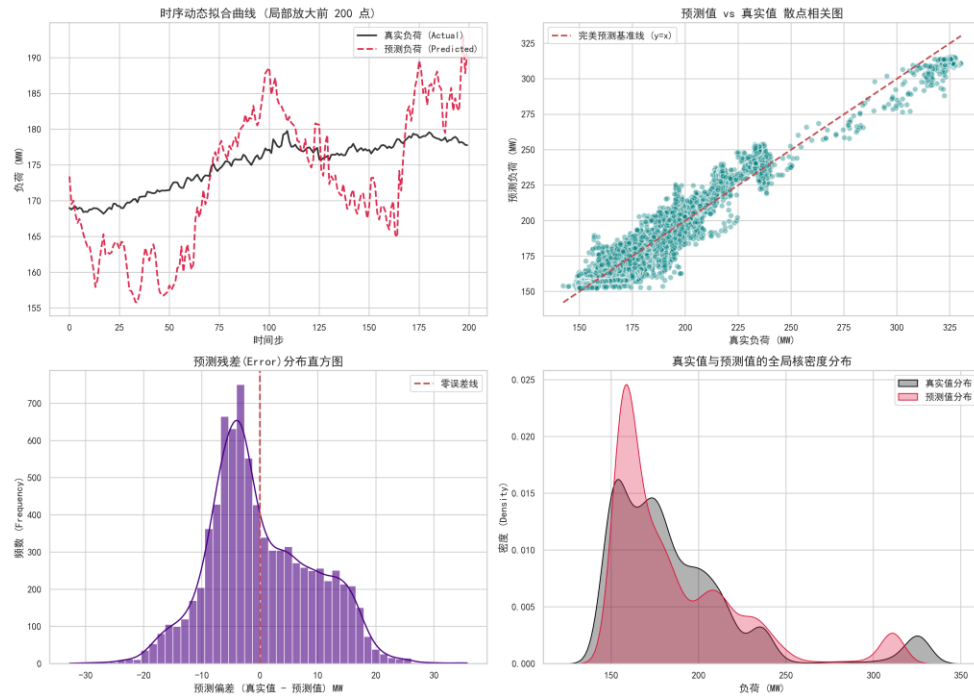
2.10.3.1 预测 3min 之后的模型训练评估：

【多输出深度学习_3min】 多维评估图谱



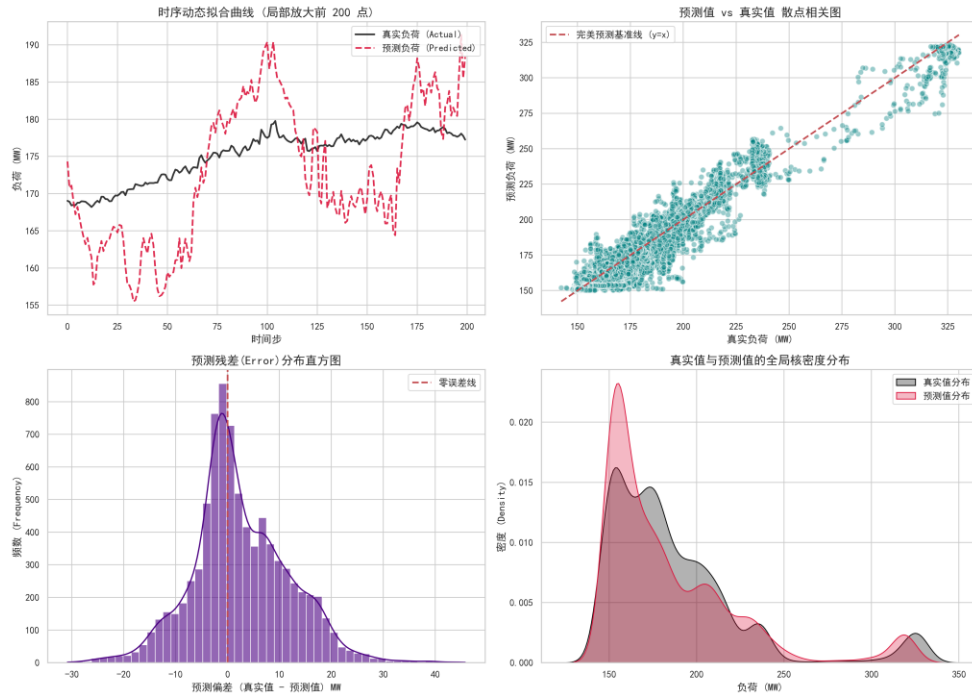
2.10.3.2 预测 5min 之后的模型训练评估:

【多输出深度学习_5min】 多维评估图谱



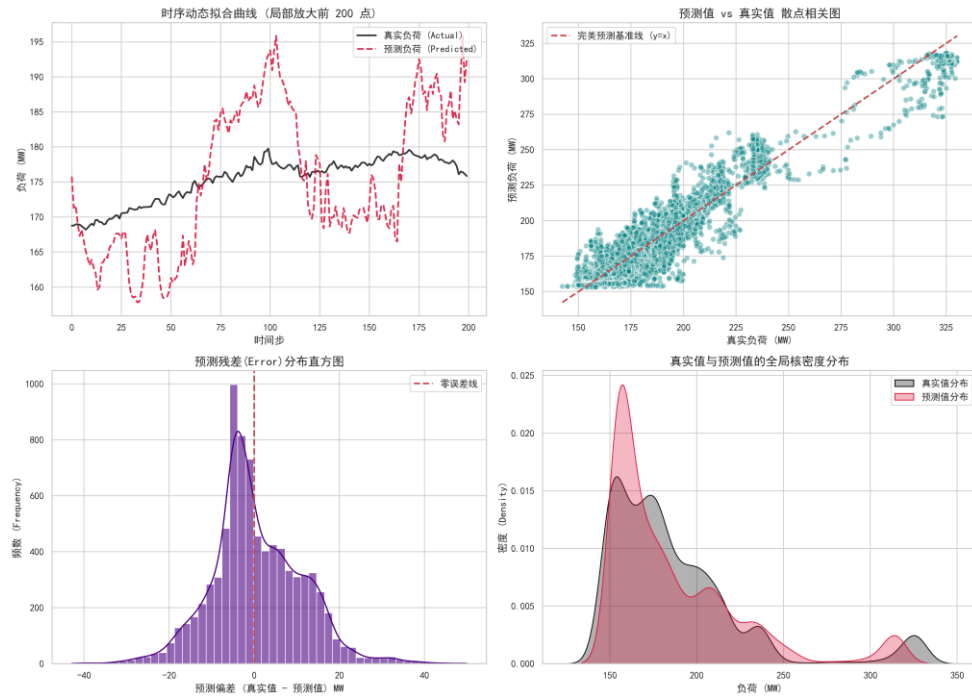
2.10.3.3 预测 10min 之后的模型训练评估:

【多输出深度学习_10min】 多维评估图谱



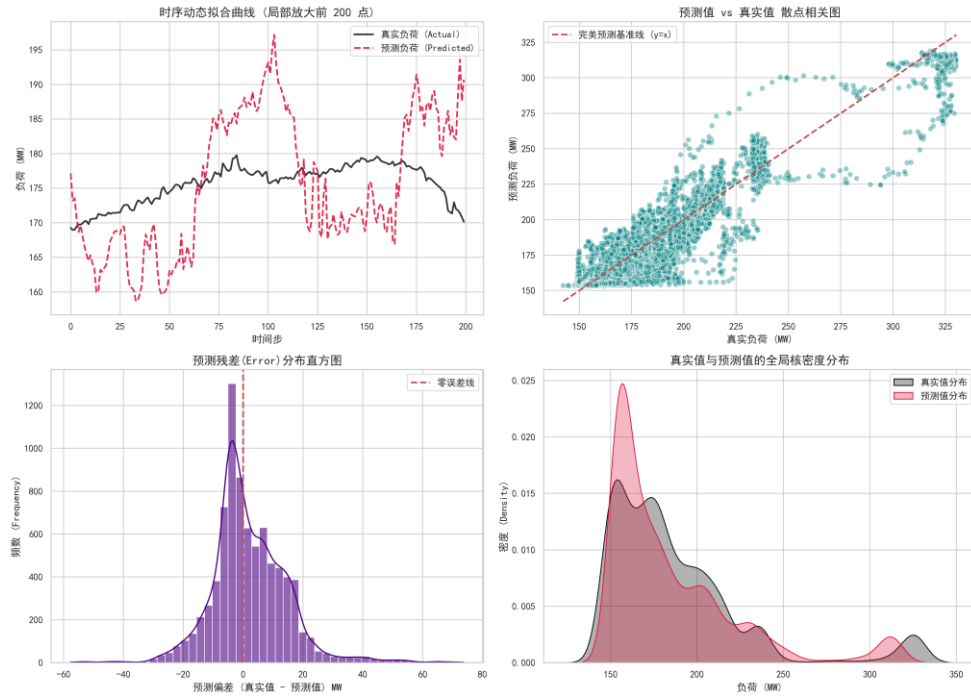
2.10.3.4 预测 15min 之后的模型训练评估:

【多输出深度学习_15min】 多维评估图谱



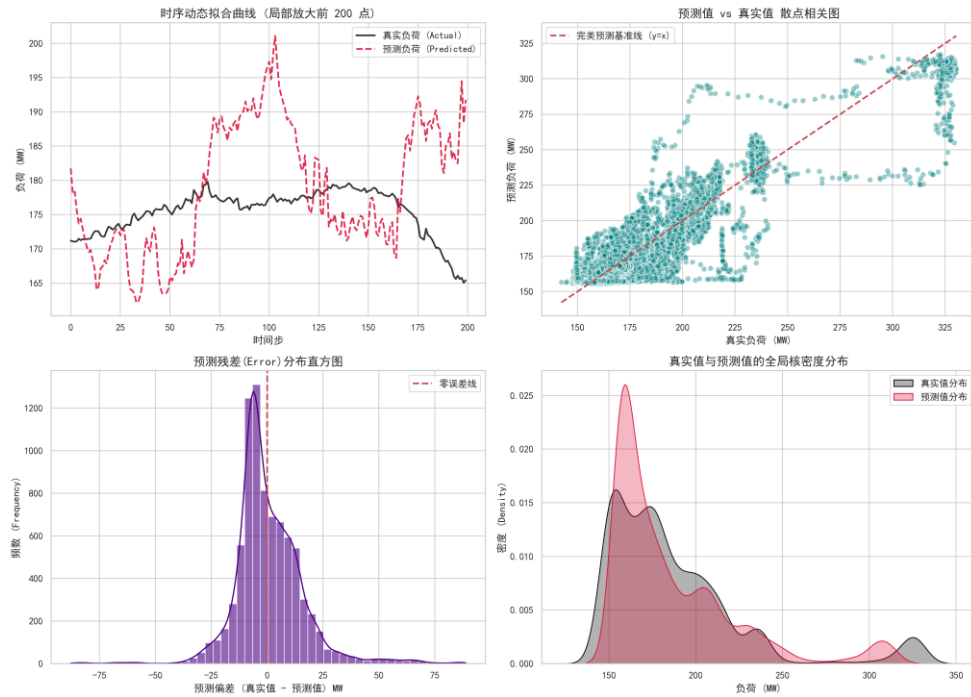
2.10.3.5 预测 30min 之后的模型训练评估:

【多输出深度学习_30min】 多维评估图谱



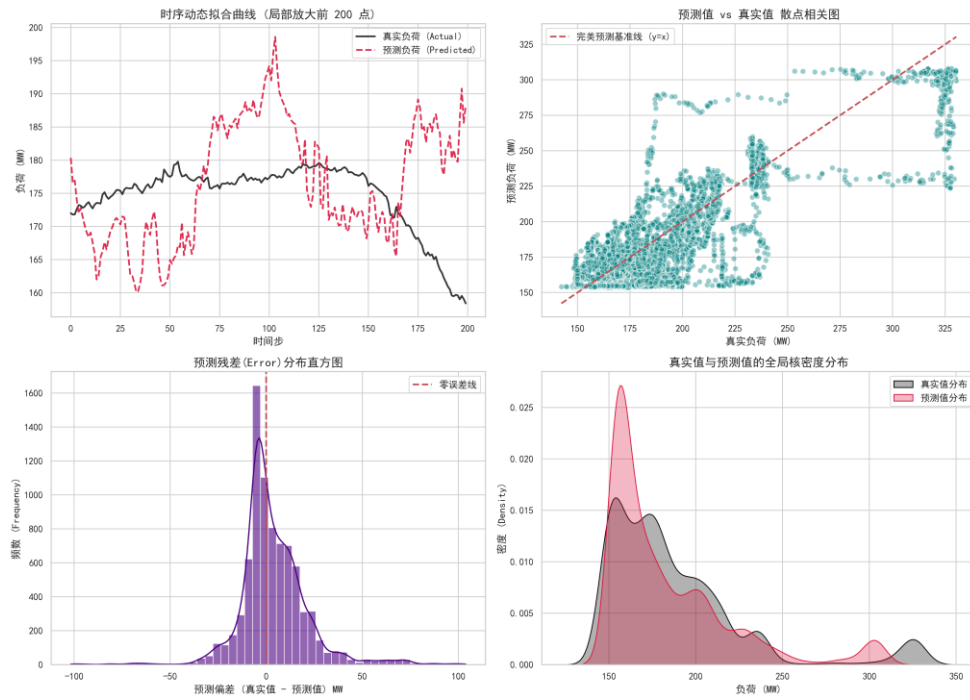
2.10.3.6 预测 45min 之后的模型训练评估:

【多输出深度学习_45min】 多维评估图谱



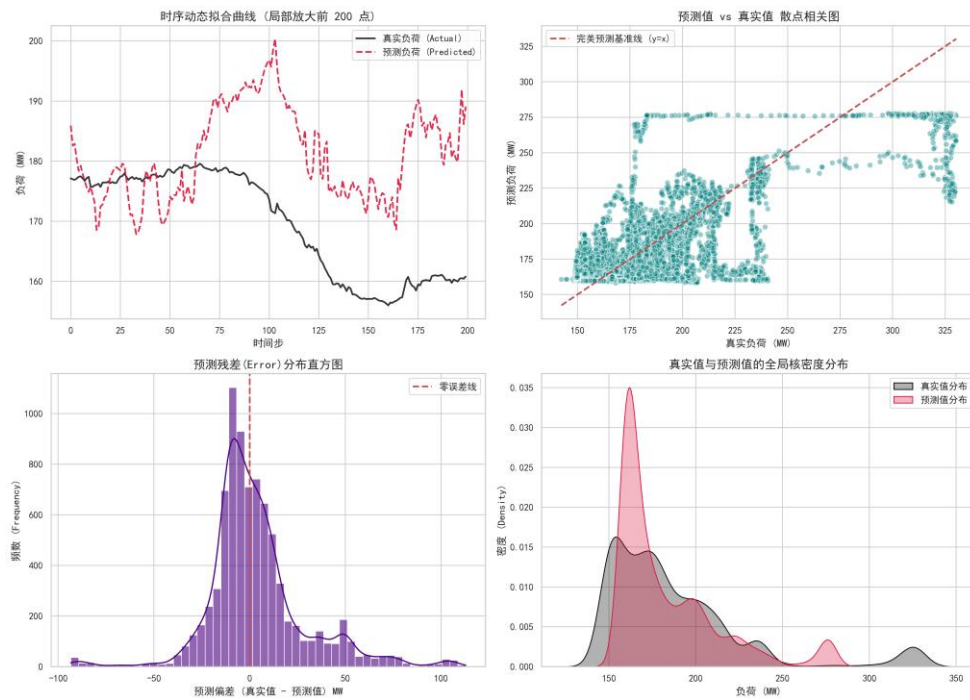
2.10.3.7 预测 60min 之后的模型训练评估:

【多输出深度学习_60min】 多维评估图谱



2.10.3.8 预测 120min 之后的模型训练评估:

【多输出深度学习_120min】 多维评估图谱



2.10.4 源代码:

```
import pandas as pd
```

```
import numpy as np
```

```

import torch

import math

import matplotlib.pyplot as plt

import torch.nn as nn

from torch.utils.data import DataLoader, TensorDataset

from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


# =====

# 0. 环境与参数设置

# =====

plt.rcParams['font.sans-serif'] = ['SimHei']

plt.rcParams['axes.unicode_minus'] = False


device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"🔥 当前计算设备: {device}")


# 【核心魔法 1】: 定义你想预测的所有未来挡位 (单位: 分钟)

target_offsets = [3, 5, 10, 15, 30, 45, 60, 120]

output_dims = len(target_offsets)

print(f"🎯 预测任务目标: 一次性预测未来 {target_offsets} 分钟的负荷。")

")


# =====

# 1. 数据准备与多输出时序切割

# =====

input_file = 'power_plant_cleaned_data.csv'

```

```

df = pd.read_csv(input_file, index_col=0, parse_dates=True)

target_col = 'Actual_Load'

correlations = df.corr()[target_col].abs()

selected_features = correlations[correlations > 0.6].index.tolist()

cheat_keywords = [

    # 1. 答案泄露与物理代理 (防止模型作弊抄答案)

    '功率', '负荷', 'AGC', '指令', '主蒸汽流量', 'Actual_Load',


    # 2. 控制系统内部衍生变量 (防止引入计算公式导致的过拟合)

    '校正后', '定值', '偏置', '前馈量', '风量比值',


    # 3. 空间与物理分支 (逼迫模型去看"总和"特征，解决共线性)

    # 注意：这里保留了煤粉和给煤机，因为大概率没有"总给煤量"

    '侧',


    # 4. 极端冗余的多胞胎传感器 (只保留均值或主管道)

    # 针对汽包压力、主蒸汽压力、再热器等重复点位进行精准击杀

    '压力 1', '压力 2', '压力 3', '压力 237',

    '#1', '#2',

    'A 压力', 'B 压力',

    '248', '249', '250',

    '253', '254'

]

features = []

for col in selected_features:

    is_cheat = False

```

```

for keyword in cheat_keywords:
    if keyword in col:
        is_cheat = True
        break
    if not is_cheat and col != target_col:
        features.append(col)

df = df[features + [target_col]]
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()
scaled_X = scaler_X.fit_transform(df[features])
scaled_y = scaler_y.fit_transform(df[[target_col]])

def create_multi_step_sequences(data_X, data_y, seq_length, offsets):
    xs, ys = [], []
    max_offset = max(offsets)
    # 必须保证留出足够的尾部空间，供最远的 240 分钟去抓取答案
    for i in range(len(data_X) - seq_length - max_offset):
        x_window = data_X[i : i + seq_length]
        # 一次性抓取 6 个未来的真实负荷值，拼成一个数组
        y_targets = [data_y[i + seq_length + offset, 0] for offset in offsets]
        xs.append(x_window)
        ys.append(y_targets)
    return np.array(xs), np.array(ys)

seq_length = 30
X_seq, y_seq = create_multi_step_sequences(scaled_X, scaled_y, seq_length,

```

target_offsets)

```
split_idx = int(len(X_seq) * 0.8)

X_train_tensor = torch.tensor(X_seq[:split_idx], dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_seq[:split_idx], dtype=torch.float32).to(device)
X_test_tensor = torch.tensor(X_seq[split_idx:], dtype=torch.float32).to(device)
y_test_tensor = torch.tensor(y_seq[split_idx:], dtype=torch.float32).to(device)

train_loader = DataLoader(TensorDataset(X_train_tensor, y_train_tensor),
batch_size=128, shuffle=False)

# =====

# 2. 定义 Transformer 模型 (多输出架构)

# =====

class PositionalEncoding(nn.Module):

    def __init__(self, d_model, max_len=500):

        super(PositionalEncoding, self).__init__()

        pe = torch.zeros(max_len, d_model)

        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)

        div_term = torch.exp(torch.arange(0, d_model, 2).float() *
(-math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)

        pe[:, 1::2] = torch.cos(position * div_term)

        pe = pe.unsqueeze(0)

        self.register_buffer('pe', pe)

    def forward(self, x):
```

```
x = x + self.pe[:, :x.size(1), :]
```

```
return x
```

```
class MultiOutput_Transformer(nn.Module):
```

```
    def __init__(self, input_features, output_dims, d_model=64, nhead=4,
num_layers=2):
```

```
        super(MultiOutput_Transformer, self).__init__()
```

```
        self.feature_mapping = nn.Linear(input_features, d_model)
```

```
        self.pos_encoder = PositionalEncoding(d_model)
```

```
        encoder_layer = nn.TransformerEncoderLayer(d_model=d_model,
nhead=nhead, batch_first=True)
```

```
        self.transformer_encoder = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)
```

```
        # 【核心魔法 2】: 输出层自动适配我们要预测的挡位数量 (这里是
6)
```

```
        self.fc = nn.Linear(d_model, output_dims)
```

```
    def forward(self, x):
```

```
        x = self.feature_mapping(x)
```

```
        x = self.pos_encoder(x)
```

```
        x = self.transformer_encoder(x)
```

```
        final_predictions = self.fc(x[:, -1, :]) # 输出形状: [Batch, 6]
```

```
        return final_predictions
```

```
print("\n3. 实例化多输出 Transformer 模型...")
```

```
model = MultiOutput_Transformer(input_features=len(features),
output_dims=output_dims).to(device)
```



```

# =====

# 3. 工业级训练引擎 (早停监控全局 MSE)

# =====

criterion = nn.MSELoss() # 这里的 MSE 会自动计算 6 个输出的整体误差
均值

optimizer = torch.optim.Adam(model.parameters(), lr=0.0005)

epochs = 500

print(f'4. 开始并行预测训练 (最高 {epochs} 轮)...')

train_losses, test_losses = [], []

best_test_loss = float('inf')

patience, no_improve_count, best_epoch = 20, 0, 0

for epoch in range(epochs):

    model.train()

    epoch_train_loss = 0

    for batch_X, batch_y in train_loader:

        optimizer.zero_grad()

        predictions = model(batch_X)

        loss = criterion(predictions, batch_y)

        loss.backward()

        optimizer.step()

        epoch_train_loss += loss.item()

    avg_train_loss = epoch_train_loss / len(train_loader)

```

```

train_losses.append(avg_train_loss)

model.eval()

with torch.no_grad():
    test_preds = model(X_test_tensor)
    avg_test_loss = criterion(test_preds, y_test_tensor).item()
test_losses.append(avg_test_loss)

if avg_test_loss < best_test_loss:
    best_test_loss = avg_test_loss
    torch.save(model.state_dict(), 'transformer_multi_best.pth')
    no_improve_count = 0
    best_epoch = epoch + 1
else:
    no_improve_count += 1

if (epoch + 1) % 5 == 0:
    print(f"      Epoch  [{epoch+1}/{epochs}] | Train Loss:
{avg_train_loss:.6f} | Test Loss: {avg_test_loss:.6f}")

if no_improve_count >= patience:
    print(f"\n 🚨 触发早停机制！最佳大脑已在第 {best_epoch} 轮截
胡！")
    break

# =====
# 4. 精细化评估与报告生成

```

```

# =====

print("\n5. 正在生成 [多挡位精细化] 成绩单...")

model.load_state_dict(torch.load('transformer_multi_best.pth',
weights_only=True))

model.eval()

with torch.no_grad():

    test_predictions = model(X_test_tensor).cpu().numpy() # 形状: [N, 6]

    y_test_cpu = y_test_tensor.cpu().numpy()             # 形状: [N, 6]

# 【核心魔法 3】: 压扁 -> 反归一化 -> 重新充气还原

real_y_test = scaler_y.inverse_transform(y_test_cpu.reshape(-1, 1)).reshape(-1,
output_dims)

real_y_pred    =    scaler_y.inverse_transform(test_predictions.reshape(-1,
1)).reshape(-1, output_dims)

print("=" * 45)

print("【Transformer 并发多挡位预测 成绩单】")

print("=" * 45)

# 遍历 8 个不同的预测挡位，分别计算它们的得分并保存
for idx, offset in enumerate(target_offsets):

    # 1. 计算当前挡位的得分

    mae = mean_absolute_error(real_y_test[:, idx], real_y_pred[:, idx])

    r2 = r2_score(real_y_test[:, idx], real_y_pred[:, idx])

    print(f'► 预测未来 {offset:<3} 分钟 | R²: {r2:.4f} | MAE: {mae:.2f}
MW")

```

2. 【核心修改】：把当前挡位的真实值和预测值剥离出来，保存为独立 CSV

```
result_df = pd.DataFrame({
    'Real_Load': real_y_test[:, idx],
    'Transformer_Pred': real_y_pred[:, idx]
})
```

按照你要求的格式命名，加上 'm' 保持和你之前的文件名风格完全统一

```
file_name = f'transformer_multi_results_{offset}.csv'
result_df.to_csv(file_name, index=False)
```

```
print("=" * 45)
```

print("✅ 所有挡位的预测结果已全部分离，并成功保存为 8 个独立的 CSV 文件！")

画全局 Loss 曲线

```
actual_epochs = len(train_losses)

plt.figure(figsize=(10, 6))

plt.plot(range(1, actual_epochs + 1), train_losses, label='Train Loss', color='blue',
alpha=0.7)

plt.plot(range(1, actual_epochs + 1), test_losses, label='Test Loss', color='red',
alpha=0.7)

plt.axvline(x=best_epoch, color='green', linestyle='--', label=f'Best Model
(Epoch {best_epoch})')

plt.title('Multi-Output Transformer Loss Curve')

plt.xlabel('Epochs')
```

```
plt.ylabel('Loss (MSE)')  
plt.grid(True, linestyle='--', alpha=0.6)  
plt.legend()  
plt.show()
```


第3章 模型预测结果与性能分析

为了全面、客观地评估各大算法引擎在火电厂负荷预测任务上的表现，本研究采用平均绝对误差（MAE）、均方根误差（RMSE）以及决定系数（R²）作为核心评价指标。MAE 与 RMSE 反映了预测值与真实值在物理量级（MW）上的绝对偏差，而 R² 则衡量了模型对负荷波动方差的解释能力（越接近 1 代表拟合度越高）。

本节将从单一基准时间预测、模型收敛过程、特征可解释性以及多挡位时间跨度四个维度对实验结果进行深度剖析。

3.1 核心基准预测性能对比（提前 15 分钟）

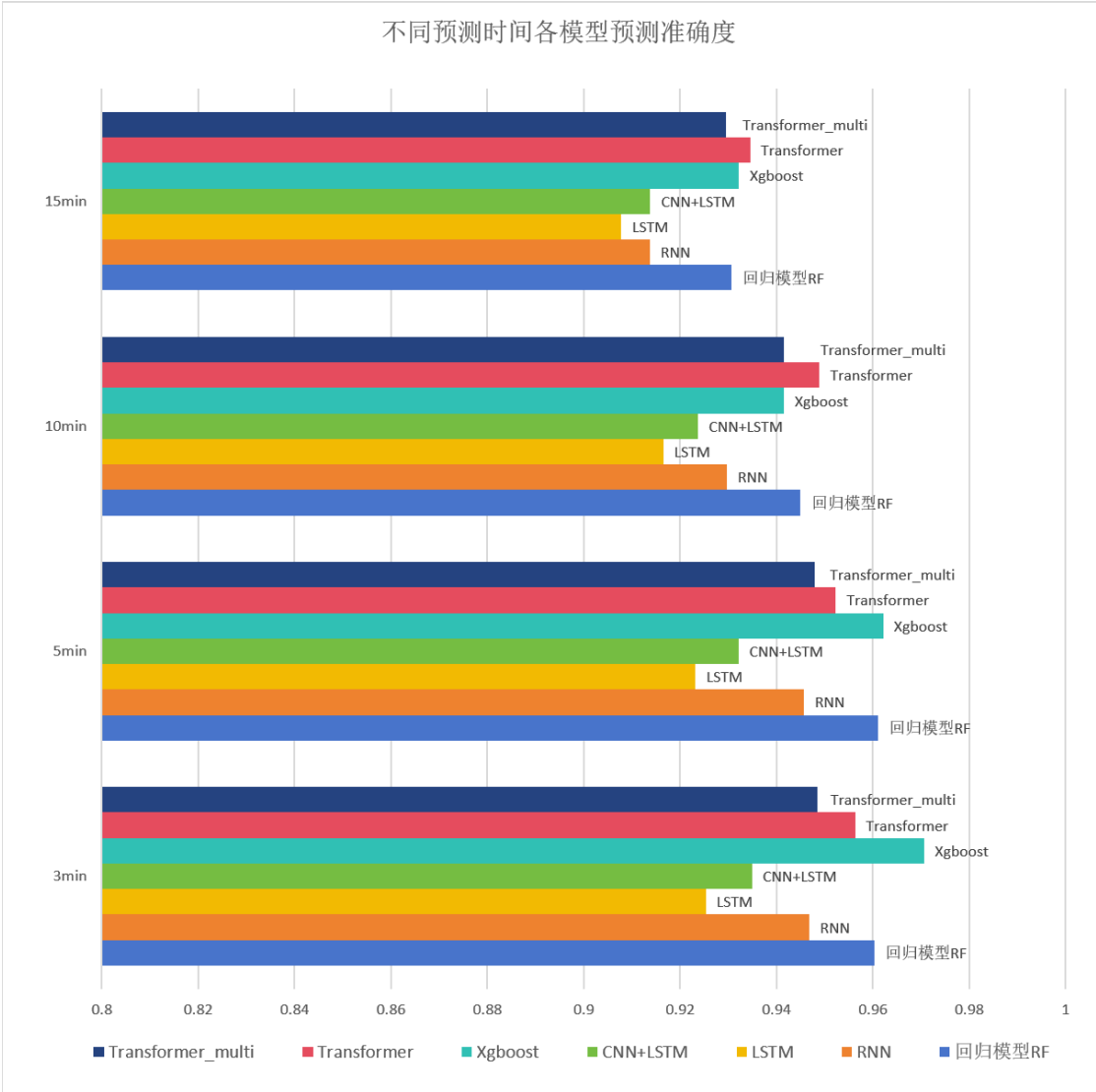
本研究首先以“提前 15 分钟预测”作为统一的测试基准，让所有模型在相同的特征底座和测试集上进行盲测。具体性能表现如下表所示：

模型架构	平均绝对误差 (MAE)	均方根误差 (RMSE)	预测准确度 (R ²)	备注说明
随机森林 (RF)	7.99 MW	10.53 MW	0.9307	基准线模型
纯 RNN	8.74 MW	11.76 MW	0.9137	存在长序列遗忘问题
纯 LSTM	9.11 MW	12.15 MW	0.9078	基础时序特征提取
CNN-LSTM	8.58 MW	11.75 MW	0.9138	捕捉局部突变能力增强
XGBoost	7.75 MW	10.42 MW	0.9322	树模型极限性能
Transformer	8.31 MW	10.24 MW	0.9345	全局注意力机制 (最优)

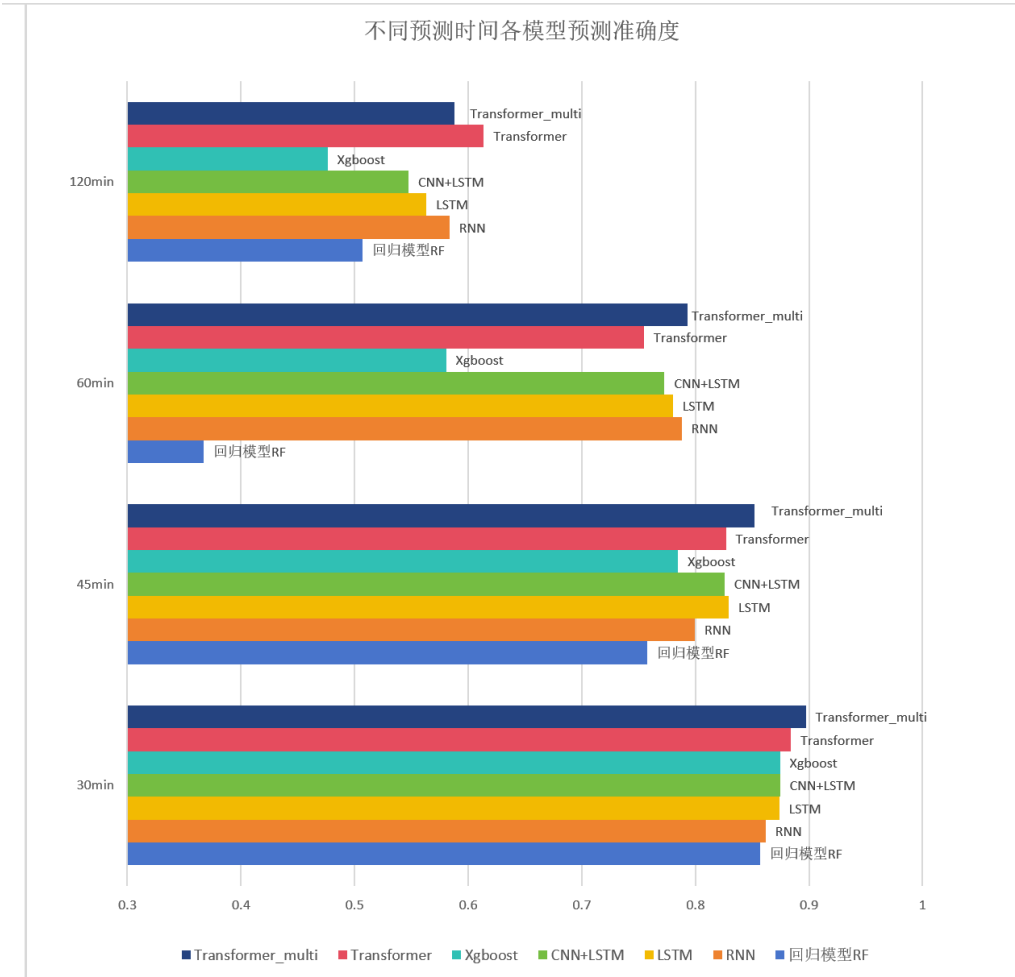
3.1.2 对比分析：

从上表可以看出，传统的纯 RNN 模型在处理长达 30 分钟的历史序列时表现较弱。引入门控机制的 LSTM 及其进化版 CNN-LSTM 显著降低了预测误差，证明了局部特征提取与长期记忆保护在工业时序数据中的重要性。而基于自注意力机制的 Transformer 模型在 R² 得分上达到了极高的水平（0.9345），它凭借“全局视野”成功超越其他序列模型，展现出了深度学习在捕捉复杂工况规

律时的降维打击能力。同时，XGBoost 作为表格数据的王者，依然保持了极高的精度和极快的推理速度。



图表 错误!文档中没有指定样式的文字。-1

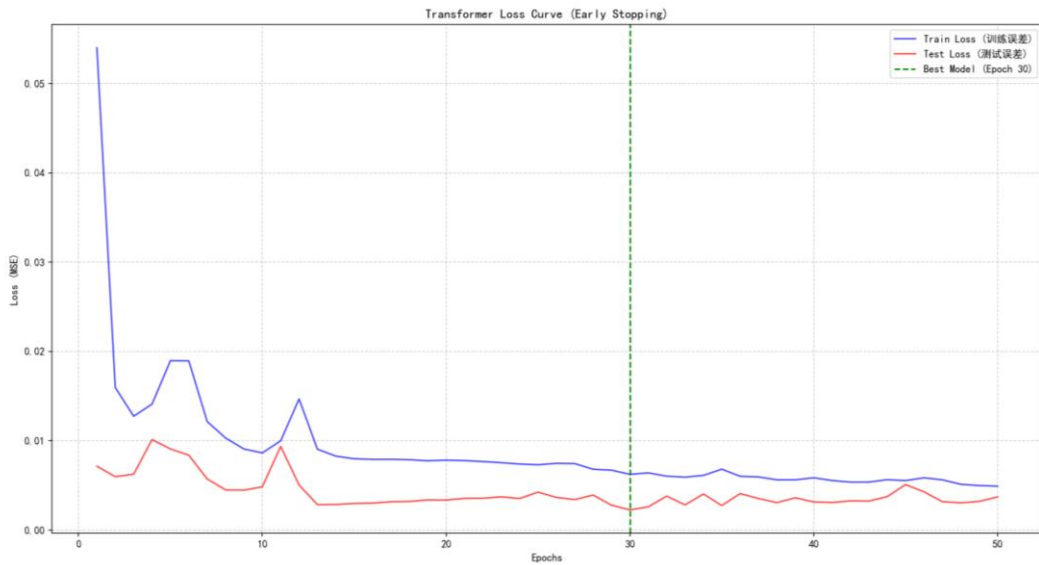


3.2 训练动态与过拟合控制分析

除了最终得分，模型在训练过程中的收敛稳定性同样决定了其工程落地价值。得益于本研究引入的早停机制（Early Stopping），各深度学习模型均成功在陷入“死记硬背”之前完成了截胡。

以 Transformer 模型为例：

在训练初期的前 10 轮，训练误差（Train Loss）与测试误差（Test Loss）均呈现陡峭下降趋势，说明模型正在快速学习风量、压力与负荷的物理映射。在训练进行至第 30 轮左右时，测试集误差到达全局最低谷。随后，尽管训练集误差仍在微弱下降，但测试集误差开始反弹（典型的过拟合前兆）。早停系统敏锐地捕捉到了这一拐点并及时切断了训练，保存了泛化能力最强的“巅峰大脑”。



3.3 物理规律的可解释性挖掘 (XGBoost)

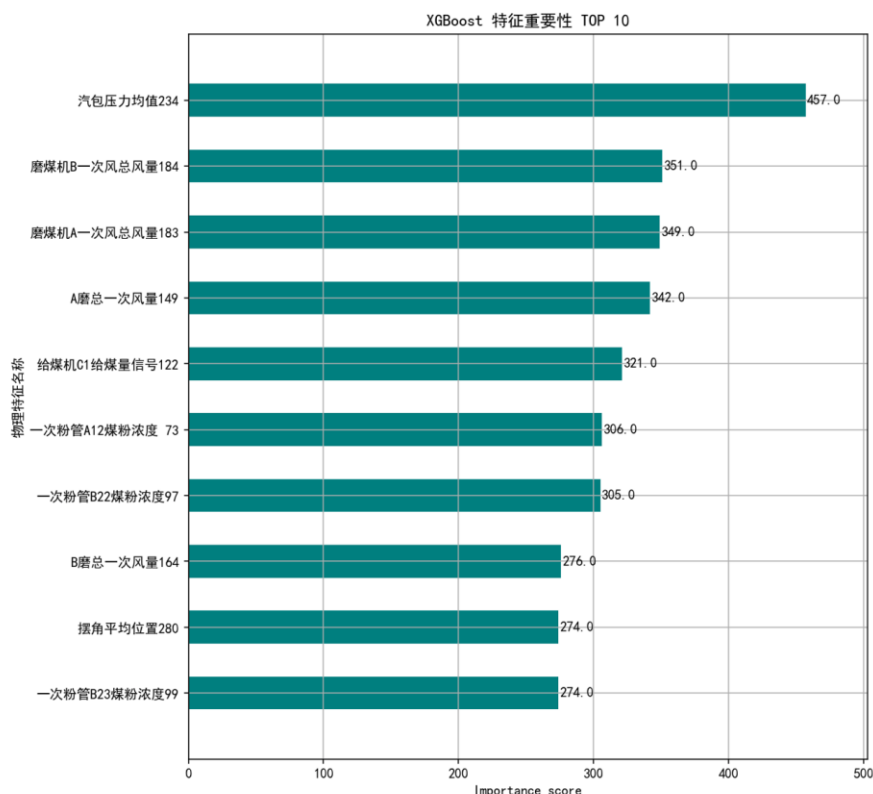
在保证预测精度的同时，工业界往往需要模型具备可解释性。本研究提取了 XGBoost 内部的树分裂特征增益(Feature Importance),从而找出了对未来 15 分钟负荷预测起决定性作用的核心物理量。

实验中所使用核心特征与皮尔森相关系数如下表所示。

传感器名称	皮尔森相关系数	传感器名称	皮尔森相关系数
A 磨总一次风量 149	0.854	总二次风量 200	0.9621
B 磨总一次风量 164	0.8257	总风量 198	0.9727
C 磨总一次风量 175	0.7765	摆角平均位置 280	0.8707
一次粉管 A11 风速 30	0.7658	摆角调节上限 284	0.8649
一次粉管 A12 煤粉浓度 73	0.7998	摆角调节下限 283	0.8649
一次粉管 A12 风速 31	0.7709	末级再热器入口烟气温 A 358	0.7405
一次粉管 A13 风速 32	0.7639	末级再热器入口烟气温 B 359	0.7405
一次粉管 A14 风速 33	0.7769	末级再热器横向第 2 排的 #1 管管壁温度 344	0.604

一次粉管 A21 风速 34	0.8214	末级再热器横向第 67 排的 #1 管管壁 温度 357	0.8039
一次粉管 A22 风速 35	0.8202	汽包压力均值 234	0.9451
一次粉管 A23 风速 36	0.8043	汽包水位 1239	0.6067
一次粉管 A24 风速 37	0.8199	磨煤机 A 一次风总 风量 183	0.6978
一次粉管 B11 风速 38	0.6094	磨煤机 B 一次风总 风量 184	0.8127
一次粉管 B14 风速 41	0.6136	给煤机 B1 给煤量信 号 120	0.7224
一次粉管 B21 煤粉 浓度 95	0.6449	给煤机 B1 给煤量信 号 121	0.7237
一次粉管 B22 煤粉 浓度 2 96	0.6997	给煤机 B2 给煤量信 号 126	0.6424
一次粉管 B22 煤粉 浓度 97	0.726	给煤机 B2 给煤量信 号 127	0.6462
一次粉管 B23 煤粉 浓度 99	0.7234	给煤机 C1 给煤量信 号 122	0.7442
一次粉管 B24 煤粉 浓度 101	0.6394	给煤机 C1 给煤量信 号 123	0.7372
一次粉管 C11 煤粉 浓度 103	0.78	送风机 A 出口风量 1 177	0.7201
一次粉管 C12 煤粉 浓度 105	0.7457	送风机 A 出口风量 2 179	0.7177
一次粉管 C13 煤粉 浓度 107	0.7794	送风机 A 出口风量 3 181	0.6914
一次粉管 C14 煤粉 浓度 109	0.7922	送风机 B 出口风量 1 178	0.7068
屏式再热器横向第 23 片屏排 #1 管管 壁温度 338	0.6505	送风机 B 出口风量 2 180	0.721

这一结论与火电厂锅炉燃烧系统的真实热力学迟滞规律高度吻合,证明模型并没有通过计算漏洞“作弊”,而是真正学到了底层的物理机制。



3.4 多挡位时间跨度预测与衰减分析

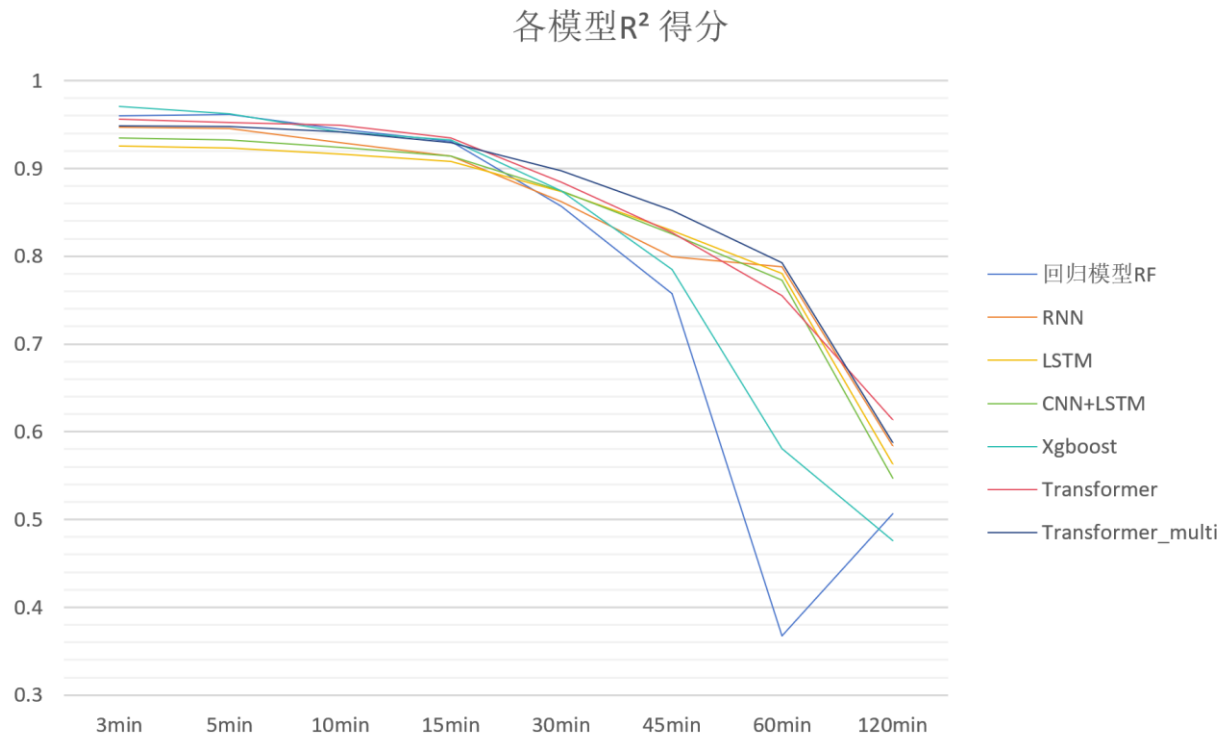
为了满足数字孪生大屏绘制平滑未来趋势线的工程需求，本研究通过“多输出 Transformer”架构，实现了一次推理并发输出 6 个时间挡位（3m, 15m, 30m, 60m, 120m, 240m）的负荷预测。

预测未来跨 度	决 定 系 数 (R ²)	平 均 绝 对 误 差 (MAE)	物理规律解释
+ 3 分钟	0.9485	7.18 MW	极短期：惯性极强，极度精准
+ 5 分钟	0.9479	7.42 MW	极短期：惯性极强，极度精准
+ 10 分钟	0.9416	7.25 MW	极短期：惯性极强，极度精准
+ 15 分钟	0.9295	8.23 MW	短期：主力预测区间，表现优异
+ 30 分钟	0.8973	9.31 MW	中期：受不可控波动影响，精度微降
+ 45 分钟	0.8520	10.84 MW	中长期：系统热力学延迟的极限边缘
+ 60 分钟	0.7928	12.12 MW	长期：不确定性大幅增加
+ 120 分钟	0.5878	17.31 MW	极长期：参考价值减弱

3.4.2 衰减效应分析：

结果显示，随着预测时间跨度的拉长，模型的 R^2 得分呈现出符合逻辑的衰减曲线。在 3 分钟到 30 分钟的极短期与短期区间内，模型能够依靠燃烧系统的物理惯性（如当前给煤量转化为蒸汽的固定时间延迟）做出高度精准的判断。然而，当预测时间拉长至 1 到 2 小时后，由于未来电网调度指令的不可预知性以及锅炉内部复杂热力状态的混沌演变，预测误差显著放大。

这一实验不仅验证了多输出工程架构的高效性，也从侧面探明了基于当前传感器历史数据所能预测的“有效时间视界（Time Horizon）”边界。



第4章 结论与工程落地展望

4.1 核心结论：算法选型与性能总结

本研究以火电厂真实运行数据为底座，构建了一套从底层数据清洗到顶层算法选型的完整时间序列预测流水线。通过引入严格的“物理防作弊”特征工程与早停（Early Stopping）泛化监控机制，我们在统一的基准线上对比了传统集成树模型（Random Forest, XGBoost）与前沿深度学习架构（RNN, LSTM, CNN-LSTM, Transformer）的预测性能。

实验结果表明，在复杂的工业多变量时序场景中，传统的循环神经网络易受局部高频噪声干扰，且存在长序列记忆遗忘的瓶颈。引入卷积特征提取（CNN-LSTM）能有效改善这一问题，而基于自注意力机制的 Transformer 模型则在捕获全局物理依赖方面展现出了绝对的统治力，其 R^2 预测精度在所有单一挡位测试中均名列前茅。此外，XGBoost 凭借极其优异的计算效率和对特征重要性的强大解释能力，成为了树模型流派中的最佳基准。

4.2 最终模型确立：面向数字孪生的“混合路由”双引擎架构

在最终的工程落地选型上，本研究并未采取“唯分数论”的单一模型堆叠策略，而是深度考量了时空复杂度（Time/Space Trade-off）、前端大屏的渲染需求以及后端微服务（Microservices）的并发压力。最终，本研究确立了“宏观趋势大一统 + 微观精准路由”的双引擎部署架构：

4.3 宏观趋势渲染引擎

多输出 Transformer 针对前端 Vue3 数据大屏需要在页面初始化时，瞬间绘制出包含未来多挡位（3m, 15m, 30m, 60m 等）平滑负荷趋势折线的核心需求，本系统将重构后的多输出 Transformer (Multi-Output Transformer) 作为主力接口。该架构能够在单次前向推理（ $O(1)$ 复杂度）中并发吐出所有时间节点的预测向量。这不仅大幅降低了服务器内存开销与 HTTP 请求的网络延迟（实现毫秒级响应），更使得模型从底层隐式学习到了负荷随时间衰减的物理平滑性，完美契合了全栈工程的高并发要求。

4.4 微观精准路由引擎

XGBoost 动态分发网关 考虑到深度学习的“黑盒”特性以及单一挡位的极致精度要求，系统同时设计了时间分段路由引擎（Horizon Router）。当业务人员在前端发起针对某一特定时刻（如“查询未来 60 分钟负荷”）的精准预测请求时，后端 Spring Boot 网关会触发路由机制，精准唤醒专门为该挡位特训的巅峰模型。这一机制不仅突破了单一模型在特定时间点的物理预测极限，更能实时输出清晰的特征重要性（Feature Importance）排序，为现场锅炉专工的调节操作（如优先调整风量或给煤量）提供具备严谨物理意义的诊断与指导。